Quiz review

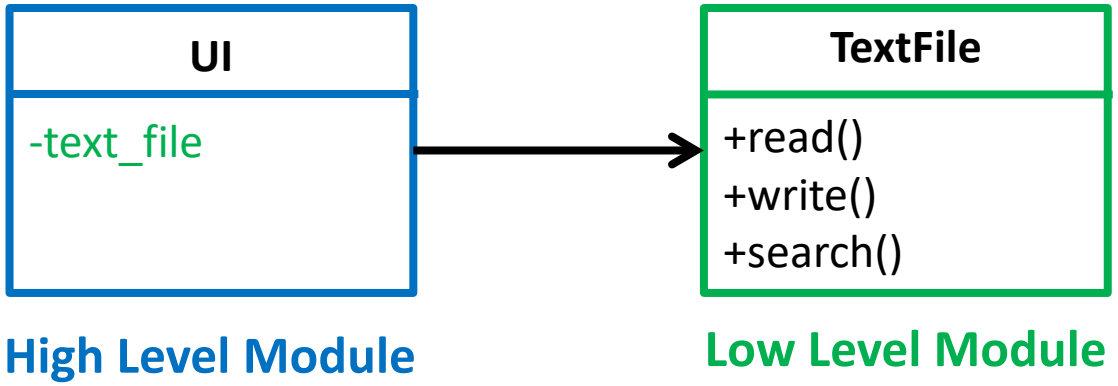
WEEK 14

Quiz review

Which of these is FALSE regarding the Dependency Inversion Principle.

- ◯ Low-level modules should not depend on high-level modules.
- ◯ Abstractions depend on details
- ◯ High-level modules should not depend on low-level modules.
- ◯ High-level and Low-level modules should depend on abstractions.

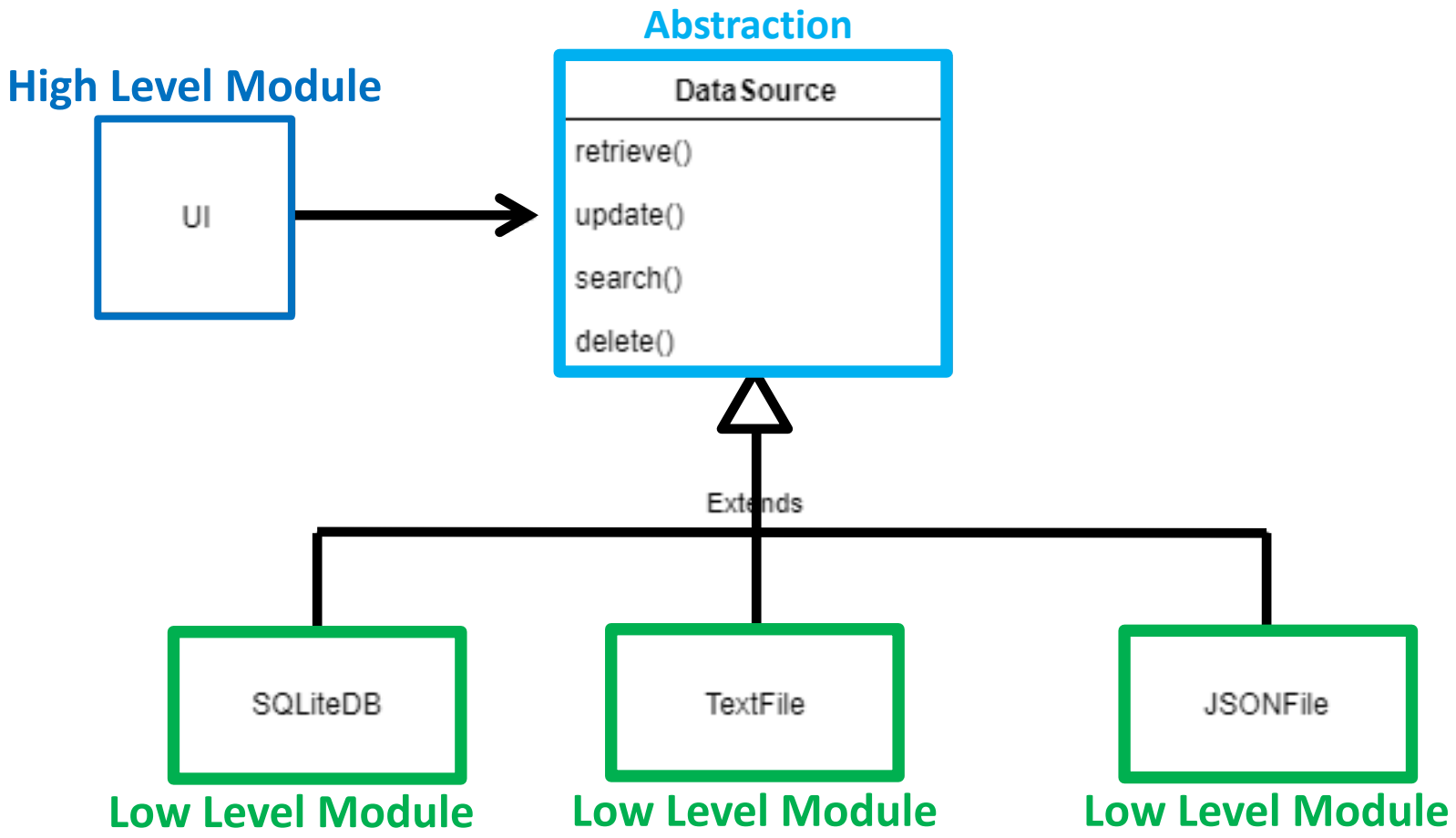
High Level & Low Level Modules



<u>This is not very good design.</u>

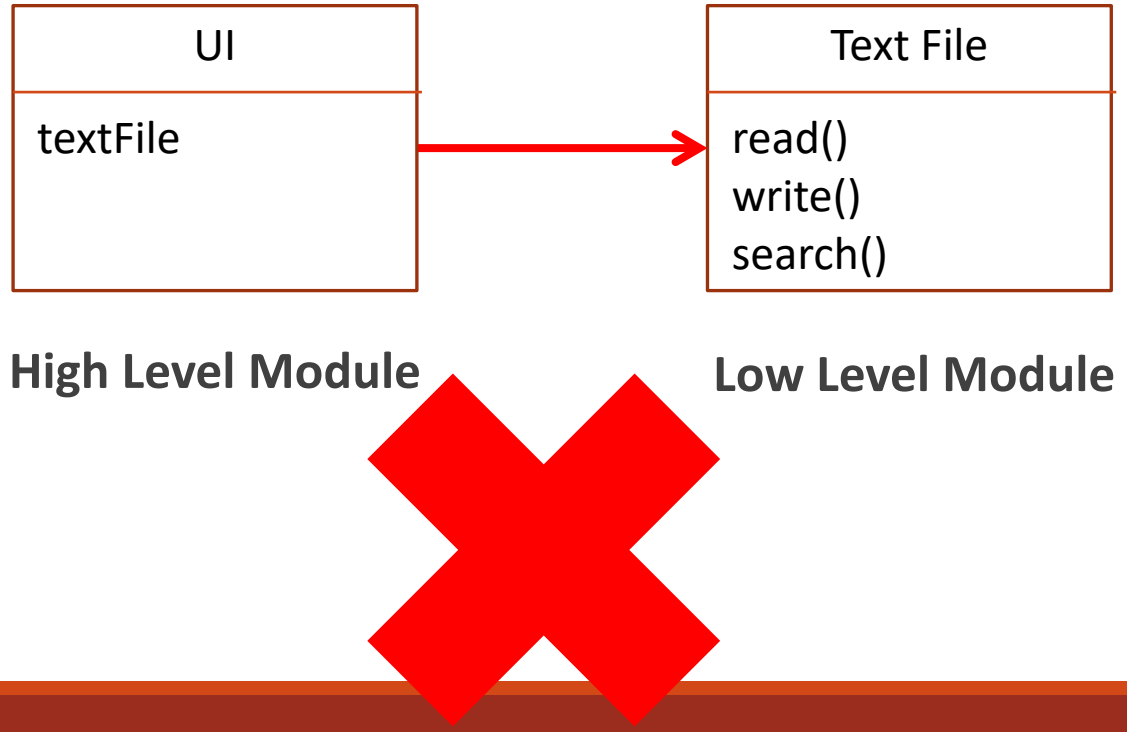
We don't want this dependency. We want to be able to change or replace the lower level modules without tampering with the complex code in the higher level modules

Dependency inversion principle

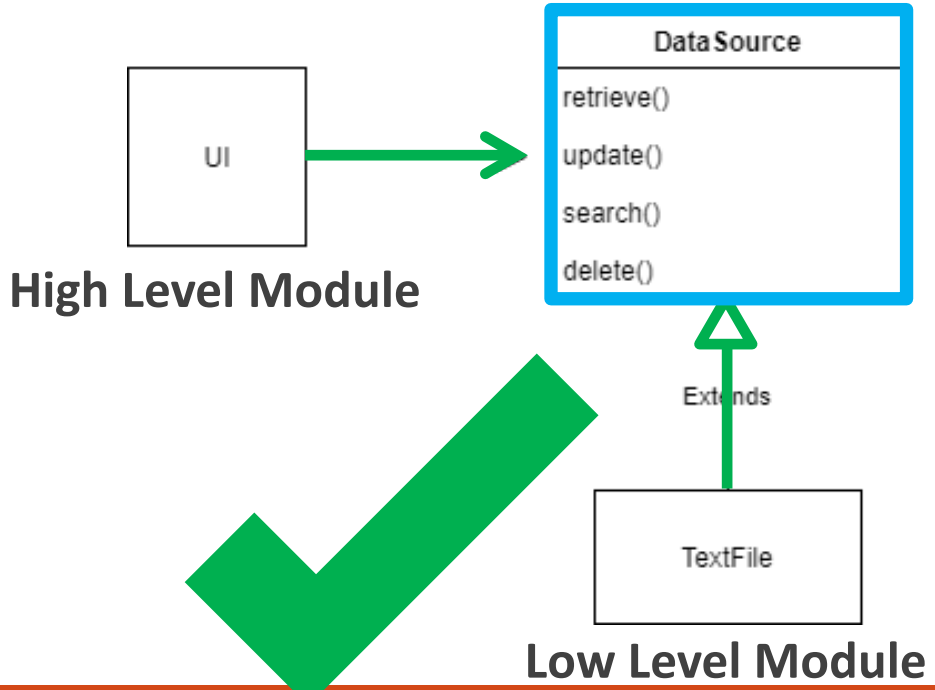


Dependency Inversion Principle

1. **High-level modules should not depend on low-level modules.**

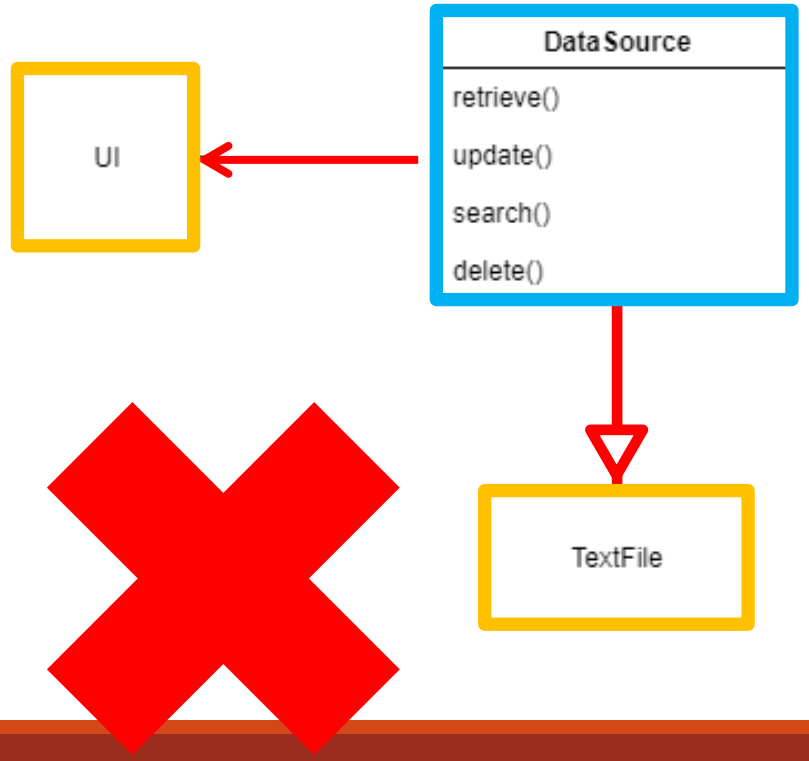


High-level modules and **low-level modules should depend on abstractions.**

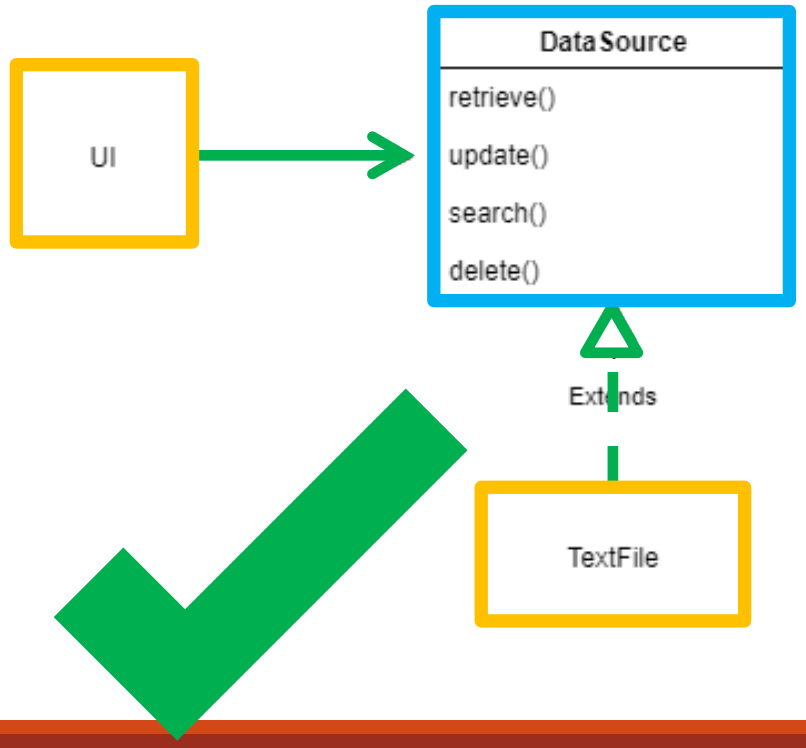


Dependency Inversion Principle

2. **Abstractions should not depend on details**



Details should depend on abstractions



Quiz review

Which of these is FALSE regarding the Dependency Inversion Principle.

- ◯ Low-level modules should not depend on high-level modules.
- ◯ Abstractions depend on details
- ◯ High-level modules should not depend on low-level modules.
- ◯ High-level and Low-level modules should depend on abstractions.

Quiz review

Which SOLID Design principle would you apply to avoid scenarios like this?

```
def take_damage(self):
  if self._type == "Troll":
    print("Dodged!")
  elif self._type == "Evil Turtle":
    print("Blocked!")
  elif self._type == "Human Underling":
    print(f"{self.name} took damage!")
  else
    print("Missed!")
```

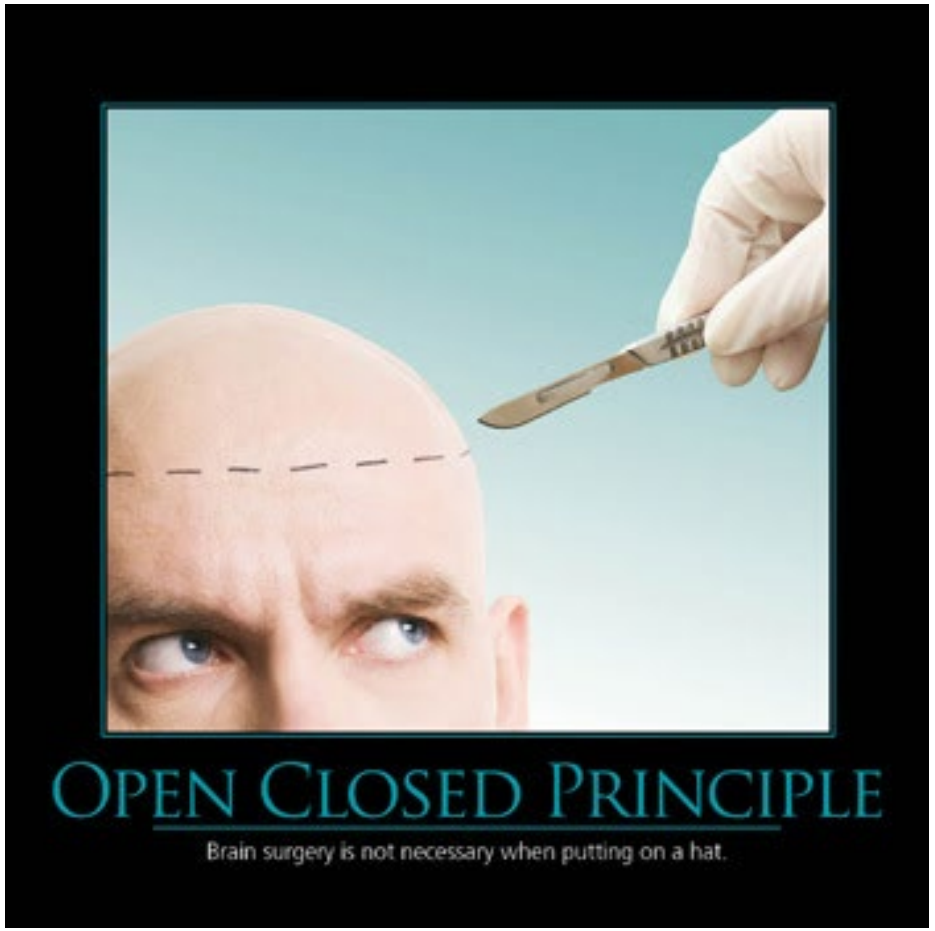
- ◯ Object Inversion Principle
- ◯ Open-Closed Principle
- ◯ Law of Demeter

Open Closed Principle

The Open/Closed Principle states that software entities (classes, modules, functions, etc.) should be open for extension, but closed for modification.

Once a class is made, it's behaviour shouldn't be modified to support special cases. It should be **CLOSED for modification**.

Instead, we extend (or inherit) from the class and override behaviours. That is, it's **OPEN for extension**.



Now over time, what if our UIWidget class looked like this?

```
class UIWidget:
    def draw(self):
        if self._type == "Button":
            #complicated button draw logic here
        elif self._type == “TextBox”:
            # complicated draw logic for a generic UI box
        elif self._type == “ScrollBar”:
            # complicated draw logic for a generic UI box
        elif self._type == “Label”:
            # complicated draw logic for a generic UI box
        else:
            # complicated draw logic for a generic UI box
```

UIWidget
- type : string
+draw()

This is **terrible design**! If we wanted to change the logic / algorithm behind drawing a button, we would need to edit this massive draw method.

An error or bug here would affect all the widgets on screen and crash our whole system!

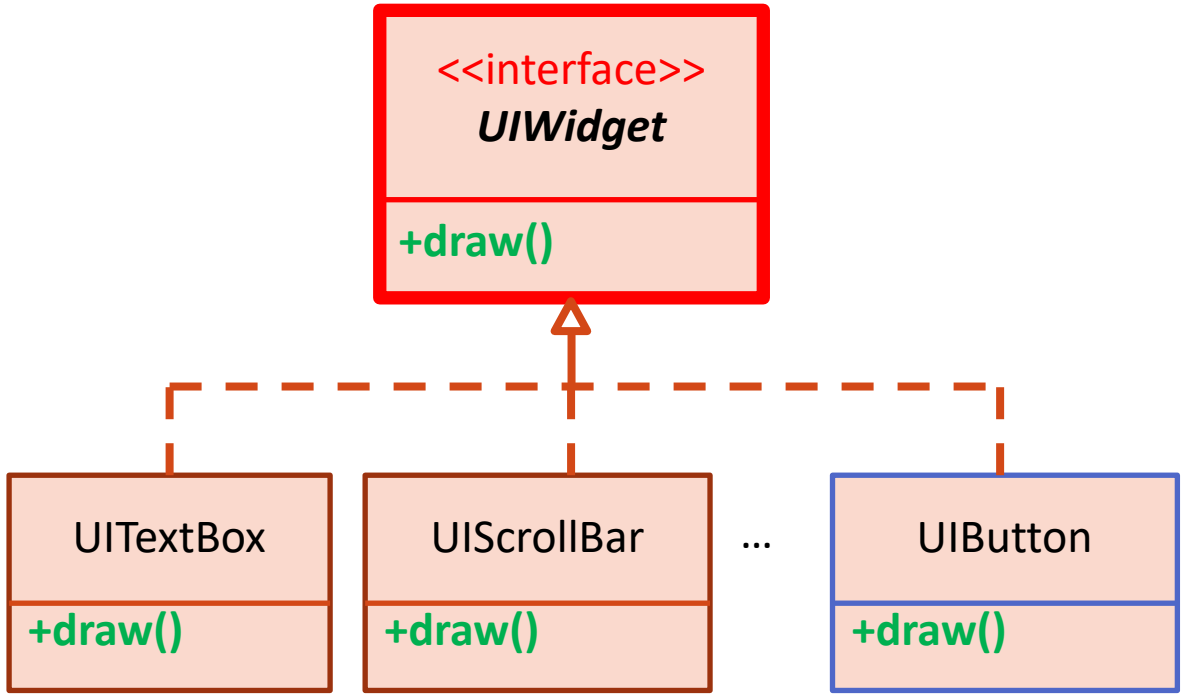
We use Inheritance (and overriding)!

```
class UIWidget(abc.ABC):
    @abc.abstractmethod
    def draw(self):
        pass

class UIButton(UIWidget):
    def draw(self):
        # complicated draw code for text box goes here

class UITextBox(UIWidget):
    def draw(self):
        # complicated draw code for text box goes here

class UIScrollBar(UIWidget):
    def draw(self):
        # complicated draw code for scroll bar goes here
```



We used abstraction to do this.

The **UIWidget class** is now an **interface** ensuring that each UI component that inherits from it has the **same interface**.

Law of Demeter

"Each unit should have only limited knowledge about other units: only units "closely" related to the current unit. **Each unit should only talk to its friends**; don't talk to strangers."

Don't have train wrecks like these:

```
obj.getX().getY().getZ().doSomething();
```

Or

```
MyCharacterUI.GetCharacterData().CreateCharacter().Save()
```

Poor Demeter is often forgotten, some people even call it "Suggestion of Demeter" since this is difficult to avoid

Law of Demeter

Real life - Wallet example

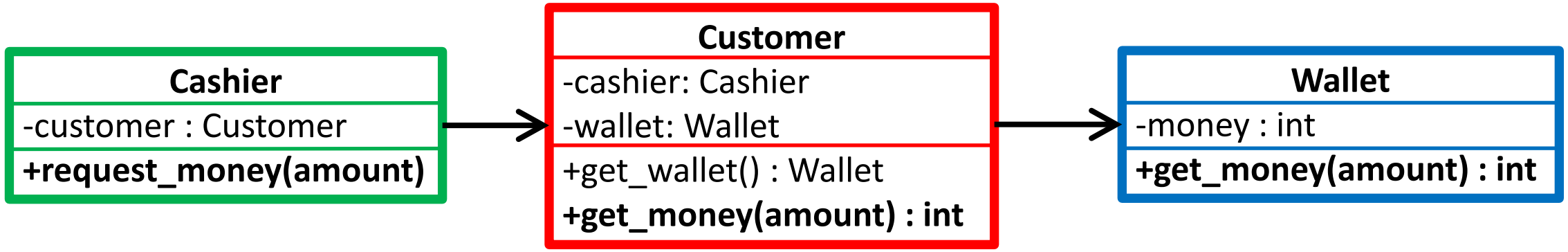
Law of Demeter

- Imagine if **Cashier**, **Customer**, and **Wallet** are classes

Goal: Cashier wants to get **int money** from the **Wallet**

- **Cashier** *should not* reach in **Customer**, then reach into their **Wallet** to get the money.
  - In **Cashier** **take_money()**: `money = self._customer.get_wallet().get_money(amount)` #BAD
- Classes **should not** have knowledge of public members several classes away

Law of Demeter



Goal: Cashier wants to get **int money** from the Wallet

- Cashier can access Customer, but should not access Wallet through Customer
- Refactor code so Cashier requests money from Customer. Customer accesses own Wallet to give money
  - In Cashier **request_money()**: `money = self._customer.get_money(amount) #OK!`
  - In Customer **get_money()**: `return self._wallet.get_money(amount) #OK!`
- Classes **should** have knowledge of public members of **neighboring classes**

Object inversion principle???

That was made up…it doesn’t exist

Quiz review

Which SOLID Design principle would you apply to avoid scenarios like this?

```
def take_damage(self):
  if self._type == "Troll":
    print("Dodged!")
  elif self._type == "Evil Turtle":
    print("Blocked!")
  elif self._type == "Human Underling":
    print(f"{self.name} took damage!")
  else
    print("Missed!")
```

- ◯ Object Inversion Principle
- ◯ Open-Closed Principle
- ◯ Law of Demeter

Given the following profiling snapshot, what are the number of recursive calls in **auction_entities.py:100(__call__)**

```
         336 function calls (238 primitive calls) in 0.000 seconds

   Ordered by: call count

   ncalls  tottime  percall  cumtime  percall filename:lineno(function)
       76    0.000    0.000    0.000    0.000 {method 'random' of '_random.Random' objects}
     70/4    0.000    0.000    0.000    0.000 auction_entities.py:100(__call__)
       53    0.000    0.000    0.000    0.000 auction_entities.py:66(current_bid)
       32    0.000    0.000    0.000    0.000 auction_entities.py:115(__str__)
       27    0.000    0.000    0.000    0.000 {built-in method builtins.print}
     17/1    0.000    0.000    0.000    0.000 auction_entities.py:57(_notify_bidders)
     17/1    0.000    0.000    0.000    0.000 auction_entities.py:74(accept_bid)
       17    0.000    0.000    0.000    0.000 auction_entities.py:70(highest_bidder)
       10    0.000    0.000    0.000    0.000 {method 'append' of 'list' objects}
```

- 53
- 70
- 66
- 4

cProfile results

Q: What's all this?

A: Stats from executing your program

Q: How are we supposed to read this?

A: I'll show you

Q: Can't we use PyCharm for this?

A: Yes can be done in command line or PyCharm

```
>>> import cProfile
>>> import re
>>> cProfile.run('re.compile("foo|bar")')
         214 function calls (207 primitive calls) in 0.000 seconds

   Ordered by: standard name

   ncalls  tottime  percall  cumtime  percall filename:lineno(function)
        1    0.000    0.000    0.000    0.000 <string>:1(<module>)
        2    0.000    0.000    0.000    0.000 enum.py:284(__call__)
        2    0.000    0.000    0.000    0.000 enum.py:526(__new__)
        1    0.000    0.000    0.000    0.000 enum.py:836(__and__)
        1    0.000    0.000    0.000    0.000 re.py:232(compile)
        1    0.000    0.000    0.000    0.000 re.py:271(_compile)
        1    0.000    0.000    0.000    0.000 sre_compile.py:249(_compile_charset)
        1    0.000    0.000    0.000    0.000 sre_compile.py:276(_optimize_charset)
        2    0.000    0.000    0.000    0.000 sre_compile.py:453(_get_iscased)
        1    0.000    0.000    0.000    0.000 sre_compile.py:461(_get_literal_prefix)
        1    0.000    0.000    0.000    0.000 sre_compile.py:492(_get_charset_prefix)
        1    0.000    0.000    0.000    0.000 sre_compile.py:536(_compile_info)
        2    0.000    0.000    0.000    0.000 sre_compile.py:595(isstring)
        1    0.000    0.000    0.000    0.000 sre_compile.py:598(_code)
      3/1    0.000    0.000    0.000    0.000 sre_compile.py:71(_compile)
        1    0.000    0.000    0.000    0.000 sre_compile.py:759(compile)
        3    0.000    0.000    0.000    0.000 sre_parse.py:111(__init__)
        7    0.000    0.000    0.000    0.000 sre_parse.py:160(__len__)
       18    0.000    0.000    0.000    0.000 sre_parse.py:164(__getitem__)
        7    0.000    0.000    0.000    0.000 sre_parse.py:172(append)
      3/1    0.000    0.000    0.000    0.000 sre_parse.py:174(getwidth)
        1    0.000    0.000    0.000    0.000 sre_parse.py:224(__init__)
        8    0.000    0.000    0.000    0.000 sre_parse.py:233(__next)
        2    0.000    0.000    0.000    0.000 sre_parse.py:249(match)
        6    0.000    0.000    0.000    0.000 sre_parse.py:254(get)
        1    0.000    0.000    0.000    0.000 sre_parse.py:286(tell)
        1    0.000    0.000    0.000    0.000 sre_parse.py:417(_parse_sub)
        2    0.000    0.000    0.000    0.000 sre_parse.py:475(_parse)
        1    0.000    0.000    0.000    0.000 sre_parse.py:76(__init__)
        2    0.000    0.000    0.000    0.000 sre_parse.py:81(groups)
        1    0.000    0.000    0.000    0.000 sre_parse.py:903(fix_flags)
        1    0.000    0.000    0.000    0.000 sre_parse.py:919(parse)
        1    0.000    0.000    0.000    0.000 {built-in method _sre.compile}
        1    0.000    0.000    0.000    0.000 {built-in method builtins.exec}
       25    0.000    0.000    0.000    0.000 {built-in method builtins.isinstance}
    29/26    0.000    0.000    0.000    0.000 {built-in method builtins.len}
        2    0.000    0.000    0.000    0.000 {built-in method builtins.max}
        9    0.000    0.000    0.000    0.000 {built-in method builtins.min}
        6    0.000    0.000    0.000    0.000 {built-in method builtins.ord}
       48    0.000    0.000    0.000    0.000 {method 'append' of 'list' objects}
        1    0.000    0.000    0.000    0.000 {method 'disable' of '_lsprof.Profiler' objects}
        5    0.000    0.000    0.000    0.000 {method 'find' of 'bytearray' objects}
        1    0.000    0.000    0.000    0.000 {method 'items' of 'dict' objects}

>>>
```

cProfile results

The first line indicates that **214 function calls were monitored**

Of those calls, **207 were *primitive***

Primitive calls are non recursive calls

```
>>> import cProfile
>>> import re
>>> cProfile.run('re.compile("foo|bar")')
         214 function calls (207 primitive calls) in 0.000 seconds

   Ordered by: standard name

   ncalls  tottime  percall  cumtime  percall filename:lineno(function)
        1    0.000    0.000    0.000    0.000 <string>:1(<module>)
        2    0.000    0.000    0.000    0.000 enum.py:284(__call__)
        2    0.000    0.000    0.000    0.000 enum.py:526(__new__)
        1    0.000    0.000    0.000    0.000 enum.py:836(__and__)
        1    0.000    0.000    0.000    0.000 re.py:232(compile)
        1    0.000    0.000    0.000    0.000 re.py:271(_compile)
        1    0.000    0.000    0.000    0.000 sre_compile.py:249(_compile_charset)
        1    0.000    0.000    0.000    0.000 sre_compile.py:276(_optimize_charset)
        2    0.000    0.000    0.000    0.000 sre_compile.py:453(_get_iscased)
        1    0.000    0.000    0.000    0.000 sre_compile.py:461(_get_literal_prefix)
        1    0.000    0.000    0.000    0.000 sre_compile.py:492(_get_charset_prefix)
        1    0.000    0.000    0.000    0.000 sre_compile.py:536(_compile_info)
        2    0.000    0.000    0.000    0.000 sre_compile.py:595(isstring)
        1    0.000    0.000    0.000    0.000 sre_compile.py:598(_code)
      3/1    0.000    0.000    0.000    0.000 sre_compile.py:71(_compile)
        1    0.000    0.000    0.000    0.000 sre_compile.py:759(compile)
        3    0.000    0.000    0.000    0.000 sre_parse.py:111(__init__)
        7    0.000    0.000    0.000    0.000 sre_parse.py:160(__len__)
       18    0.000    0.000    0.000    0.000 sre_parse.py:164(__getitem__)
        7    0.000    0.000    0.000    0.000 sre_parse.py:172(append)
      3/1    0.000    0.000    0.000    0.000 sre_parse.py:174(getwidth)
        1    0.000    0.000    0.000    0.000 sre_parse.py:224(__init__)
        8    0.000    0.000    0.000    0.000 sre_parse.py:233(__next)
        2    0.000    0.000    0.000    0.000 sre_parse.py:249(match)
        6    0.000    0.000    0.000    0.000 sre_parse.py:254(get)
        1    0.000    0.000    0.000    0.000 sre_parse.py:286(tell)
        1    0.000    0.000    0.000    0.000 sre_parse.py:417(_parse_sub)
        2    0.000    0.000    0.000    0.000 sre_parse.py:475(_parse)
        1    0.000    0.000    0.000    0.000 sre_parse.py:76(__init__)
        2    0.000    0.000    0.000    0.000 sre_parse.py:81(groups)
        1    0.000    0.000    0.000    0.000 sre_parse.py:903(fix_flags)
        1    0.000    0.000    0.000    0.000 sre_parse.py:919(parse)
```


```
>> import re
>> cProfile.run('re.compile("foo|bar")')
         214 function calls (207 primitive calls) in 0.000 seconds

   Ordered by: standard name
```

cProfile results

The next line tells us that the text string in the **far-right column** was used to **sort the output**

In this case the string is `filename:lineno(function)`

```
214 function calls (2

Ordered by: standard name
```

```
>>> import cProfile
>>> import re
>>> cProfile.run('re.compile("foo|bar")')
         214 function calls (207 primitive calls) in 0.000 seconds

   Ordered by: standard name

   ncalls  tottime  percall  cumtime  percall filename:lineno(function)
        1    0.000    0.000    0.000    0.000 <string>:1(<module>)
        2    0.000    0.000    0.000    0.000 enum.py:284(__call__)
        2    0.000    0.000    0.000    0.000 enum.py:526(__new__)
        1    0.000    0.000    0.000    0.000 enum.py:836(__and__)
        1    0.000    0.000    0.000    0.000 re.py:232(compile)
        1    0.000    0.000    0.000    0.000 re.py:271(_compile)
        1    0.000    0.000    0.000    0.000 sre_compile.py:249(_compile_charset)
        1    0.000    0.000    0.000    0.000 sre_compile.py:276(_optimize_charset)
        [illegible]    0.000    0.000    0.000    0.000 sre_compile.py:453(_get_iscased)
        1    0.000    0.000    0.000    0.000 sre_compile.py:461(_get_literal_prefix)
        1    0.000    0.000    0.000    0.000 sre_compile.py:492(_get_charset_prefix)
        1    0.000    0.000    0.000    0.000 sre_compile.py:536(_compile_info)
        2    0.000    0.000    0.000    0.000 sre_compile.py:595(isstring)
        1    0.000    0.000    0.000    0.000 sre_compile.py:598(_code)
      3/1    0.000    0.000    0.000    0.000 sre_compile.py:71(_compile)
        1    0.000    0.000    0.000    0.000 sre_compile.py:759(compile)
        3    0.000    0.000    0.000    0.000 sre_parse.py:111(__init__)
        7    0.000    0.000    0.000    0.000 sre_parse.py:160(__len__)
       18    0.000    0.000    0.000    0.000 sre_parse.py:164(__getitem__)
        7    0.000    0.000    0.000    0.000 sre_parse.py:172(append)
      3/1    0.000    0.000    0.000    0.000 sre_parse.py:174(getwidth)
        1    0.000    0.000    0.000    0.000 sre_parse.py:224(__init__)
        8    0.000    0.000    0.000    0.000 sre_parse.py:233(__next)
        2    0.000    0.000    0.000    0.000 sre_parse.py:249(match)
        6    0.000    0.000    0.000    0.000 sre_parse.py:254(get)
        1    0.000    0.000    0.000    0.000 sre_parse.py:286(tell)
        1    0.000    0.000    0.000    0.000 sre_parse.py:417(_parse_sub)
        2    0.000    0.000    0.000    0.000 sre_parse.py:475(_parse)
        1    0.000    0.000    0.000    0.000 sre_parse.py:76(__init__)
        2    0.000    0.000    0.000    0.000 sre_parse.py:81(groups)
        1    0.000    0.000    0.000    0.000 sre_parse.py:903(fix_flags)
        1    0.000    0.000    0.000    0.000 sre_parse.py:919(parse)
        1    0.000    0.000    0.000    0.000 {built-in method _sre.compile}
        1    0.000    0.000    0.000    0.000 {built-in method builtins.exec}
       25    0.000    0.000    0.000    0.000 {built-in method builtins.isinstance}
    29/26    0.000    0.000    0.000    0.000 {built-in method builtins.len}
        2    0.000    0.000    0.000    0.000 {built-in method builtins.max}
        9    0.000    0.000    0.000    0.000 {built-in method builtins.min}
        6    0.000    0.000    0.000    0.000 {built-in method builtins.ord}
       48    0.000    0.000    0.000    0.000 {method 'append' of 'list' objects}
        1    0.000    0.000    0.000    0.000 {method 'disable' of '_lsprof.Profiler' objects}
        5    0.000    0.000    0.000    0.000 {method 'find' of 'bytearray' objects}
        1    0.000    0.000    0.000    0.000 {method 'items' of 'dict' objects}

>>>
```

cProfile results

The first column heading is **ncalls**

This is (wait for it…) the **number of times the function was called**

```
>>> import cProfile
>>> import re
>>> cProfile.run('re.compile("foo|bar")')
         214 function calls (207 primitive calls) in 0.000 seconds

   Ordered by: standard name

   ncalls  tottime  percall  cumtime  percall filename:lineno(function)
        1    0.000    0.000    0.000    0.000 <string>:1(<module>)
        2    0.000    0.000    0.000    0.000 enum.py:284(__call__)
        2    0.000    0.000    0.000    0.000 enum.py:526(__new__)
        1    0.000    0.000    0.000    0.000 enum.py:836(__and__)
        1    0.000    0.000    0.000    0.000 re.py:232(compile)
        1    0.000    0.000    0.000    0.000 re.py:271(_compile)
        1    0.000    0.000    0.000    0.000 sre_compile.py:249(_compile_charset)
        1    0.000    0.000    0.000    0.000 sre_compile.py:276(_optimize_charset)
        2    0.000    0.000    0.000    0.000 sre_compile.py:453(_get_iscased)
        1    0.000    0.000    0.000    0.000 sre_compile.py:461(_get_literal_prefix)
        1    0.000    0.000    0.000    0.000 sre_compile.py:492(_get_charset_prefix)
        1    0.000    0.000    0.000    0.000 sre_compile.py:536(_compile_info)
        2    0.000    0.000    0.000    0.000 sre_compile.py:595(isstring)
        1    0.000    0.000    0.000    0.000 sre_compile.py:598(_code)
      3/1    0.000    0.000    0.000    0.000 sre_compile.py:71(_compile)
        1    0.000    0.000    0.000    0.000 sre_compile.py:759(compile)
        3    0.000    0.000    0.000    0.000 sre_parse.py:111(__init__)
        7    0.000    0.000    0.000    0.000 sre_parse.py:160(__len__)
       18    0.000    0.000    0.000    0.000 sre_parse.py:164(__getitem__)
        7    0.000    0.000    0.000    0.000 sre_parse.py:172(append)
      3/1    0.000    0.000    0.000    0.000 sre_parse.py:174(getwidth)
        1    0.000    0.000    0.000    0.000 sre_parse.py:224(__init__)
        8    0.000    0.000    0.000    0.000 sre_parse.py:233(__next)
        2    0.000    0.000    0.000    0.000 sre_parse.py:249(match)
        6    0.000    0.000    0.000    0.000 sre_parse.py:254(get)
        1    0.000    0.000    0.000    0.000 sre_parse.py:286(tell)
        1    0.000    0.000    0.000    0.000 sre_parse.py:417(_parse_sub)
        2    0.000    0.000    0.000    0.000 sre_parse.py:475(_parse)
        1    0.000    0.000    0.000    0.000 sre_parse.py:76(__init__)
        2    0.000    0.000    0.000    0.000 sre_parse.py:81(groups)
        1    0.000    0.000    0.000    0.000 sre_parse.py:903(fix_flags)
        1    0.000    0.000    0.000    0.000 sre_parse.py:919(parse)
        1    0.000    0.000    0.000    0.000 {built-in method _sre.compile}
        1    0.000    0.000    0.000    0.000 {built-in method builtins.exec}
       25    0.000    0.000    0.000    0.000 {built-in method builtins.isinstance}
    29/26    0.000    0.000    0.000    0.000 {built-in method builtins.len}
        2    0.000    0.000    0.000    0.000 {built-in method builtins.max}
        9    0.000    0.000    0.000    0.000 {built-in method builtins.min}
        6    0.000    0.000    0.000    0.000 {built-in method builtins.ord}
       48    0.000    0.000    0.000    0.000 {method 'append' of 'list' objects}
        1    0.000    0.000    0.000    0.000 {method 'disable' of '_lsprof.Profiler' objects}
        5    0.000    0.000    0.000    0.000 {method 'find' of 'bytearray' objects}
        1    0.000    0.000    0.000    0.000 {method 'items' of 'dict' objects}

>>>
```

ncalls t

cProfile results

The second column heading is **tottime**

This represents the **total time spent in the function**

It does not include time spent in calls to sub-functions

```
>>> import cProfile
>>> import re
>>> cProfile.run('re.compile("foo|bar")')
         214 function calls (207 primitive calls) in 0.000 seconds

   Ordered by: standard name

   ncalls  tottime  percall  cumtime  percall filename:lineno(function)
        1    0.000    0.000    0.000    0.000 <string>:1(<module>)
        2    0.000    0.000    0.000    0.000 enum.py:284(__call__)
        2    0.000    0.000    0.000    0.000 enum.py:526(__new__)
        1    0.000    0.000    0.000    0.000 enum.py:836(__and__)
        1    0.000    0.000    0.000    0.000 re.py:232(compile)
        1    0.000    0.000    0.000    0.000 re.py:271(_compile)
        1    0.000    0.000    0.000    0.000 sre_compile.py:249(_compile_charset)
        1    0.000    0.000    0.000    0.000 sre_compile.py:276(_optimize_charset)
        2    0.000    0.000    0.000    0.000 sre_compile.py:453(_get_iscased)
        1    0.000    0.000    0.000    0.000 sre_compile.py:461(_get_literal_prefix)
        1    0.000    0.000    0.000    0.000 sre_compile.py:492(_get_charset_prefix)
        1    0.000    0.000    0.000    0.000 sre_compile.py:536(_compile_info)
        2    0.000    0.000    0.000    0.000 sre_compile.py:595(isstring)
        1    0.000    0.000    0.000    0.000 sre_compile.py:598(_code)
      3/1    0.000    0.000    0.000    0.000 sre_compile.py:71(_compile)
        1    0.000    0.000    0.000    0.000 sre_compile.py:759(compile)
        3    0.000    0.000    0.000    0.000 sre_parse.py:111(__init__)
        7    0.000    0.000    0.000    0.000 sre_parse.py:160(__len__)
       18    0.000    0.000    0.000    0.000 sre_parse.py:164(__getitem__)
        7    0.000    0.000    0.000    0.000 sre_parse.py:172(append)
      3/1    0.000    0.000    0.000    0.000 sre_parse.py:174(getwidth)
        1    0.000    0.000    0.000    0.000 sre_parse.py:224(__init__)
        8    0.000    0.000    0.000    0.000 sre_parse.py:233(__next)
        2    0.000    0.000    0.000    0.000 sre_parse.py:249(match)
        6    0.000    0.000    0.000    0.000 sre_parse.py:254(get)
        1    0.000    0.000    0.000    0.000 sre_parse.py:286(tell)
        1    0.000    0.000    0.000    0.000 sre_parse.py:417(_parse_sub)
        2    0.000    0.000    0.000    0.000 sre_parse.py:475(_parse)
        1    0.000    0.000    0.000    0.000 sre_parse.py:76(__init__)
        2    0.000    0.000    0.000    0.000 sre_parse.py:81(groups)
        1    0.000    0.000    0.000    0.000 sre_parse.py:903(fix_flags)
        1    0.000    0.000    0.000    0.000 sre_parse.py:919(parse)
        1    0.000    0.000    0.000    0.000 {built-in method _sre.compile}
        1    0.000    0.000    0.000    0.000 {built-in method builtins.exec}
       25    0.000    0.000    0.000    0.000 {built-in method builtins.isinstance}
    29/26    0.000    0.000    0.000    0.000 {built-in method builtins.len}
        2    0.000    0.000    0.000    0.000 {built-in method builtins.max}
        9    0.000    0.000    0.000    0.000 {built-in method builtins.min}
        6    0.000    0.000    0.000    0.000 {built-in method builtins.ord}
       48    0.000    0.000    0.000    0.000 {method 'append' of 'list' objects}
        1    0.000    0.000    0.000    0.000 {method 'disable' of '_lsprof.Profiler' objects}
        5    0.000    0.000    0.000    0.000 {method 'find' of 'bytearray' objects}
        1    0.000    0.000    0.000    0.000 {method 'items' of 'dict' objects}

>>>
```

tottime
0.000

cProfile results

The third column heading is **percall**

percall = tottime / numcalls

We see that the **time per call** is negligible

(I expect library code to be highly optimized!)

```
>>> import cProfile
>>> import re
>>> cProfile.run('re.compile("foo|bar")')
         214 function calls (207 primitive calls) in 0.000 seconds

   Ordered by: standard name

   ncalls  tottime  percall  cumtime  percall filename:lineno(function)
        1    0.000    0.000    0.000    0.000 <string>:1(<module>)
        2    0.000    0.000    0.000    0.000 enum.py:284(__call__)
        2    0.000    0.000    0.000    0.000 enum.py:526(__new__)
        1    0.000    0.000    0.000    0.000 enum.py:836(__and__)
        1    0.000    0.000    0.000    0.000 re.py:232(compile)
        1    0.000    0.000    0.000    0.000 re.py:271(_compile)
        1    0.000    0.000    0.000    0.000 sre_compile.py:249(_compile_charset)
        1    0.000    0.000    0.000    0.000 sre_compile.py:276(_optimize_charset)
        2    0.000    0.000    0.000    0.000 sre_compile.py:453(_get_iscased)
        1    0.000    0.000    0.000    0.000 sre_compile.py:461(_get_literal_prefix)
        1    0.000    0.000    0.000    0.000 sre_compile.py:492(_get_charset_prefix)
        1    0.000    0.000    0.000    0.000 sre_compile.py:536(_compile_info)
        2    0.000    0.000    0.000    0.000 sre_compile.py:595(isstring)
        1    0.000    0.000    0.000    0.000 sre_compile.py:598(_code)
      3/1    0.000    0.000    0.000    0.000 sre_compile.py:71(_compile)
        1    0.000    0.000    0.000    0.000 sre_compile.py:759(compile)
        3    0.000    0.000    0.000    0.000 sre_parse.py:111(__init__)
        7    0.000    0.000    0.000    0.000 sre_parse.py:160(__len__)
       18    0.000    0.000    0.000    0.000 sre_parse.py:164(__getitem__)
        7    0.000    0.000    0.000    0.000 sre_parse.py:172(append)
      3/1    0.000    0.000    0.000    0.000 sre_parse.py:174(getwidth)
        1    0.000    0.000    0.000    0.000 sre_parse.py:224(__init__)
        8    0.000    0.000    0.000    0.000 sre_parse.py:233(__next)
        2    0.000    0.000    0.000    0.000 sre_parse.py:249(match)
        6    0.000    0.000    0.000    0.000 sre_parse.py:254(get)
        1    0.000    0.000    0.000    0.000 sre_parse.py:286(tell)
        1    0.000    0.000    0.000    0.000 sre_parse.py:417(_parse_sub)
        2    0.000    0.000    0.000    0.000 sre_parse.py:475(_parse)
        1    0.000    0.000    0.000    0.000 sre_parse.py:76(__init__)
        2    0.000    0.000    0.000    0.000 sre_parse.py:81(groups)
        1    0.000    0.000    0.000    0.000 sre_parse.py:903(fix_flags)
        1    0.000    0.000    0.000    0.000 sre_parse.py:919(parse)
        1    0.000    0.000    0.000    0.000 {built-in method _sre.compile}
        1    0.000    0.000    0.000    0.000 {built-in method builtins.exec}
       25    0.000    0.000    0.000    0.000 {built-in method builtins.isinstance}
   29/26    0.000    0.000    0.000    0.000 {built-in method builtins.len}
        2    0.000    0.000    0.000    0.000 {built-in method builtins.max}
        9    0.000    0.000    0.000    0.000 {built-in method builtins.min}
        6    0.000    0.000    0.000    0.000 {built-in method builtins.ord}
       48    0.000    0.000    0.000    0.000 {method 'append' of 'list' objects}
        1    0.000    0.000    0.000    0.000 {method 'disable' of '_lsprof.Profiler' objects}
        5    0.000    0.000    0.000    0.000 {method 'find' of 'bytearray' objects}
        1    0.000    0.000    0.000    0.000 {method 'items' of 'dict' objects}

>>>
```

percall

cProfile results

The fourth column heading is **cumtime** (cumulative time).

This is the **cumulative time spent in this and all subfunctions**

Counts from invocation till exit

This figure is accurate even for recursive functions.

cumtime p

```
>>> import cProfile
>>> import re
>>> cProfile.run('re.compile("foo|bar")')
         214 function calls (207 primitive calls) in 0.000 seconds

   Ordered by: standard name

   ncalls  tottime  percall  cumtime  percall filename:lineno(function)
        1    0.000    0.000    0.000    0.000 <string>:1(<module>)
        2    0.000    0.000    0.000    0.000 enum.py:284(__call__)
        2    0.000    0.000    0.000    0.000 enum.py:526(__new__)
        1    0.000    0.000    0.000    0.000 enum.py:836(__and__)
        1    0.000    0.000    0.000    0.000 re.py:232(compile)
        1    0.000    0.000    0.000    0.000 re.py:271(_compile)
        1    0.000    0.000    0.000    0.000 sre_compile.py:249(_compile_charset)
        1    0.000    0.000    0.000    0.000 sre_compile.py:276(_optimize_charset)
        2    0.000    0.000    0.000    0.000 sre_compile.py:453(_get_iscased)
        1    0.000    0.000    0.000    0.000 sre_compile.py:461(_get_literal_prefix)
        1    0.000    0.000    0.000    0.000 sre_compile.py:492(_get_charset_prefix)
        1    0.000    0.000    0.000    0.000 sre_compile.py:536(_compile_info)
        2    0.000    0.000    0.000    0.000 sre_compile.py:595(isstring)
        1    0.000    0.000    0.000    0.000 sre_compile.py:598(_code)
        1    0.000    0.000    0.000    0.000 sre_compile.py:71(_compile)
        1    0.000    0.000    0.000    0.000 sre_compile.py:759(compile)
        3    0.000    0.000    0.000    0.000 sre_parse.py:111(__init__)
        7    0.000    0.000    0.000    0.000 sre_parse.py:160(__len__)
       18    0.000    0.000    0.000    0.000 sre_parse.py:164(__getitem__)
        7    0.000    0.000    0.000    0.000 sre_parse.py:172(append)
      3/1    0.000    0.000    0.000    0.000 sre_parse.py:174(getwidth)
        1    0.000    0.000    0.000    0.000 sre_parse.py:224(__init__)
        8    0.000    0.000    0.000    0.000 sre_parse.py:233(__next)
        2    0.000    0.000    0.000    0.000 sre_parse.py:249(match)
        6    0.000    0.000    0.000    0.000 sre_parse.py:254(get)
        1    0.000    0.000    0.000    0.000 sre_parse.py:286(tell)
        1    0.000    0.000    0.000    0.000 sre_parse.py:417(_parse_sub)
        2    0.000    0.000    0.000    0.000 sre_parse.py:475(_parse)
        1    0.000    0.000    0.000    0.000 sre_parse.py:76(__init__)
        2    0.000    0.000    0.000    0.000 sre_parse.py:81(groups)
        1    0.000    0.000    0.000    0.000 sre_parse.py:903(fix_flags)
        1    0.000    0.000    0.000    0.000 sre_parse.py:919(parse)
        1    0.000    0.000    0.000    0.000 {built-in method _sre.compile}
        1    0.000    0.000    0.000    0.000 {built-in method builtins.exec}
       25    0.000    0.000    0.000    0.000 {built-in method builtins.isinstance}
    29/26    0.000    0.000    0.000    0.000 {built-in method builtins.len}
        2    0.000    0.000    0.000    0.000 {built-in method builtins.max}
        9    0.000    0.000    0.000    0.000 {built-in method builtins.min}
        6    0.000    0.000    0.000    0.000 {built-in method builtins.ord}
       48    0.000    0.000    0.000    0.000 {method 'append' of 'list' objects}
        1    0.000    0.000    0.000    0.000 {method 'disable' of '_lsprof.Profiler' objects}
        5    0.000    0.000    0.000    0.000 {method 'find' of 'bytearray' objects}
        1    0.000    0.000    0.000    0.000 {method 'items' of 'dict' objects}

>>>
```

cProfile results

The fifth column heading is **percall** again. Wait. What?

This is another call measure

This one **divides cumtime by primitive calls**

```
>>> import cProfile
>>> import re
>>> cProfile.run('re.compile("foo|bar")')
         214 function calls (207 primitive calls) in 0.000 seconds

   Ordered by: standard name

   ncalls  tottime  percall  cumtime  percall filename:lineno(function)
        1    0.000    0.000    0.000    0.000 <string>:1(<module>)
        2    0.000    0.000    0.000    0.000 enum.py:284(__call__)
        2    0.000    0.000    0.000    0.000 enum.py:526(__new__)
        1    0.000    0.000    0.000    0.000 enum.py:836(__and__)
        1    0.000    0.000    0.000    0.000 re.py:232(compile)
        1    0.000    0.000    0.000    0.000 re.py:271(_compile)
        1    0.000    0.000    0.000    0.000 sre_compile.py:249(_compile_charset)
        1    0.000    0.000    0.000    0.000 sre_compile.py:276(_optimize_charset)
        2    0.000    0.000    0.000    0.000 sre_compile.py:453(_get_iscased)
        1    0.000    0.000    0.000    0.000 sre_compile.py:461(_get_literal_prefix)
        1    0.000    0.000    0.000    0.000 sre_compile.py:492(_get_charset_prefix)
        1    0.000    0.000    0.000    0.000 sre_compile.py:536(_compile_info)
        2    0.000    0.000    0.000    0.000 sre_compile.py:595(isstring)
        1    0.000    0.000    0.000    0.000 sre_compile.py:598(_code)
      3/1    0.000    0.000    0.000    0.000 sre_compile.py:71(_compile)
        1    0.000    0.000    0.000    0.000 sre_compile.py:759(compile)
        3    0.000    0.000    0.000    0.000 sre_parse.py:111(__init__)
        7    0.000    0.000    0.000    0.000 sre_parse.py:160(__len__)
       18    0.000    0.000    0.000    0.000 sre_parse.py:164(__getitem__)
        7    0.000    0.000    0.000    0.000 sre_parse.py:172(append)
      3/1    0.000    0.000    0.000    0.000 sre_parse.py:174(getwidth)
        1    0.000    0.000    0.000    0.000 sre_parse.py:224(__init__)
        8    0.000    0.000    0.000    0.000 sre_parse.py:233(__next)
        2    0.000    0.000    0.000    0.000 sre_parse.py:249(match)
        6    0.000    0.000    0.000    0.000 sre_parse.py:254(get)
        1    0.000    0.000    0.000    0.000 sre_parse.py:286(tell)
        1    0.000    0.000    0.000    0.000 sre_parse.py:417(_parse_sub)
        2    0.000    0.000    0.000    0.000 sre_parse.py:475(_parse)
        1    0.000    0.000    0.000    0.000 sre_parse.py:76(__init__)
        2    0.000    0.000    0.000    0.000 sre_parse.py:81(groups)
        1    0.000    0.000    0.000    0.000 sre_parse.py:903(fix_flags)
        1    0.000    0.000    0.000    0.000 sre_parse.py:919(parse)
        1    0.000    0.000    0.000    0.000 {built-in method _sre.compile}
        1    0.000    0.000    0.000    0.000 {built-in method builtins.exec}
       25    0.000    0.000    0.000    0.000 {built-in method builtins.isinstance}
    29/26    0.000    0.000    0.000    0.000 {built-in method builtins.len}
        2    0.000    0.000    0.000    0.000 {built-in method builtins.max}
        9    0.000    0.000    0.000    0.000 {built-in method builtins.min}
        6    0.000    0.000    0.000    0.000 {built-in method builtins.ord}
       48    0.000    0.000    0.000    0.000 {method 'append' of 'list' objects}
        1    0.000    0.000    0.000    0.000 {method 'disable' of '_lsprof.Profiler' objects}
        5    0.000    0.000    0.000    0.000 {method 'find' of 'bytearray' objects}
        1    0.000    0.000    0.000    0.000 {method 'items' of 'dict' objects}

>>>
```

```
percall
```

cProfile results

When there are **two values in the ncalls** column, it means the **function recursed**:

The **first number** is the **total number of calls**

The **second value** is the **number of primitive calls**

- Primitive calls do NOT include recursive calls

3/1

- **3** total calls
- **1** primitive call
- (**3**-**1**) = **2** recursive calls

```
>>> import cProfile
>>> import re
>>> cProfile.run('re.compile("foo|bar")')
         214 function calls (207 primitive calls) in 0.000 seconds

   Ordered by: standard name

   ncalls  tottime  percall  cumtime  percall filename:lineno(function)
        1    0.000    0.000    0.000    0.000 <string>:1(<module>)
        2    0.000    0.000    0.000    0.000 enum.py:284(__call__)
        2    0.000    0.000    0.000    0.000 enum.py:526(__new__)
        1    0.000    0.000    0.000    0.000 enum.py:836(__and__)
        1    0.000    0.000    0.000    0.000 re.py:232(compile)
        1    0.000    0.000    0.000    0.000 re.py:271(_compile)
        1    0.000    0.000    0.000    0.000 sre_compile.py:249(_compile_charset)
        1    0.000    0.000    0.000    0.000 sre_compile.py:276(_optimize_charset)
        2    0.000    0.000    0.000    0.000 sre_compile.py:453(_get_iscased)
        [illegible]  0.000    0.000    0.000 sre_compile.py:461(_get_literal_prefix)
        [illegible]  0.000    0.000    0.000 sre_compile.py:492(_get_charset_prefix)
        [illegible]  0.000    0.000    0.000 sre_compile.py:536(_compile_info)
        [illegible]  0.000    0.000    0.000 sre_compile.py:595(isstring)
        1    0.000    0.000    0.000    0.000 sre_compile.py:598(_code)
      3/1    0.000    0.000    0.000    0.000 sre_compile.py:71(_compile)
        1    0.000    0.000    0.000    0.000 sre_compile.py:759(compile)
        3    0.000    0.000    0.000    0.000 sre_parse.py:111(__init__)
        7    0.000    0.000    0.000    0.000 sre_parse.py:160(__len__)
       18    0.000    0.000    0.000    0.000 sre_parse.py:164(__getitem__)
        7    0.000    0.000    0.000    0.000 sre_parse.py:172(append)
      3/1    0.000    0.000    0.000    0.000 sre_parse.py:174(getwidth)
        1    0.000    0.000    0.000    0.000 sre_parse.py:224(__init__)
        8    0.000    0.000    0.000    0.000 sre_parse.py:233(__next)
        2    0.000    0.000    0.000    0.000 sre_parse.py:249(match)
        6    0.000    0.000    0.000    0.000 sre_parse.py:254(get)
        1    0.000    0.000    0.000    0.000 sre_parse.py:286(tell)
        1    0.000    0.000    0.000    0.000 sre_parse.py:417(_parse_sub)
        2    0.000    0.000    0.000    0.000 sre_parse.py:475(_parse)
        [illegible]  0.000    0.000    0.000 sre_parse.py:76(__init__)
        [illegible]  0.000    0.000    0.000 sre_parse.py:81(groups)
        [illegible]  0.000    0.000    0.000 sre_parse.py:903(fix_flags)
        [illegible]  0.000    0.000    0.000 sre_parse.py:919(parse)
        [illegible]  0.000    0.000    0.000 {built-in method _sre.compile}
        [illegible]  0.000    0.000    0.000 {built-in method builtins.exec}
       25    0.000    0.000    0.000    0.000 {built-in method builtins.isinstance}
    29/26    0.000    0.000    0.000    0.000 {built-in method builtins.len}
        2    0.000    0.000    0.000    0.000 {built-in method builtins.max}
        9    0.000    0.000    0.000    0.000 {built-in method builtins.min}
        6    0.000    0.000    0.000    0.000 {built-in method builtins.ord}
       48    0.000    0.000    0.000    0.000 {method 'append' of 'list' objects}
        1    0.000    0.000    0.000    0.000 {method 'disable' of '_lsprof.Profiler' objects}
        5    0.000    0.000    0.000    0.000 {method 'find' of 'bytearray' objects}
        1    0.000    0.000    0.000    0.000 {method 'items' of 'dict' objects}

>>>
```

3/1

29/26

Given the following profiling snapshot, what are the number of recursive calls in **auction_entities.py:100(__call__)**

```
         336 function calls (238 primitive calls) in 0.000 seconds

   Ordered by: call count

   ncalls  tottime  percall  cumtime  percall filename:lineno(function)
       76    0.000    0.000    0.000    0.000 {method 'random' of '_random.Random' objects}
     70/4    0.000    0.000    0.000    0.000 auction_entities.py:100(__call__)
       53    0.000    0.000    0.000    0.000 auction_entities.py:66(current_bid)
       32    0.000    0.000    0.000    0.000 auction_entities.py:115(__str__)
       27    0.000    0.000    0.000    0.000 {built-in method builtins.print}
     17/1    0.000    0.000    0.000    0.000 auction_entities.py:57(_notify_bidders)
     17/1    0.000    0.000    0.000    0.000 auction_entities.py:74(accept_bid)
       17    0.000    0.000    0.000    0.000 auction_entities.py:70(highest_bidder)
       10    0.000    0.000    0.000    0.000 {method 'append' of 'list' objects}
```

- ◯ 53
- ◯ 70
- ◯ 66
- ◯ 4

Quiz review

What is the output if the following code was run?

```
print(type(filter(lambda x: x == 1, [1, 2, 3])))
```

- ◯ <class 'iterator'>
- ◯ <class 'function'>
- ◯ <class 'list'>
- ◯ <class 'filter'>

Lambda Functions – Anonymous Functions

Lambda Functions are also known as Anonymous Functions

These are functions without a name

Think of defining a temporary function which has a small scope.

Regular function:

```
def my_func(parameters):
    expression
```

Syntax:

```
lambda parameters: expression
```

Lambda Functions – Anonymous Functions

Regular function:

```
def my_func(x):
    return x * 5
```

Lambda function

```
lambda x: x * 5
```

This is a lambda function that takes 1 argument `x` and **`returns`** `x * 5`

Lambda Functions - example

Can store lambdas into a **variable**.

Can use that **variable** as if it was the lambda

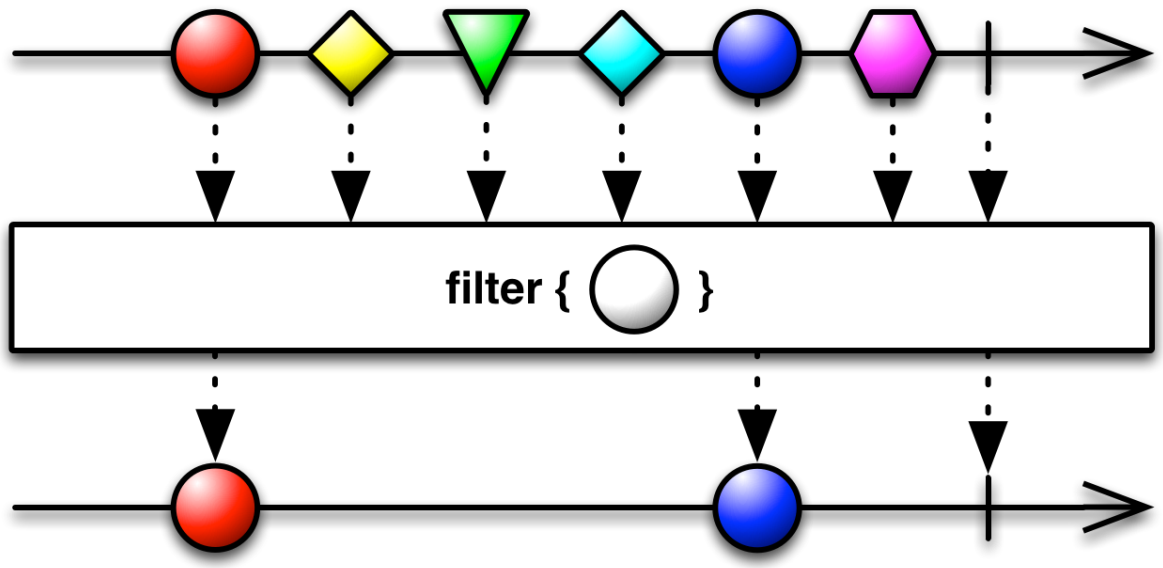
```
multiple_of_five = lambda x: x * 5
print(multiple_of_five(10)) #50
```

<u>Lambda functions aren't multi-line!</u>

Filter

filter(function, sequence)

- Does exactly what the name suggests. Filters a number of elements from a given sequence.
- The first argument 'function' needs to be a function that returns True or False.
- This function is then applied to each element of the second argument, an iterable.
  - **filter(*function*, *iterable*)**
- If the result is false, that element is skipped. Otherwise it is included in the resultant filter object. This is an iterator similar to the map object.
- *filter()* can only work with **one argument** and **one sequence**.



Filter(function, sequence) – Example

Say we want to do the most cliché example.

Let's create a filter that extracts all the even numbers from a list with numbers 1 to 9.

Notice how the **lambda does not return the true/false value**

```
num_list = list(range(1, 10))
even_nums_filter = filter(lambda x: x % 2 == 0, num_list)
print(type(even_nums_filter)) #<class 'filter'>
print(list(even_nums_filter)) #[2, 4, 6, 8]
```

Filter(function, sequence) – Example

Say we want to do the most cliché example.

Let's create a filter that extracts all the even numbers from a list with numbers 1 to 9.

The same as before but using a **regular function** instead of *lambda*. Notice how we **must return the true/false value when using a function**

```
def even(x):
    return x % 2 == 0

num_list = list(range(1, 10))
even_nums_filter = filter(even, num_list)
print(type(even_nums_filter)) #<class 'filter'>
print(list(even_nums_filter)) #[2, 4, 6, 8]
```

Quiz review

What is the output if the following code was run?

```
print(type(filter(lambda x: x == 1, [1, 2, 3])))
```

- ◯ <class 'iterator'>
- ◯ <class 'function'>
- ◯ <class 'list'>
- ◯ <class 'filter'>

Quiz review

What would be the output if the following code is run?

```
print("COMP", 3522, "students are amazing", end = "!!")
```

- Type Error
- COMP 3522 students are amazing!!
- COMP3522students are amazing!!
- None of the above

A note on print()

There are different versions of the print( ) function:

1. print(‘Hello world’)
2. print(“Hello world”)
3. print(“Hello world”, **end=‘ ’**) #default end is newline
4. print(“I\’m ” **+** str(age) **+** “ years old”)
5. print(“I\’m”**,** age**,** “years old”)

You can learn more here:

https://docs.python.org/3.13/library/functions.html

Quiz review

What would be the output if the following code is run?

```
print("COMP", 3522, "students are amazing", end = "!!")
```

- ◯ Type Error
- ◯ COMP 3522 students are amazing!!
- ◯ COMP3522students are amazing!!
- ◯ None of the above

What would be the value of self._age if the following code was executed?

```
class Person():
    def set_age(self, an_age):
        if an_age >= 0 and an_age <= 150:
            self._age = an_age

    def get_age(self):
        return self._age

    age = property(get_age, set_age)

    def __init__(self, name, an_age):
        self._name = name
        self.age = an_age

zorak = Person("Zorak the destroyer", 40)
zorak.age = 290
print(zorak.age)
```

- The code would throw an error.
- 290
- 40
- None

There Is A Standard!

Developers still need to communicate that it is not safe to access certain attributes in a class/object.

Single Underscore

```
class Cat:
    def __init__(self, age):
        self._age = age
```

A single underscore communicates to other developers that they should ideally not be touching this variable and modifying it is not recommended.

Conceptually similar to "**protected**" instance variables

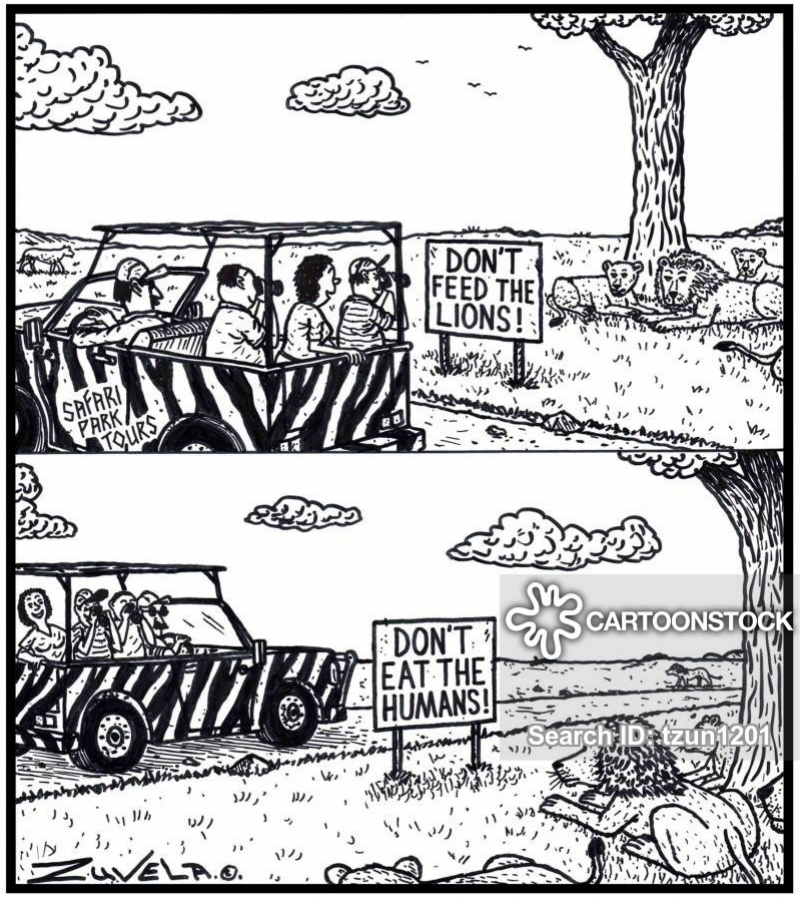
Double Underscores and Name Mangling

A variable name prefaced with double underscores (Dunder) is a way of saying “STAY AWAY”. Conceptually similar to “**private**” instance variables

Python makes it difficult to access this variable through **Name Mangling** , that means it’s only accessible by prefacing it with the **class name** and some more **underscores**.

Since it’s a bother to access the variables, developers are discouraged from detecting them and going near them.

```
class Cat:
    def __init__(self, age):
        self.__age = age
…
my_cat = Cat(10)
my_cat.age = 5 #doesn’t change cat __age. Creates new age variable
print(my_cat.__age) # ERROR
my_cat._Cat__age = 5 # Okay, but not good form
```



Properties vs Accessors / Mutators

A property is a way to encapsulate the getter (Accessor) and setter (Mutator) of an attribute.

Anyone interfacing the code with your library can still access the data attribute through a variable name.

The variable name automatically invokes the Accessor and Mutator.

Eg:

```
# set inside class definition
temperature = property(get_temperature, set_temperature)

..

..

my_engine.temperature = 70  # this would invoke set_temperature(70)
```

property_example.py

What would be the value of self._age if the following code was executed?

```
class Person():
    def set_age(self, an_age):
        if an_age >= 0 and an_age <= 150:
            self._age = an_age

    def get_age(self):
        return self._age

    age = property(get_age, set_age)

    def __init__(self, name, an_age):
        self._name = name
        self.age = an_age

zorak = Person("Zorak the destroyer", 40)
zorak.age = 290
print(zorak.age)
```

- ◯ The code would throw an error.
- ◯ 290
- ◯ 40
- ◯ None

Quiz review

Examine this code:

```
def unpack(some_container_object):
    #code to unpack some_container_object
    print(a, b, c)

unpack([1,2,3])
```

Replace the **commented code** with the option that will properly unpack the some_container_object

- ◯ a, b, c = some_container_object[0,1,2]
- ◯ a, b, c = some_container_object
- ◯ a, b, c = [some_container_object]
- ◯ a, b, c = *some_container_object

Variable Argument List - “Packing Arguments”

We specify an argument with an * preceding its name. The standard/convention is *args

This is known as **packing** arguments. All arguments are packed into a **tuple** named args

```
def my_print(*args):
    for item in args:
        print(item)

my_print('dog', 5, {1:'hi', 2:'hello'})
```

Variable Argument List - “Unpacking”

The * (asterisk) can be used with **containers** as well. This is known as **unpacking**

In the example below, the **list** is “unpacked” into the individual string, integer, and list objects

```
def my_print(in_str, in_int, in_dict):
    print(in_str, in_int, in_dict)

my_list = ['dog', 5, {1: 'hi', 2: 'hello'}]
my_print(*my_list) #unpack list
```

team_variable_arguments.py

Quiz review

Examine this code:

```
def unpack(some_container_object):
    #code to unpack some_container_object
    print(a, b, c)

unpack([1,2,3])
```

Replace the **commented code** with the option that will properly unpack the some_container_object

- ◯ a, b, c = some_container_object[0,1,2]
- ◯ a, b, c = some_container_object
- ◯ a, b, c = [some_container_object]
- ◯ a, b, c = *some_container_object

```python
class Team:
    def __init__(self, members):
        self._members = members


my_team = ['Jen','Bob','Jacky']
team = Team(my_team)
print('Jen' in team)
```

- TypeError: argument of type 'Team' is not iterable
- True
- False
- Jen

The Container protocol

Provides the method **__contains__(self, item)**

When the membership operator **in** or **not in** is invoked on an object, this is the method invoked

This is how a class implements the membership test operator

__contains__(self, item) must **return true** if the item is **in** self, **else false**

Implemented by str, list, range, tuple, bytes, set,dict

team_container.py

The Iterable protocol

Provides the method **__iter__(self)**

Invoked to generate an iterator object for a container

Should **return a new iterator** object that can iterate over all the objects in the container

Invoked by

1. passing the object to the global Python `iter( )` function
2. using the for-loop, i.e., `for item in iterable`

team_iterable.py

```python
class Team:
    def __init__(self, members):
        self._members = members


my_team = ['Jen','Bob','Jacky']
team = Team(my_team)
print('Jen' in team)
```

- TypeError: argument of type 'Team' is not iterable
- True
- False
- Jen

Quiz review

Which of the following are the **three broad categories of design patterns**?
(Select all that apply.)

- [ ] Structural
- [ ] Functional
- [ ] Creational
- [ ] Behavioral
- [ ] Architectural

Categorizing Design Patterns

- **Behavioural**

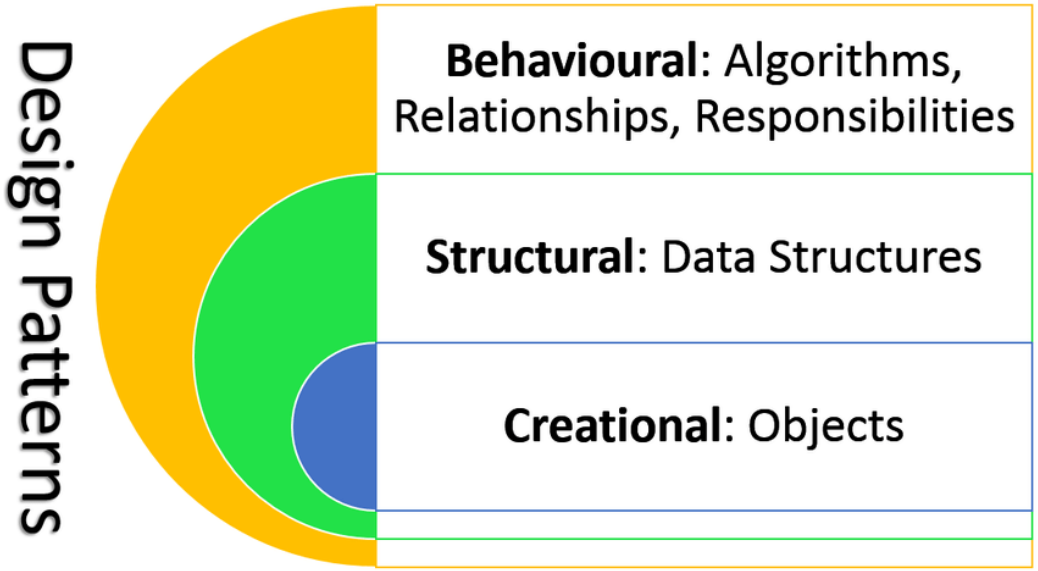
Focused on communication and interaction between objects. How do we get objects talking to each other while minimizing coupling?

- **Structural**

How do classes and objects combine to form structures in our programs? Focus on architecting to allow for maximum flexibility and maintainability.

- **Creational**

All about class instantiation. Different strategies and techniques to instantiate an object, or group of objects

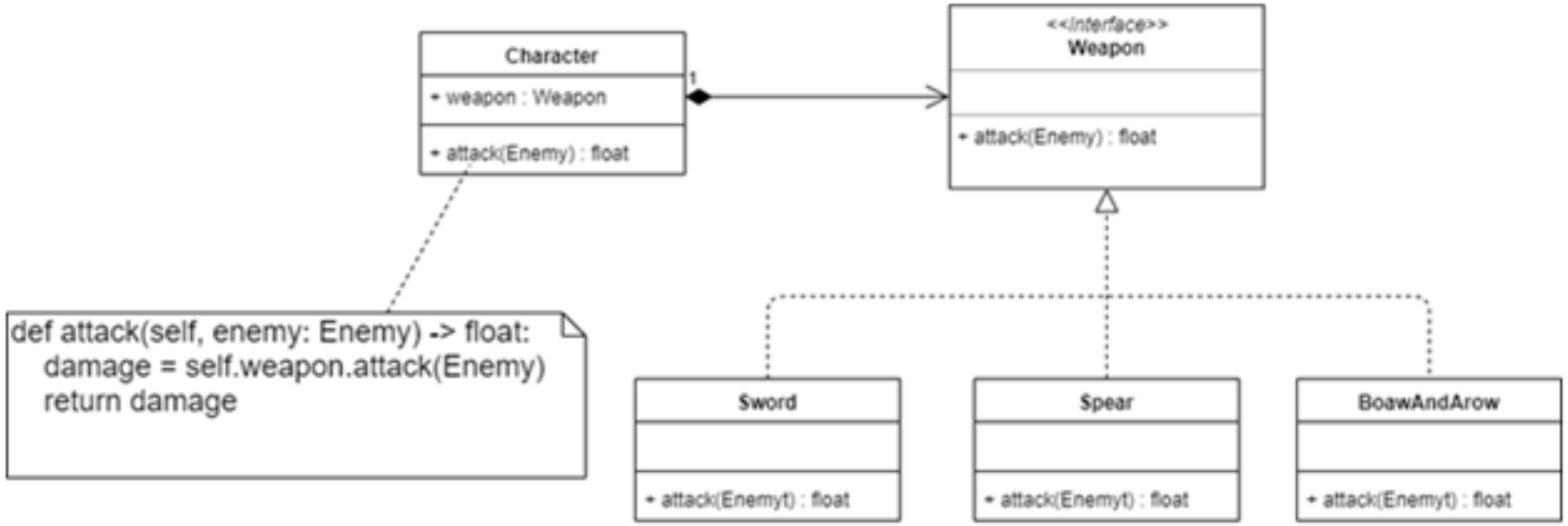


Quiz review

Which of the following are the **three broad categories of design patterns**?
(Select all that apply.)

- [ ] Structural
- [ ] Functional
- [ ] Creational
- [ ] Behavioral
- [ ] Architectural

Pick the design pattern that best matches the class diagram



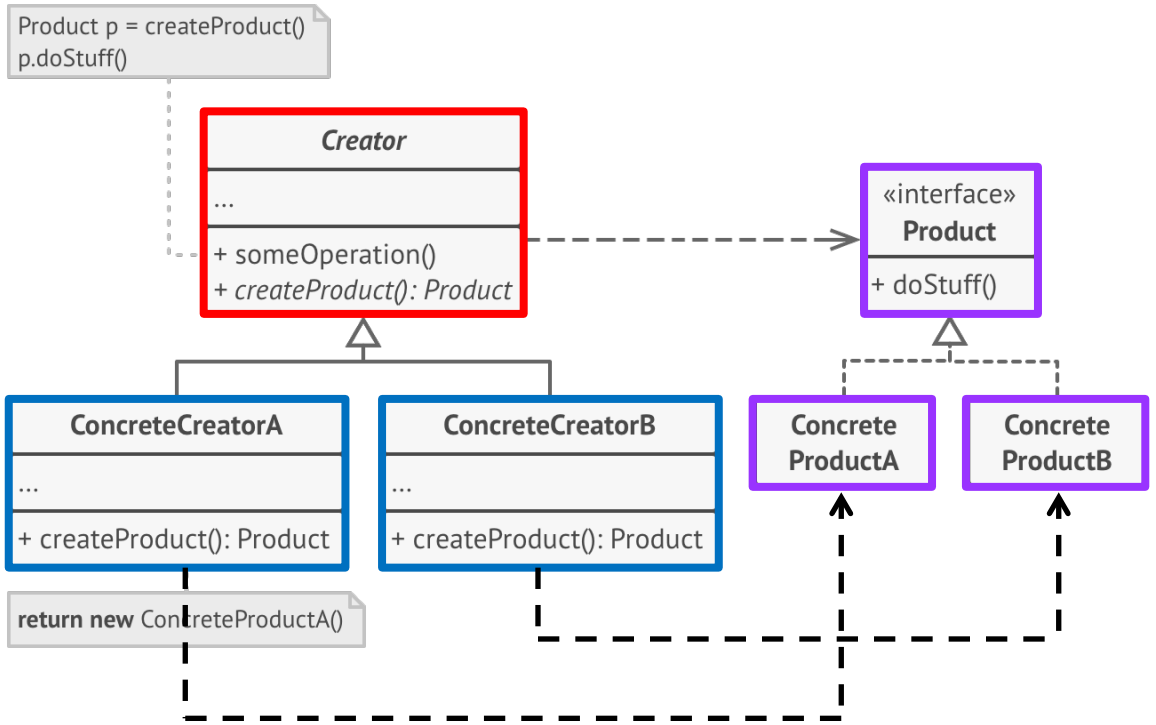
- Factory
- State
- Strategy
- Chain of Responsibility

Creational - Factory

When you want to separate creation code from client code.

Use this when: you want a **client or service to only depend on a base class for object creation and not each concrete class.** In other words, the client depends on a base factory for object creation, instead of each individual concrete factory

Example: Forum that needs access to different types of **users** (**Guest**, **member**). **Factory** can produce both types and the forum uses them without knowing the **concrete type**



forum_user_factory.py

Behavioural – Strategy

Use this when: an **object** needs to switch **behaviours** at runtime

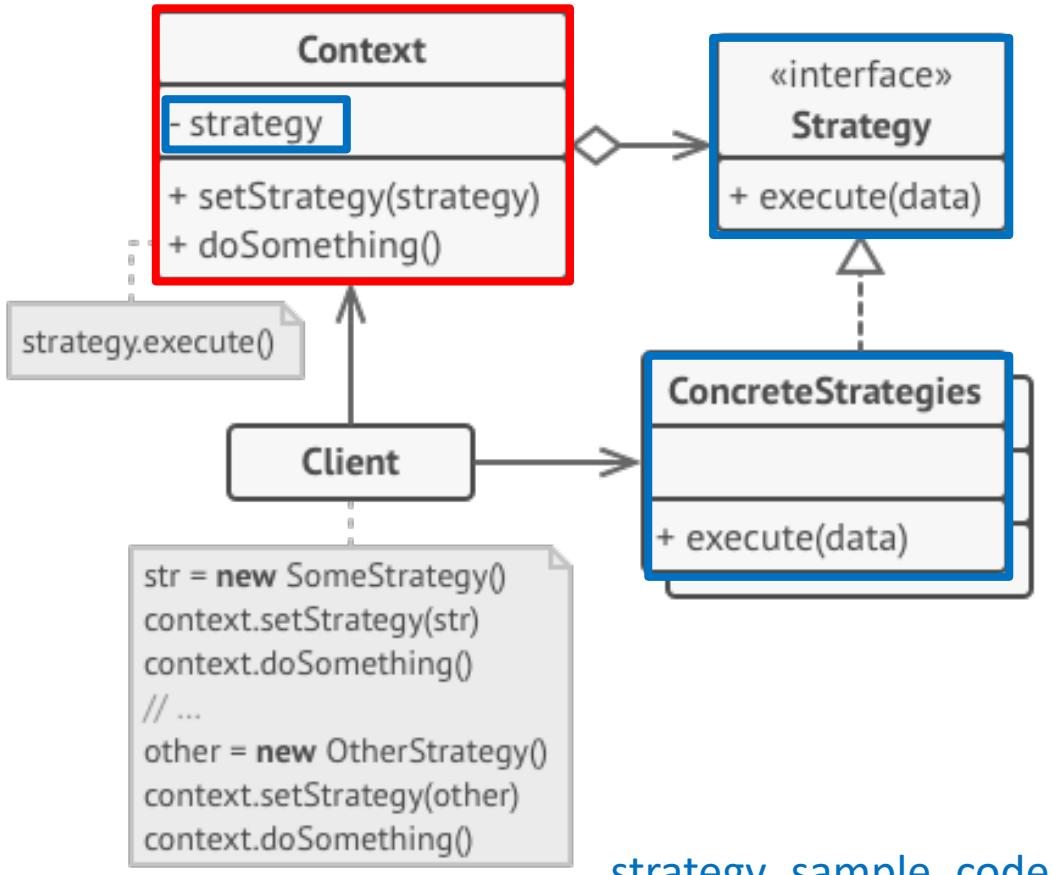
Example:

A game **character** wants to use different **weapons** when attacking

The character and weapon classes contain attack methods.

Change the character's attack behavior by swapping the weapon objects (axe, sword, hammer)

Character's attack method forwards calls to the weapon's attack method



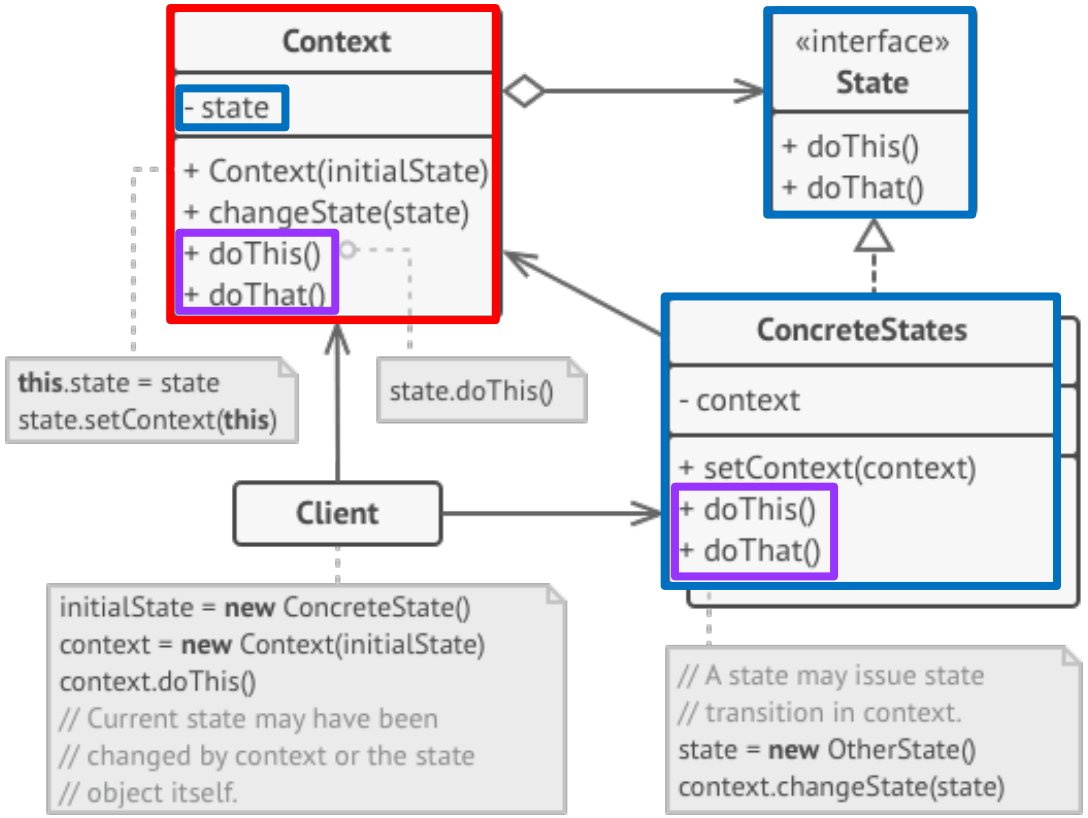
strategy_sample_code.py

Behavioural – State

When the same **object** can take on **different behaviours** depending on its **state**.

Having different specialized behaviours within the same class. The **context** changes their **state** by swapping out their internal state object. This causes all the **behaviours** in the context to use the new state object's behaviors

Use this when: you need almost all the behaviors of an object to change depending on its internal state

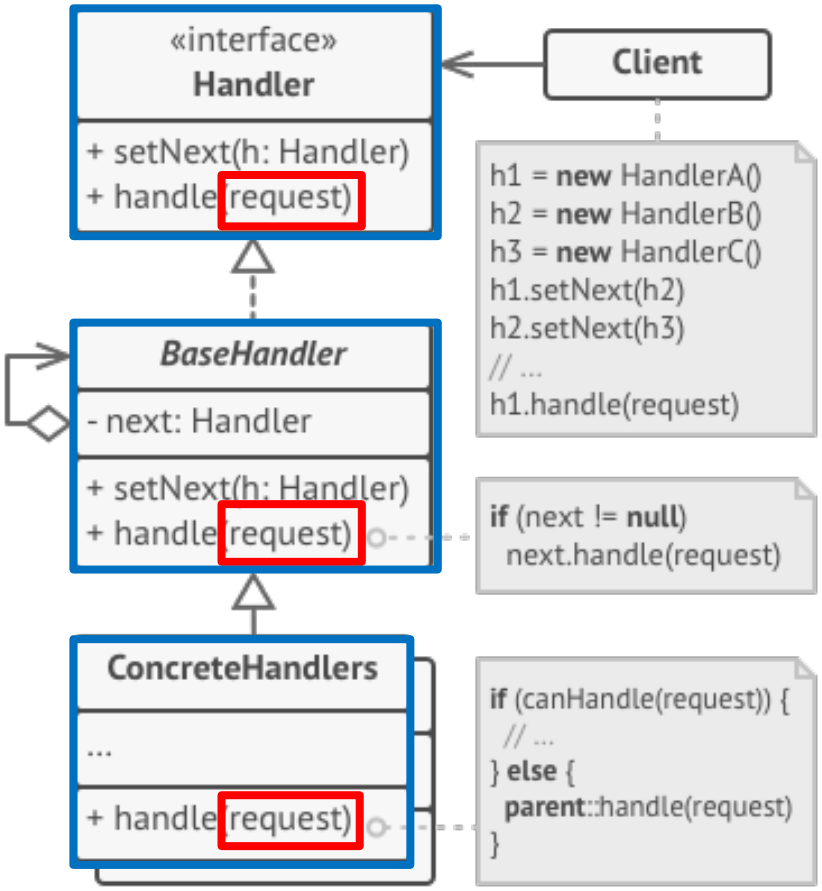


Behavioural – Chain of responsibility

Use this when: a **request** needs to go through a **series of processing steps**. Decouples the sequencing of these steps and makes them re-usable

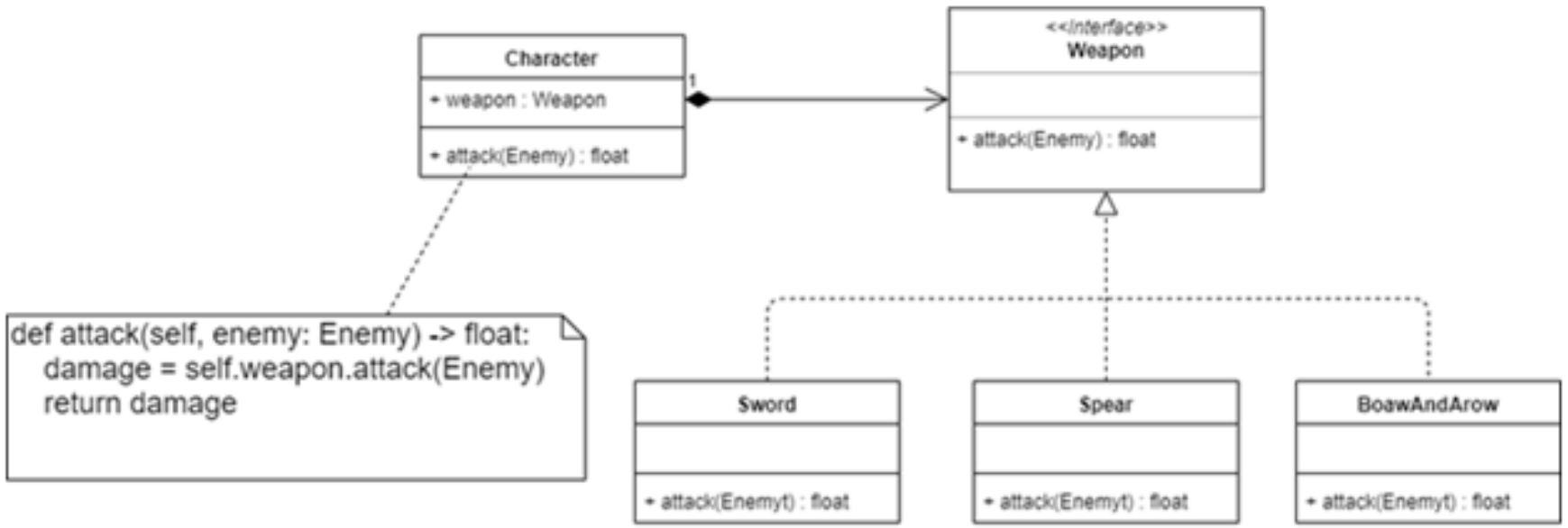
Example: A **document** needs to go through **several stages of validation**. Need to validate:

- **courses have correct name and year**
- **check fees paid**
- **student information correct**



chain_of_responsibility.py

Pick the design pattern that best matches the class diagram



- ◯ Factory
- ◯ State
- ◯ Strategy
- ◯ Chain of Responsibility

Quiz review

Select the design patterns that use the Wrapper concept for their implementation

- [ ] Facade
- [ ] Abstract Factory
- [ ] Proxy
- [ ] Factory
- [ ] Bridge

Wrapper

What is a wrapper?

A class that **‘wraps’ around another class**.

The **wrapper** has the **same interface** as the ‘**wrappee**’.

Remember, this is interface in the OOP sense. The wrapper has the same public method and attributes as the wrappee. We can implement this using an ABC or by leaving it informal (duck typing).

All calls to the wrapper call the subsequent methods in the wrappee or the wrapped object.

What is a wrapper?

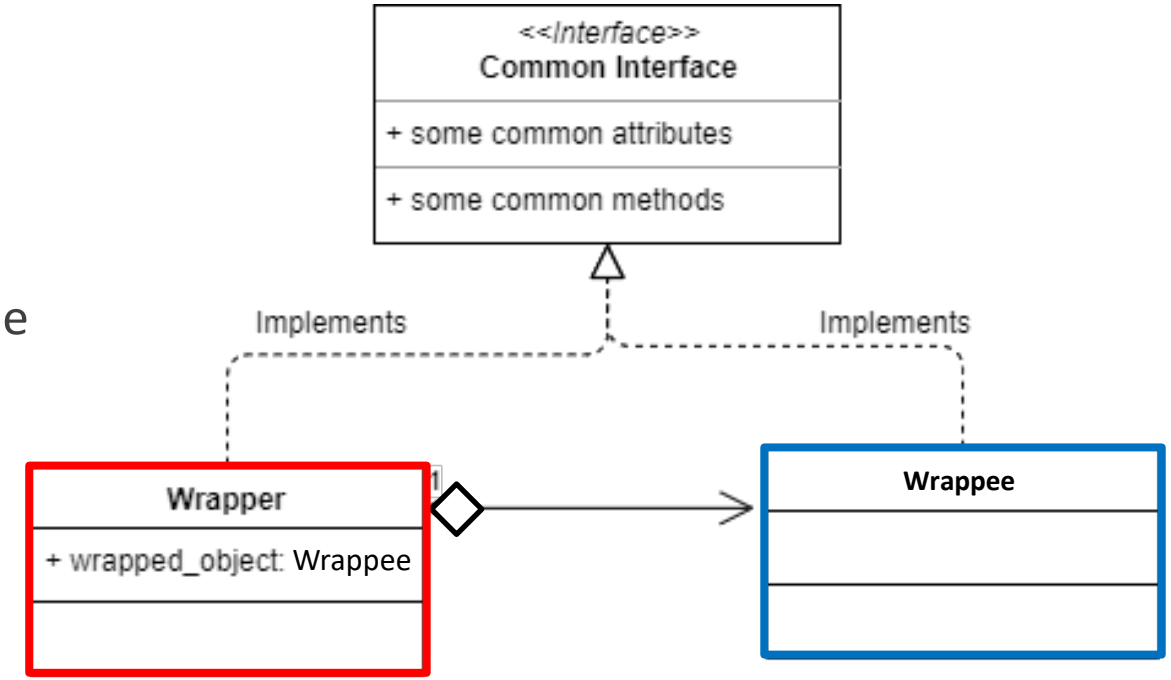
A **class** that **'wraps' around another class**. The **wrapper** uses a **has-a** relationship with **wrappee**.

In this example, the wrapper has the same public method and attributes as the wrappee. We can implement this using an interface or abstract base class

All calls to the wrapper call the subsequent methods in the wrappee or the wrapped object.

Wrapping an object, with another object of the same interface allows us the flexibility to perform extra processing before and after we make the call to the wrappee

Note – the wrapper does NOT always have to have the same interface as wrappee



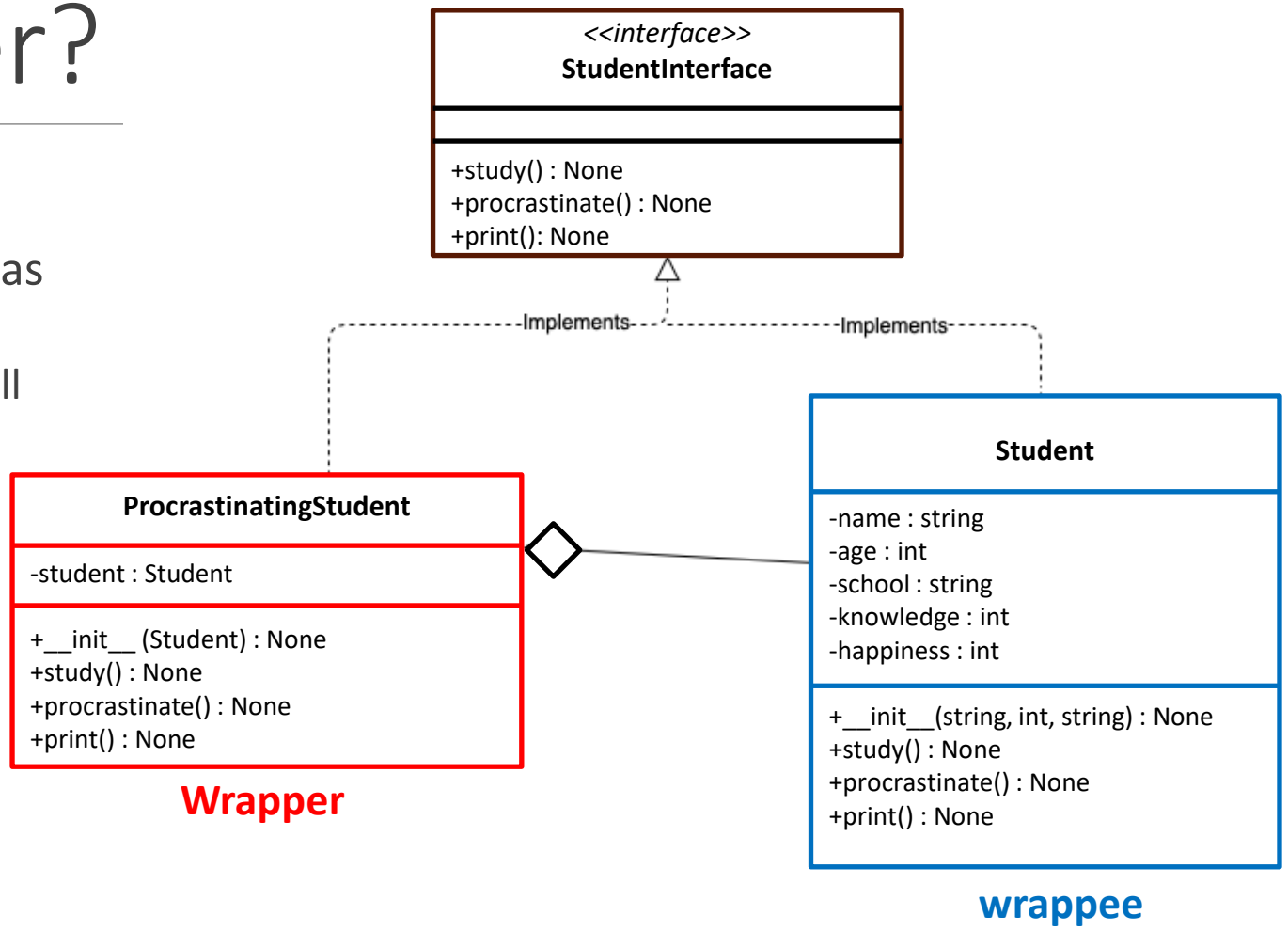
What is a wrapper?

Student is the wrappee

ProcrastinatingStudent is a wrapper. It has an instance of student

- Any calls to its methods will eventually call the student instance's methods, but with possibly additional behavior

Both classes implement the methods defined in the **StudentInterface**



Wrapper.py

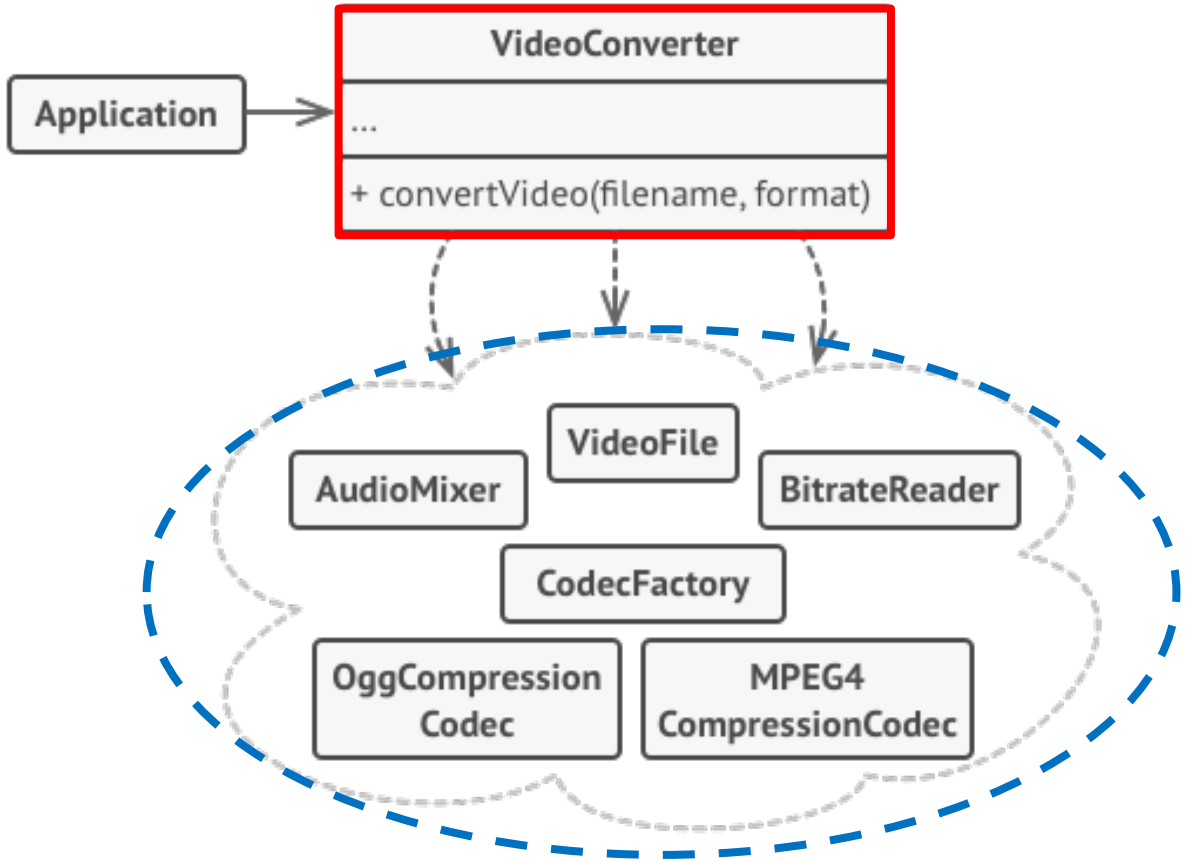
Structural – Façade

A **simplified interface** encapsulates a **complex system or API**. Client only interacts with the façade instead of complex systems

Use this when: you want to **hide the complexity of a system** from the client, but allow them to use it in a **simplified form**.

Example: **Video converter**. Only want to expose the convert video function to the client. But behind the scenes, there are **many systems** the facade must call to perform the action

simple_facade.py
video_package_facade_example.py



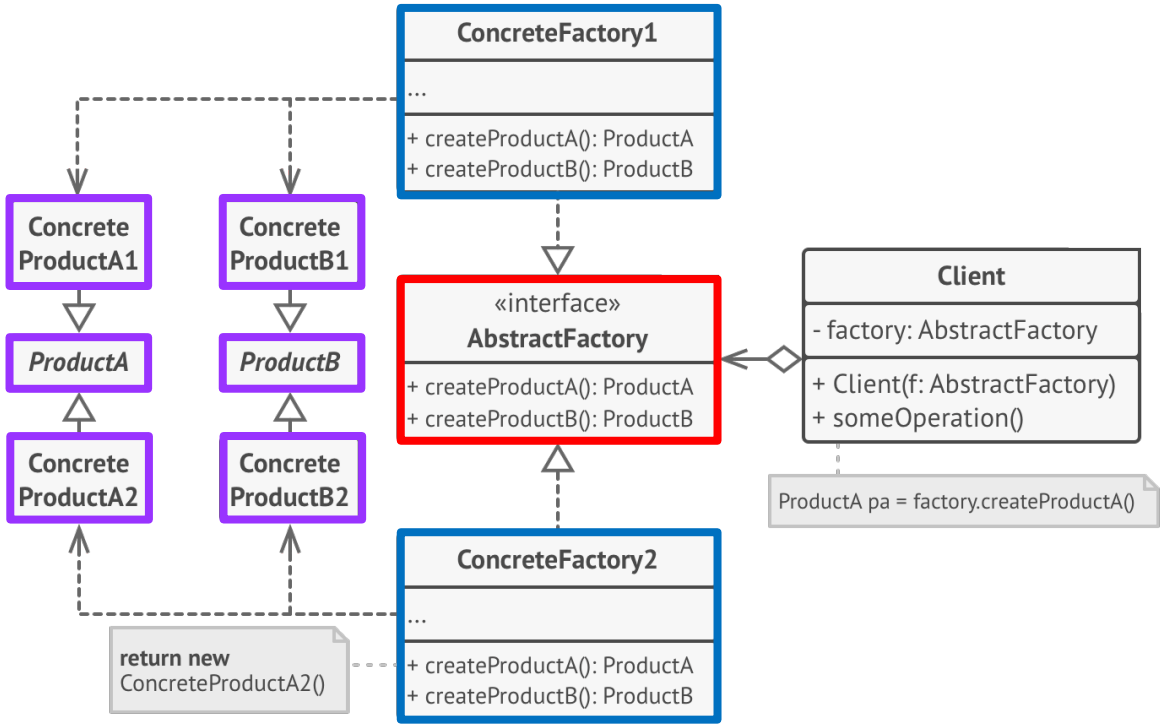
Creational – Abstract Factory

A more evolved form of the Factory Pattern

Allows for the creation of a group of related objects .

Use this when: you want to create **different variants of families of objects.** Client also relies on **base factory** for creation instead of each **individual concrete factory**

Example: Client wants to order different themes of **furniture**. Client gets passed the **appropriate Factory,** which creates a set of **themed furniture**.



game_abstract_factory.py

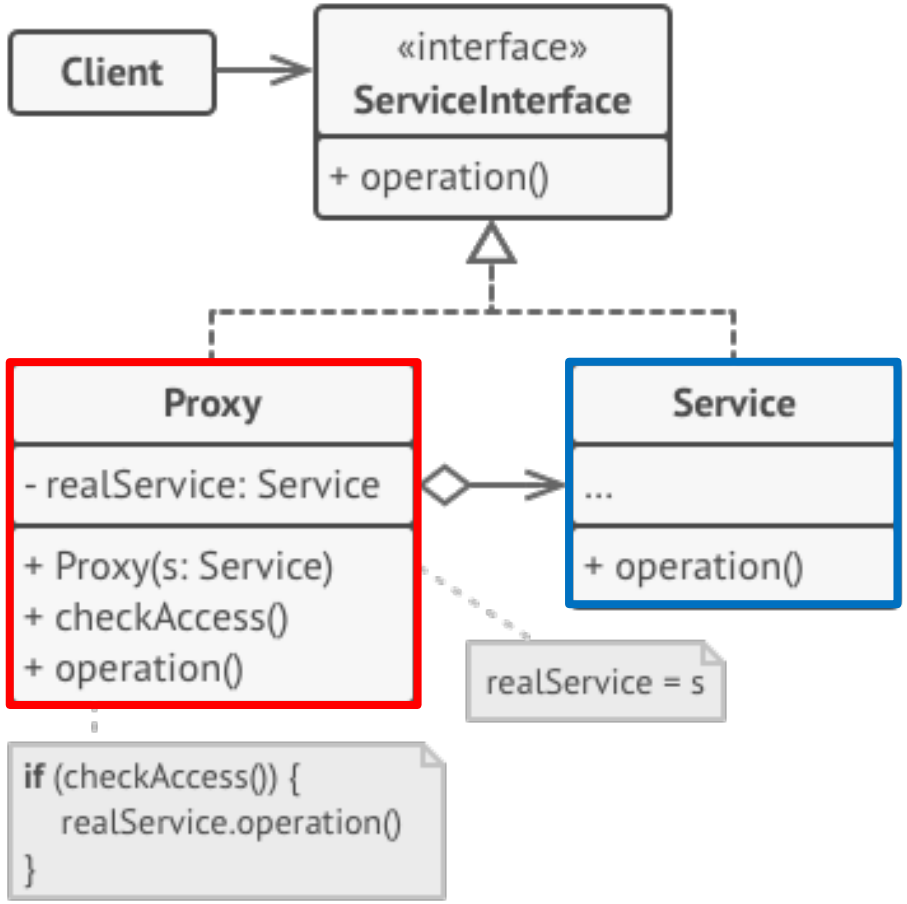
Structural – Proxy

A class functioning as an interface to something else. A **proxy is a wrapper** or agent object that is being called by the client to access the **real serving object** behind the scenes

Use this when: you have a **real service or object that you wish to hide the client** from directly interacting with.

Example: Youtube **database** is a real service. Prevent all users in the world to access one database. Create a **proxy object** that intercepts requests and forwards to Youtube database. Repeat requests are cached

simple_proxy.py
movie_database_proxy_example.py

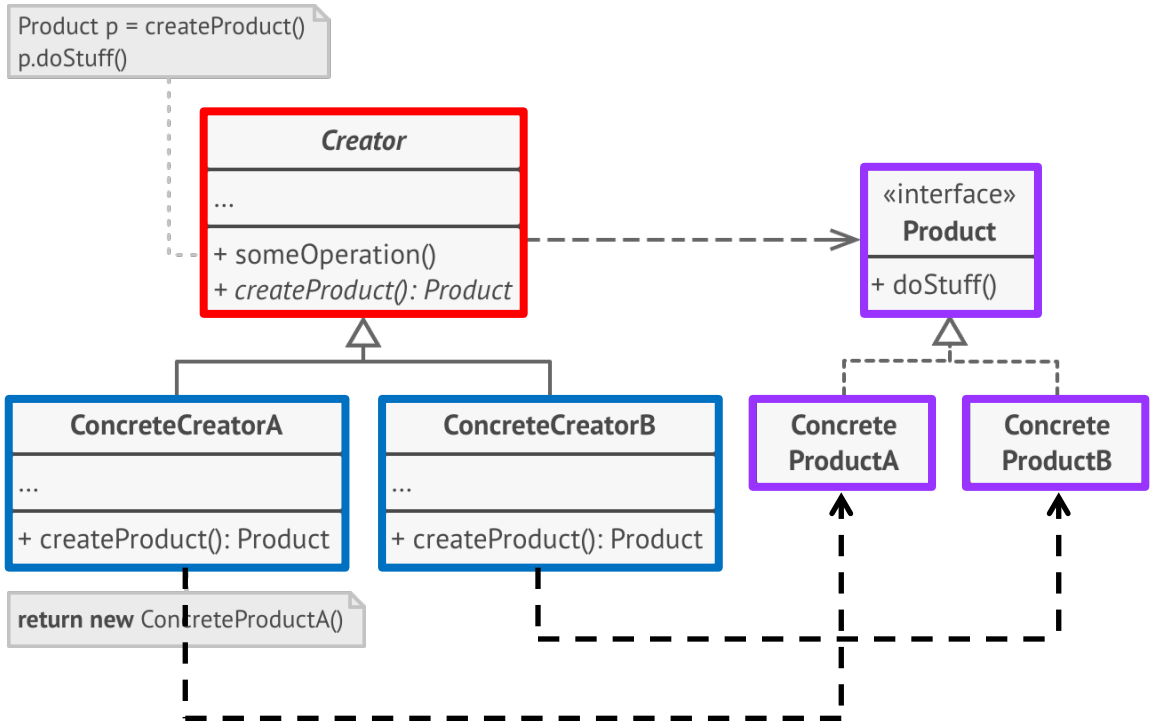


Creational - Factory

When you want to separate creation code from client code.

Use this when: you want a **client or service to only depend on a base class for object creation and not each concrete class.** In other words, the client depends on a base factory for object creation, instead of each individual concrete factory

Example: Forum that needs access to different types of **users** (**Guest**, **member**). **Factory** can produce both types and the forum uses them without knowing the **concrete type**



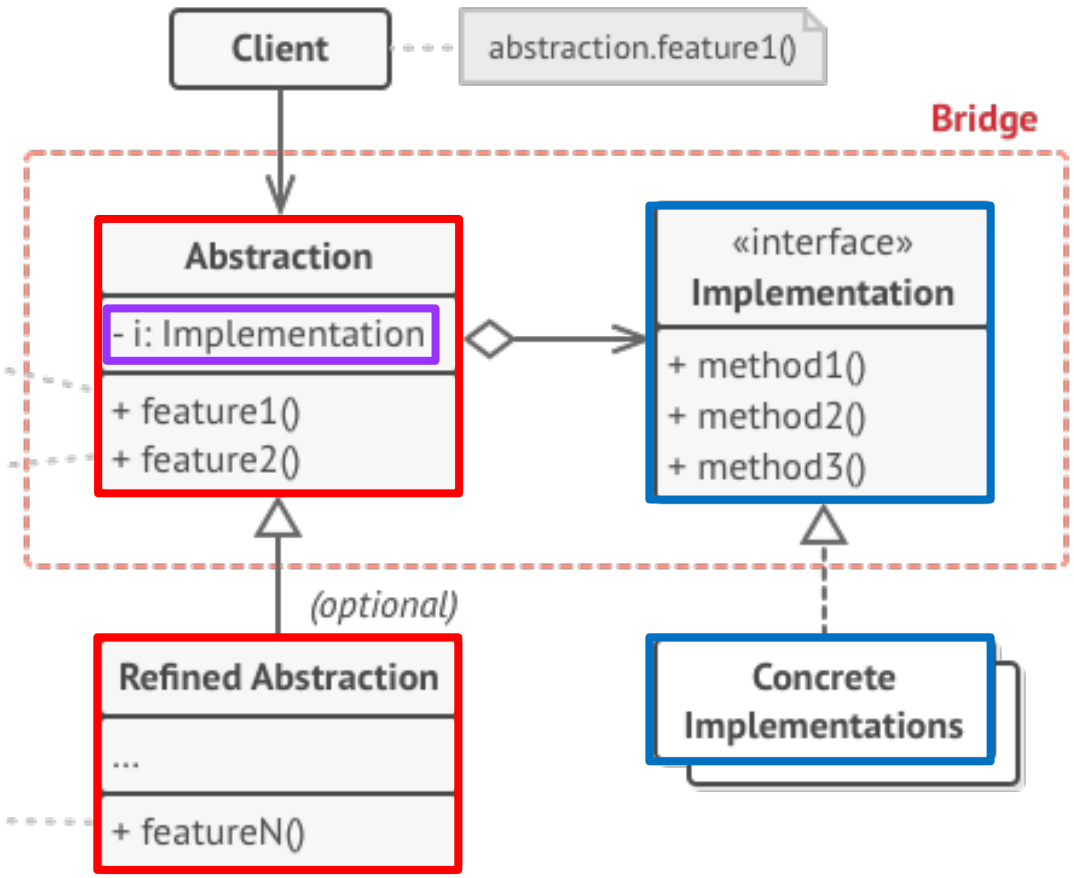
forum_user_factory.py

Structural – Bridge

The bridge pattern allows the **Abstraction** and the **Implementation** to be developed independently and the client code can access only the Abstraction part without being concerned about the Implementation part

Use this when: you want to **break down a large class into smaller components**. These components are connected to the main abstraction class via an **instance** of the implementation

Example: Creating a TV **remote** class that controls a **Device**



Quiz review

Select the design patterns that use the Wrapper concept for their implementation

- [ ] Facade
- [ ] Abstract Factory
- [ ] Proxy
- [ ] Factory
- [ ] Bridge

Quiz review

Match the design pattern to its appropriate definition

- [ ] Provide a simplified interface to a framework, resource or any set of classes
- [ ] Breaking down a large class or a large set of coupled classes into abstractions and implementations
- [ ] A wrapper that acts as a placeholder for an existing entity and control access to it

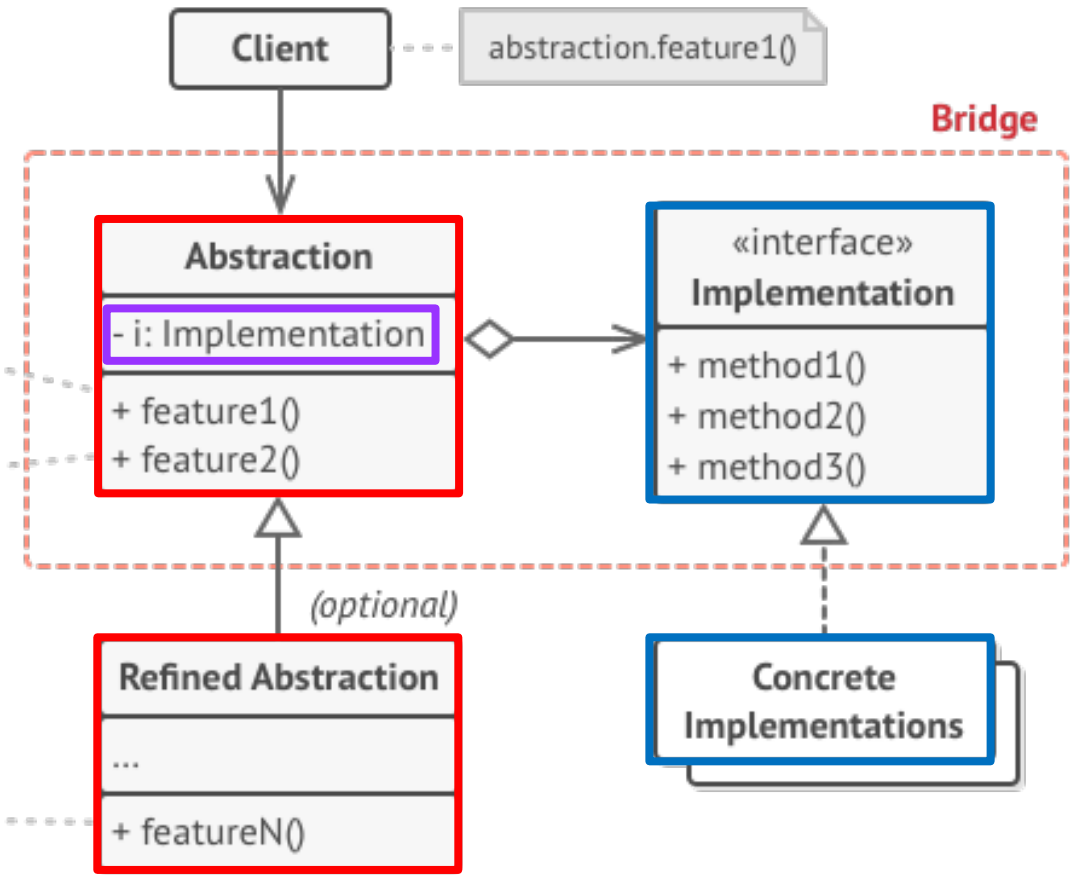
1. Bridge
2. Facade
3. Proxy

Structural – Bridge

The bridge pattern allows the **Abstraction** and the **Implementation** to be developed independently and the client code can access only the Abstraction part without being concerned about the Implementation part

Use this when: you want to **break down a large class into smaller components**. These components are connected to the main abstraction class via an **instance** of the implementation

Example: Creating a TV **remote** class that controls a **Device**



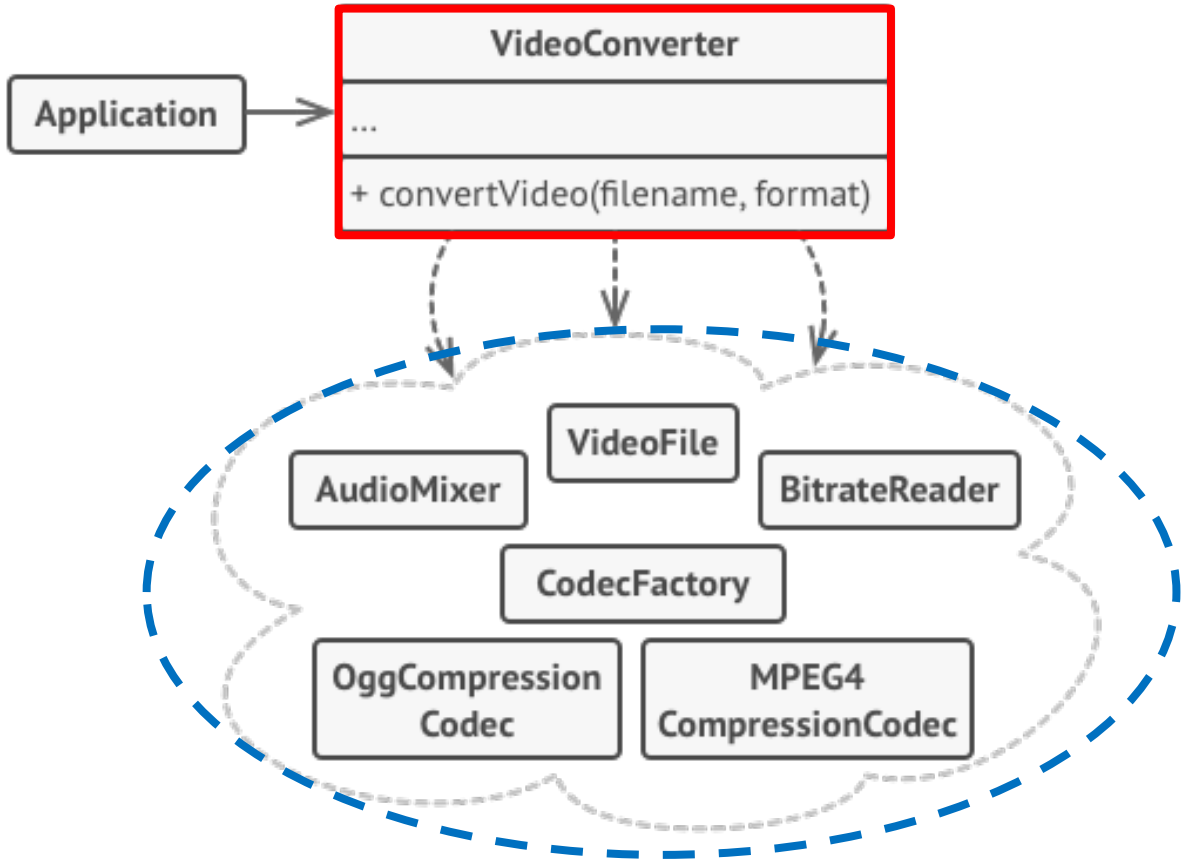
Structural – Façade

A **simplified interface** encapsulates a **complex system or API**. Client only interacts with the façade instead of complex systems

Use this when: you want to **hide the complexity of a system** from the client, but allow them to use it in a **simplified form**.

Example: **Video converter**. Only want to expose the convert video function to the client. But behind the scenes, there are **many systems** the facade must call to perform the action

simple_facade.py
video_package_facade_example.py



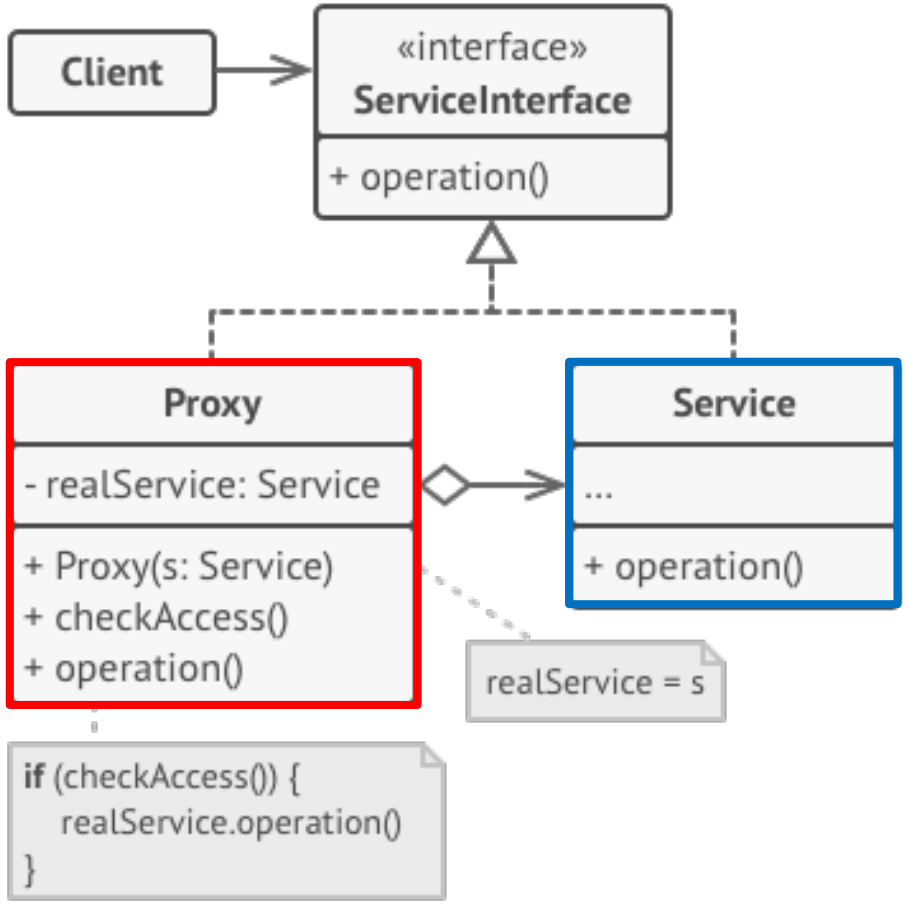
Structural – Proxy

A class functioning as an interface to something else. A **proxy is a wrapper** or agent object that is being called by the client to access the **real serving object** behind the scenes

Use this when: you have a **real service or object that you wish to hide the client** from directly interacting with.

Example: Youtube **database** is a real service. Prevent all users in the world to access one database. Create a **proxy object** that intercepts requests and forwards to Youtube database. Repeat requests are cached

simple_proxy.py
movie_database_proxy_example.py



Quiz review

Match the design pattern to its appropriate definition

[ ]	Provide a simplified interface to a framework, resource or any set of classes
[ ]	Breaking down a large class or a large set of coupled classes into abstractions and implementations
[ ]	A wrapper that acts as a placeholder for an existing entity and control access to it

1. Bridge
2. Facade
3. Proxy

Quiz review

Match the specific design pattern to the broader category of design patterns

- [ ] Abstract Factory
- [ ] Proxy
- [ ] Facade
- [ ] Bridge
- [ ] Factory

1. Behavioral design patterns - Focused on communication and interaction between objects. How do we get objects talking to each other while minimizing coupling?
2. Structural design patterns - Focus on how classes and objects combine to form structures in our programs? Focus on architecting to allow for maximum flexibility and maintainability.
3. Creational design patterns - All about class instantiation. Different strategies and techniques to instantiate an object, or group of objects

Categorizing Design Patterns

- **Behavioural**

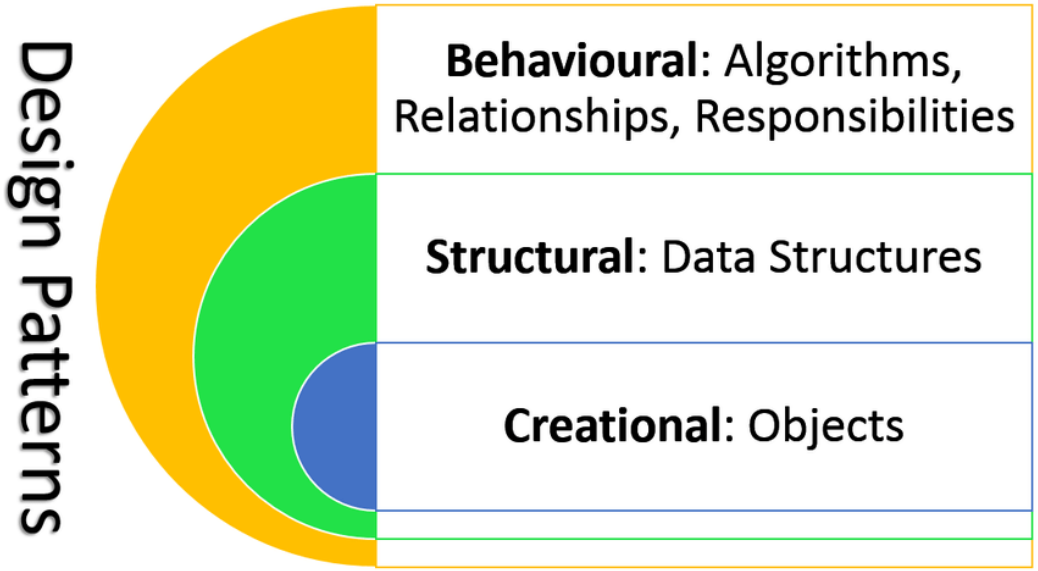
Focused on **communication** and interaction between objects. How do we get objects talking to each other while minimizing coupling?

- **Structural**

How do classes and objects **combine to form structures** in our programs? Focus on architecting to allow for maximum flexibility and maintainability.

- **Creational**

All about class **instantiation**. Different strategies and techniques to instantiate an object, or group of objects



Behavioural , Structural, Creational

- **Behavioural**

Focus on Communication

- **Structural**

Combine to form structures

- **Creational**

All about class instantiation

Wrappers – combine classes to form new structures

Bridge – decouples abstraction from implementation

Façade – simple interface hides complex sub system

Proxy – placeholder object controls access to important object

Set up classes to focus on class instantiation

Factory – an interface for creating an object without specifying concrete class

Abstract Factory – an interface to create groups of related objects without specifying concrete classes

Quiz review

Match the specific design pattern to the broader category of design patterns

- [ ] Abstract Factory
- [ ] Proxy
- [ ] Facade
- [ ] Bridge
- [ ] Factory

1. Behavioral design patterns - Focused on communication and interaction between objects. How do we get objects talking to each other while minimizing coupling?
2. Structural design patterns - Focus on how classes and objects combine to form structures in our programs? Focus on architecting to allow for maximum flexibility and maintainability.
3. Creational design patterns - All about class instantiation. Different strategies and techniques to instantiate an object, or group of objects

Mock final available

Mock final exam link

Link

Mock final exam answers link

Link

Week 14 – Lab folder

- Mock exam format is not indicative of final exam format

Reminder: 2 cheat sheets allowed on the final exam

- Both sheets can be double sided, hand written or printed 10 point font

Final exam format – Will be writing more code by hand. No selecting correct code snippets for long coding question

DESIGN PATTERNS REVIEW

COMP3522 OBJECT ORIENTED PROGRAMMING 2

WEEK 14

What are Design Patterns

Common design solutions to common architectural problems.

Think of these as recipes.

While developing OOP programs you might come across common design problems.

We've seen a ton of these this term

Sources:

https://sourcemaking.com/design_patterns/

https://refactoring.guru/design-patterns/python

https://en.wikipedia.org/wiki/Software_design_pattern

Picking a Pattern

Step 1

- Understand the problem you are facing in terms of dependencies, modularity and abstract concepts.

Step 2

- Identify if this is a behavioural, structural or creational issue?

Step 3

- Are there any constraints that I need to follow?

Step 4

- Is there a simpler solution that works? If not, pick a pattern.

Design Patterns - Advantages

- Don’t re-invent the wheel, use a proven solution instead
- Are abstract and can be applied to different problems
- Communicate ideas and concepts between developers
- Language agnostic. Can be applied to most (if not all) OOP programs.

Design Patterns - Disadvantages

- Can make the system more complex making the system harder to maintain. Patterns are deceptively ‘simple’.
- The system may suffer from pattern overload.
- All patterns have some disadvantages and add constraints to a system. As a result, a developer may need to add a constraint they did not plan for.
- Do not lead to direct code re-use.

What are Design Patterns

What's the best way to learn design patterns?

Which patterns to learn first?

Design pattern complexity

Low	Medium	High
1. Lazy Initialization 2. Façade 3. Singleton 4. Proxy	1. Bridge 2. Observer 3. Strategy 4. Builder 5. Mediator	1. Factory 2. Abstract Factory 3. State 4. Chain of Responsibility 5. Decorator

Categorizing Design Patterns

- **Behavioural**

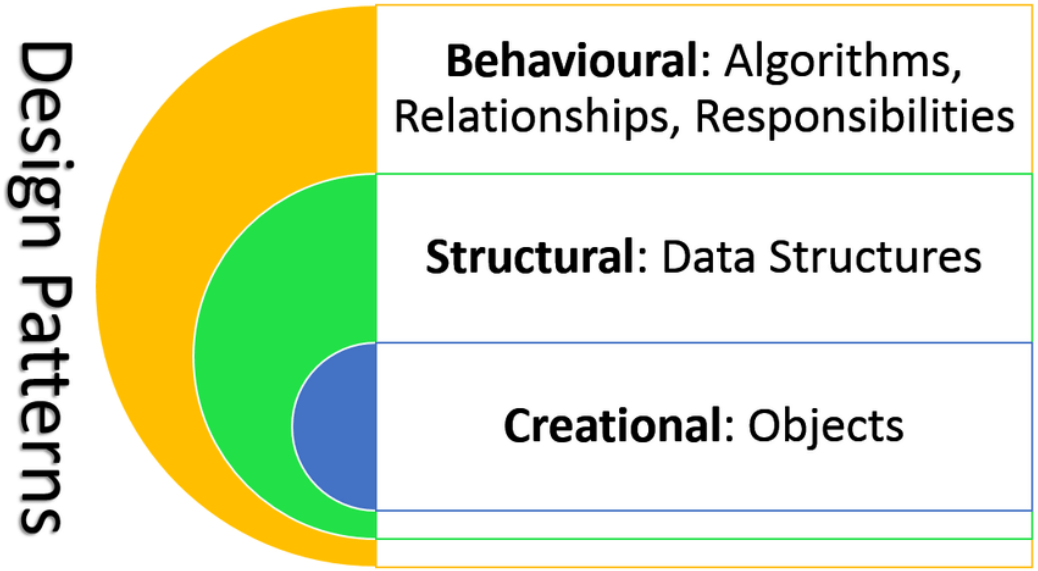
Focused on communication and interaction between objects. How do we get objects talking to each other while minimizing coupling?

- **Structural**

How do classes and objects combine to form structures in our programs? Focus on architecting to allow for maximum flexibility and maintainability.

- **Creational**

All about class instantiation. Different strategies and techniques to instantiate an object, or group of objects



Categorizing Design Patterns

- **Behavioural**

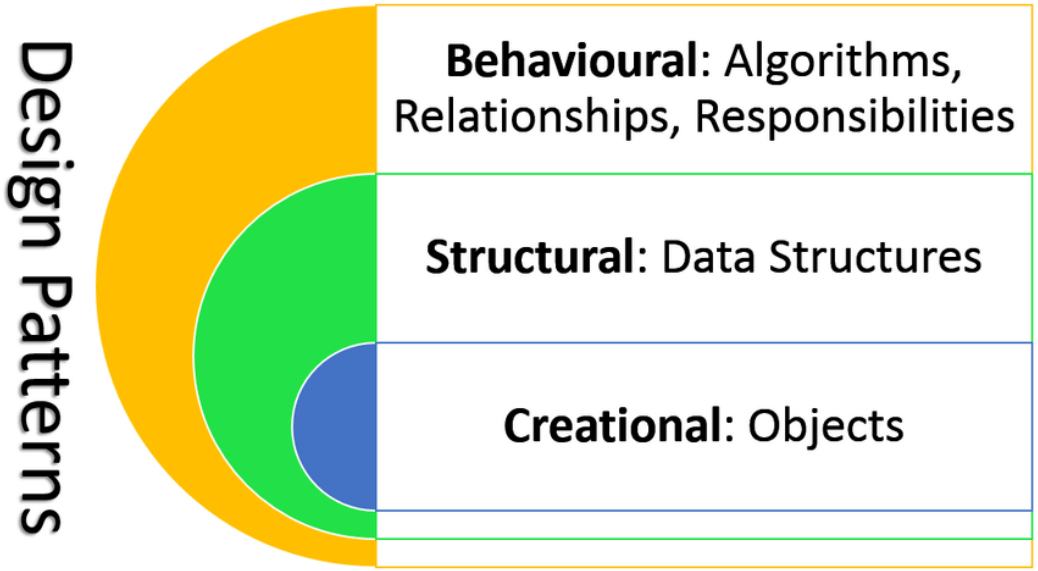
  Observer, Chain of Responsibility, Strategy, State, Mediator

- **Structural**

  Façade, Proxy, Bridge, Decorator

- **Creational**

  Singleton, Factory, Abstract Factory, Builder, Lazy Initialization

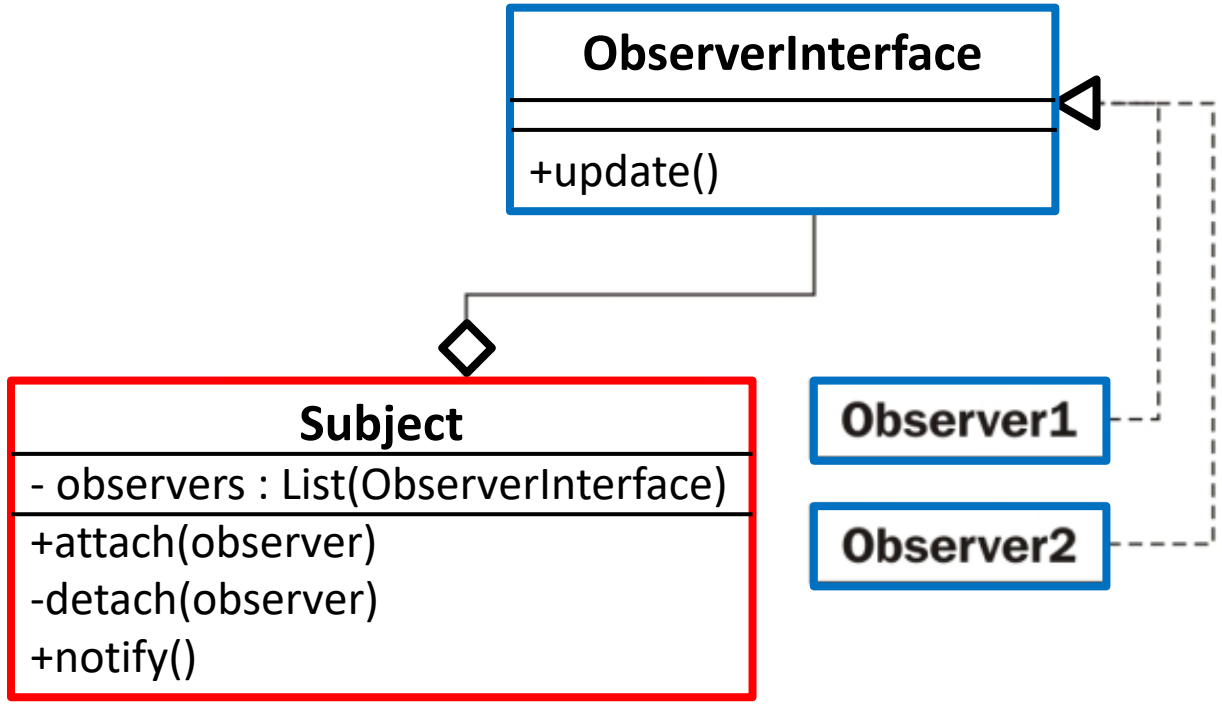


Behavioural - Observer

An object, called the **Subject**, maintains a list of its dependents, called **Observers**, and notifies them automatically of any state changes

Use this when: you need an **object** (**Subject**) to notify other **objects** (**Observers**) but limit dependencies. Core doesn't know the concrete types of observers

Example: A game wants to broadcast to many systems when the player has died. The **gameplay** object is the **Subject**, broadcasting information out to several **observers** (**game over screen**, **high score**, **save system**)



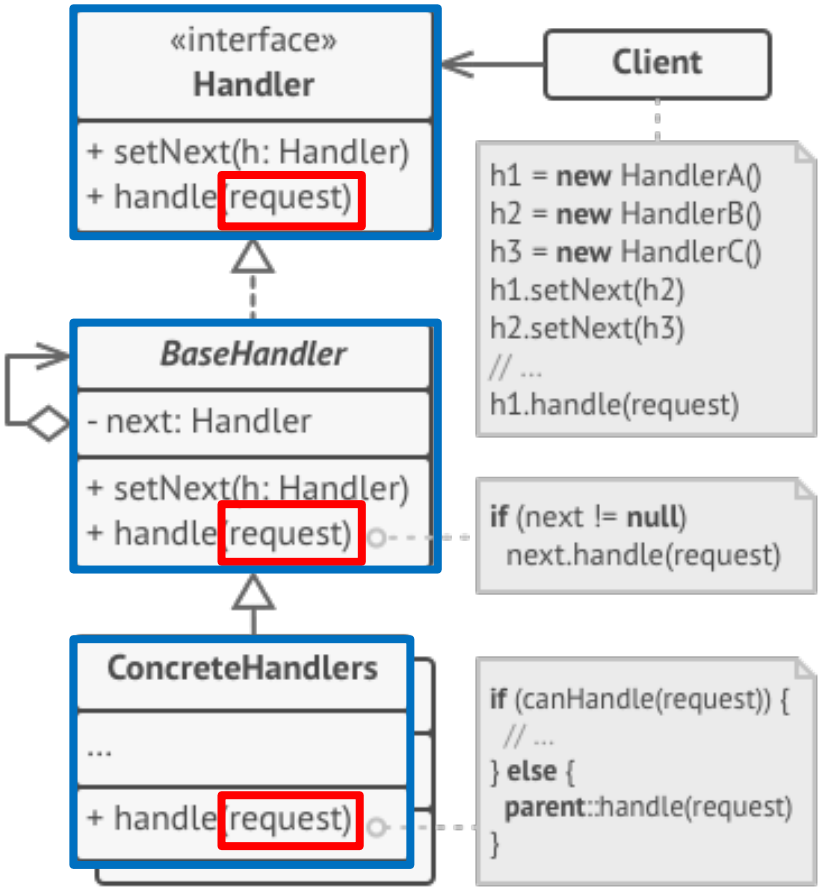
observer_pattern.py

Behavioural – Chain of responsibility

Use this when: a **request** needs to go through a **series of processing steps**. Decouples the sequencing of these steps and makes them re-usable

Example: A **document** needs to go through **several stages of validation**. Need to validate:

- **courses have correct name and year**
- **check fees paid**
- **student information correct**



chain_of_responsibility.py

Behavioural – Strategy

Use this when: an **object** needs to switch **behaviours** at runtime

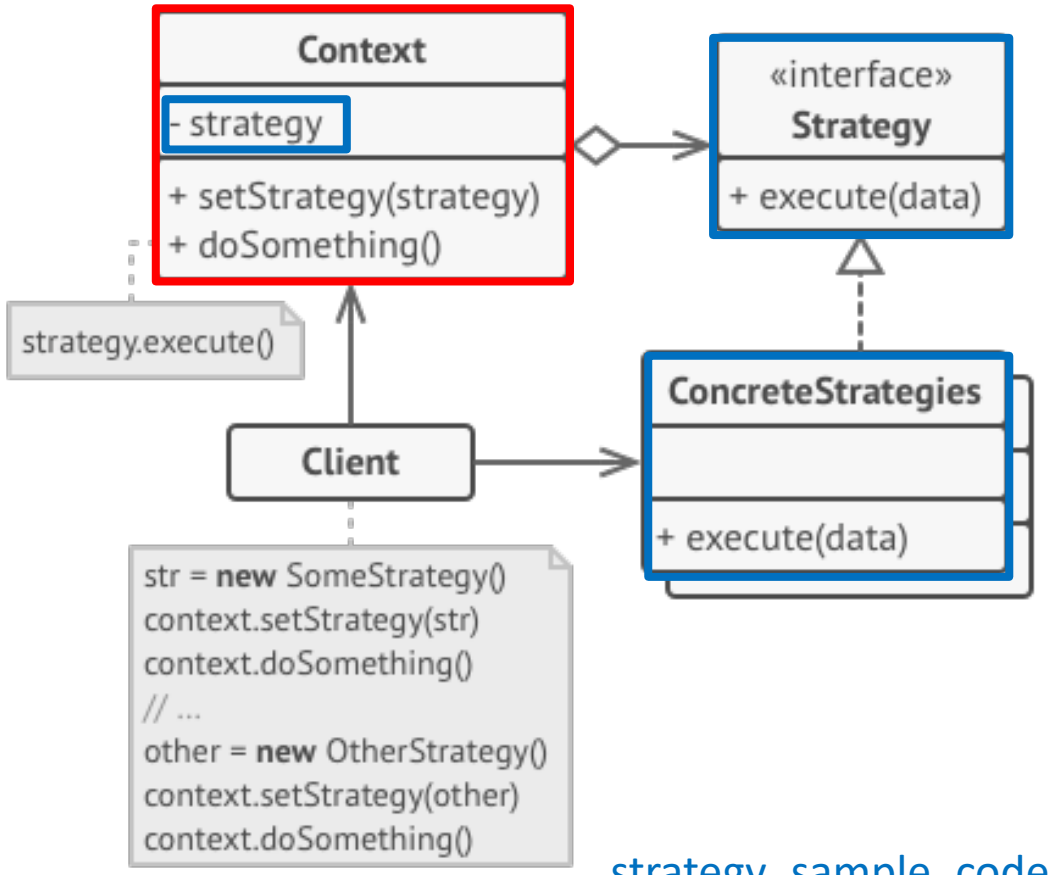
Example:

A game **character** wants to use different **weapons** when attacking

The character and weapon classes contain attack methods.

Change the character's attack behavior by swapping the weapon objects (axe, sword, hammer)

Character's attack method forwards calls to the weapon's attack method



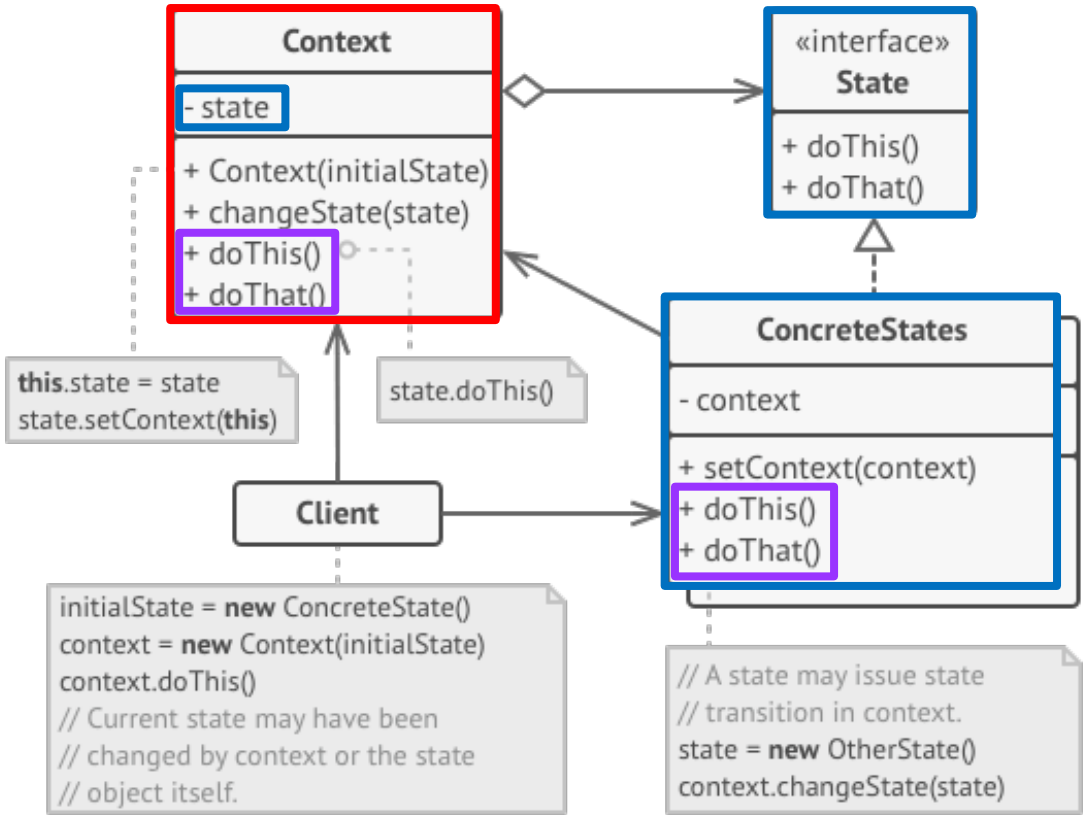
strategy_sample_code.py

Behavioural – State

When the same **object** can take on **different behaviours** depending on its **state**.

Having different specialized behaviours within the same class. The **context** changes their **state** by swapping out their internal state object. This causes all the **behaviours** in the context to use the new state object's behaviors

Use this when: you need almost all the behaviors of an object to change depending on its internal state

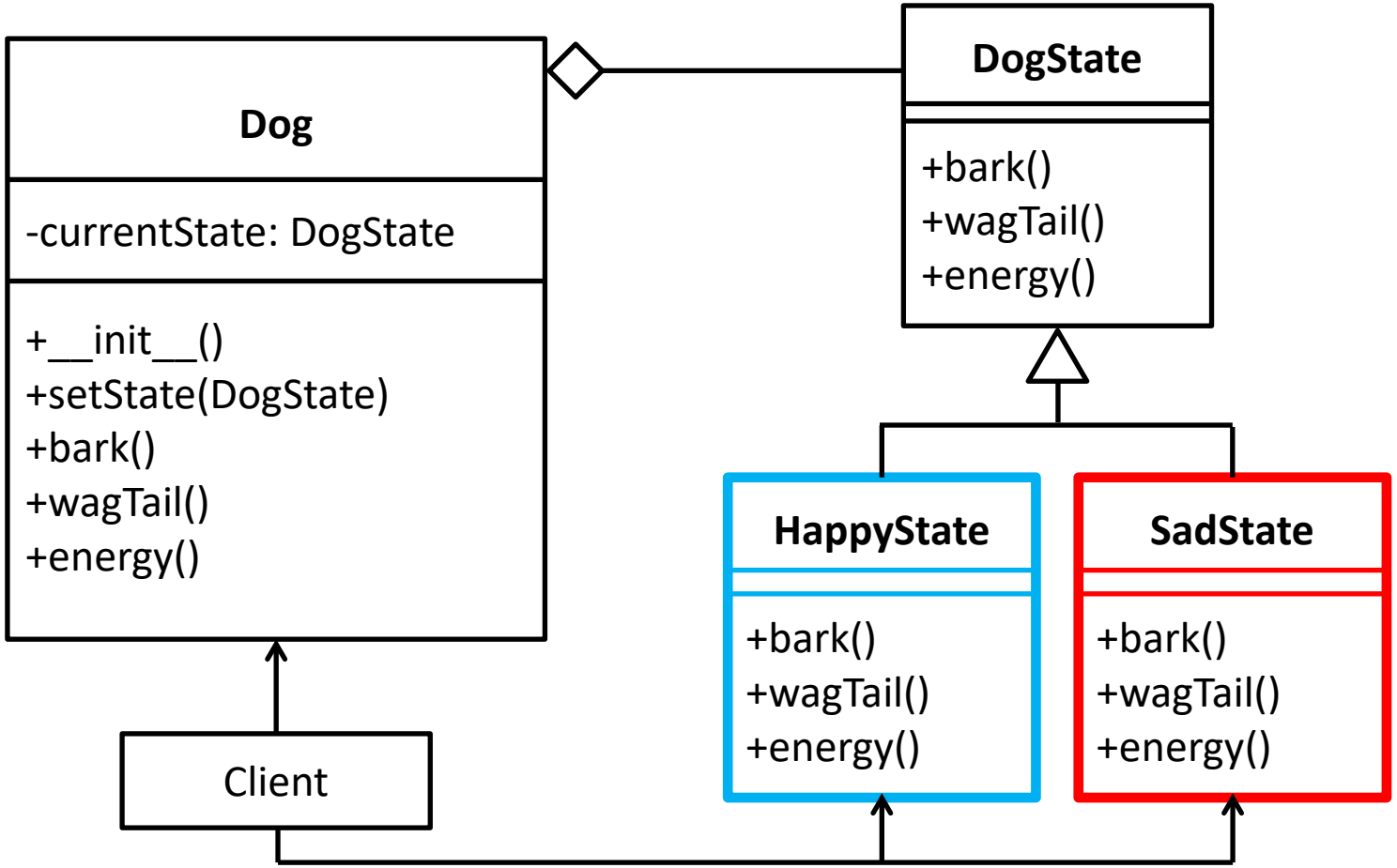


Behavioural – State

Create a system to simulate a dog behaving differently depending on its mood.

Happy – happily barking, wagging tail, high energy

Sad – whimpering, tail between legs, low energy



dog_state.py

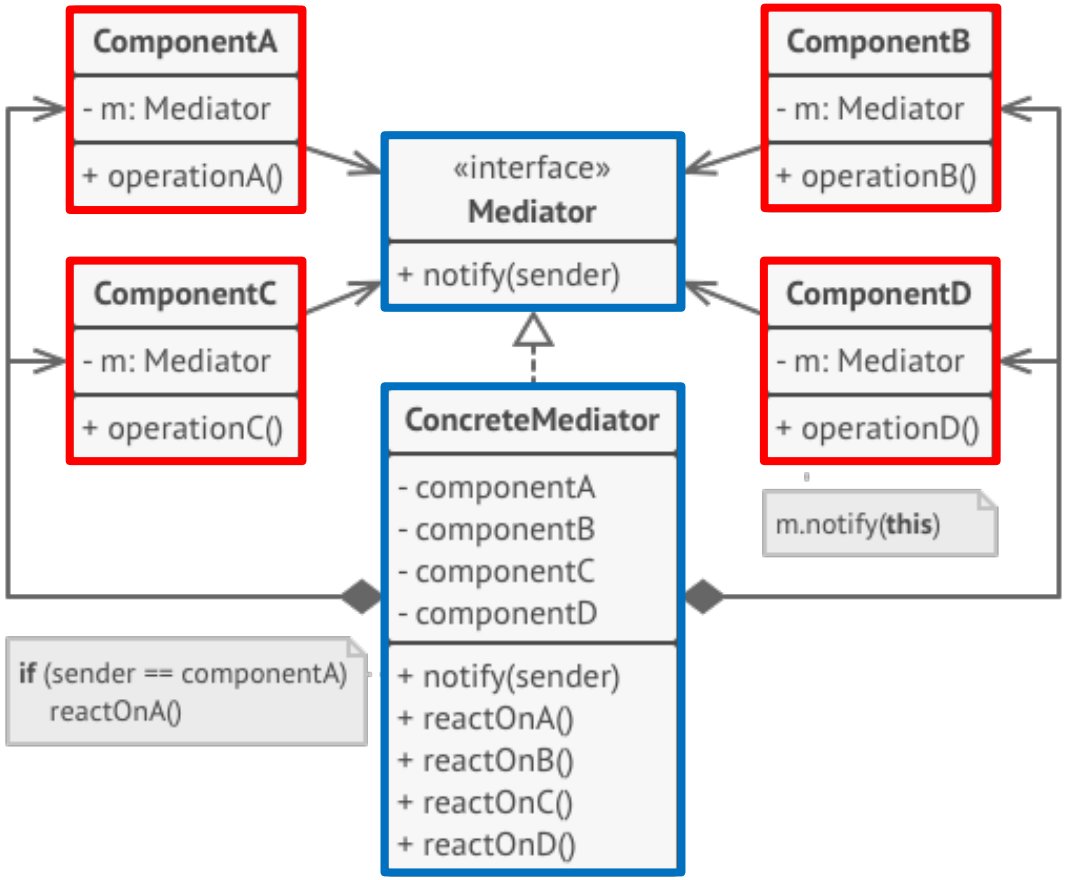
Behavioural – Mediator

Defines an **object** that encapsulates how a set of **other objects** interact

Components are unaware of other components, they only communicate with the mediator

When a **component** needs to send a message, they blindly notify **mediator**. Mediator object retrieves the notification and contains logic to call other components' behaviors

Use this when: you need to decouple **components** communicating with each other, and instead rely on a **central location** to handle communication



Behavioural – Mediator

Example: Model a traffic light system

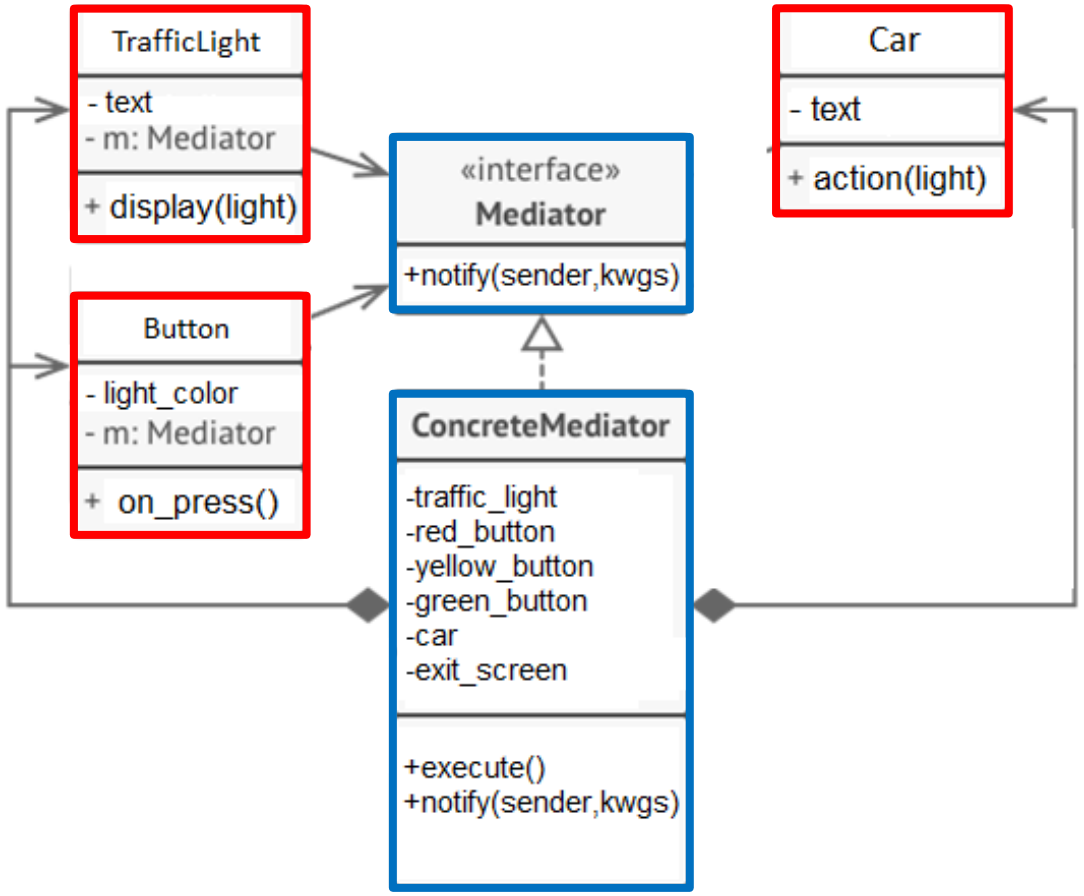
There are three **buttons**, red, yellow, green, that change the **traffic light**'s color when pressed

A **car** will:

- stop on a red light
- slow down on a yellow light
- go on a green light.

We need all these separate **components** to communicate with each other, but without coupling them

Have the **mediator** handle passing messages between all the components



mediator_sample_code.py

Categorizing Design Patterns

- **Behavioural**

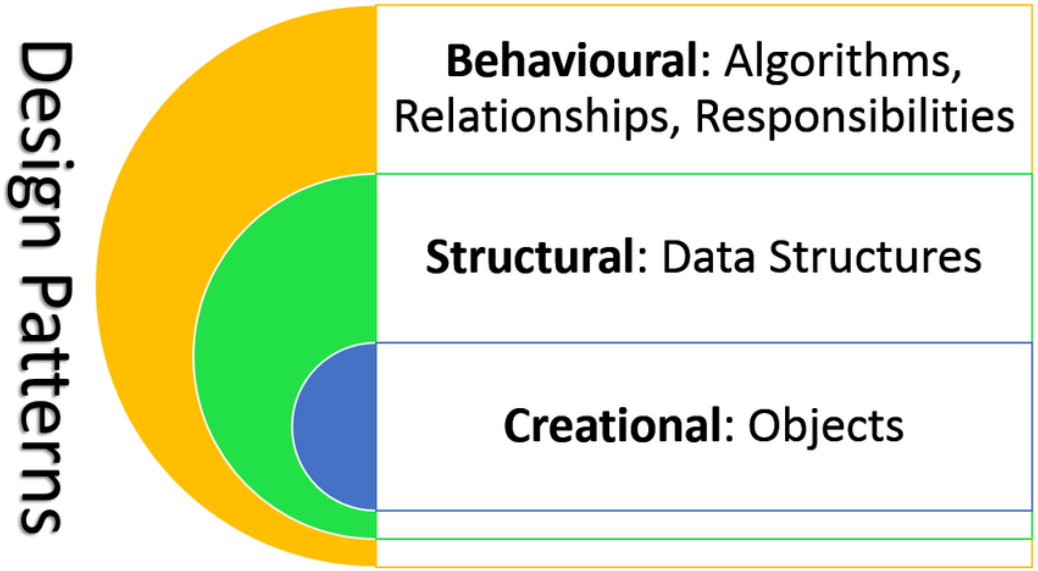
Observer, Chain of Responsibility, State, Strategy, Mediator

- **Structural**

Façade, Proxy, Bridge, Decorator

- **Creational**

Singleton, Factory, Abstract Factory, Builder, Lazy Initialization



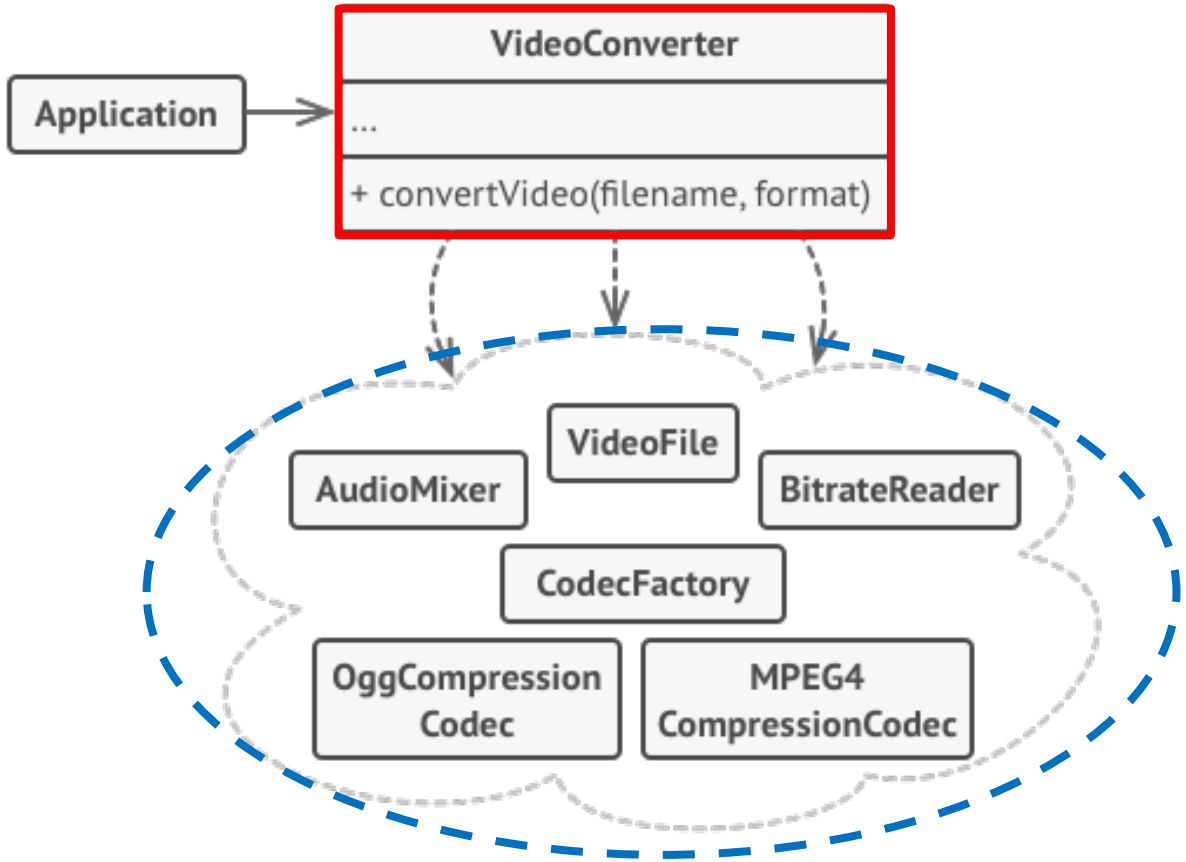
Structural – Façade

A **simplified interface** encapsulates a **complex system or API**. Client only interacts with the façade instead of complex systems

Use this when: you want to **hide the complexity of a system** from the client, but allow them to use it in a **simplified form**.

Example: **Video converter**. Only want to expose the convert video function to the client. But behind the scenes, there are **many systems** the facade must call to perform the action

simple_facade.py
video_package_facade_example.py



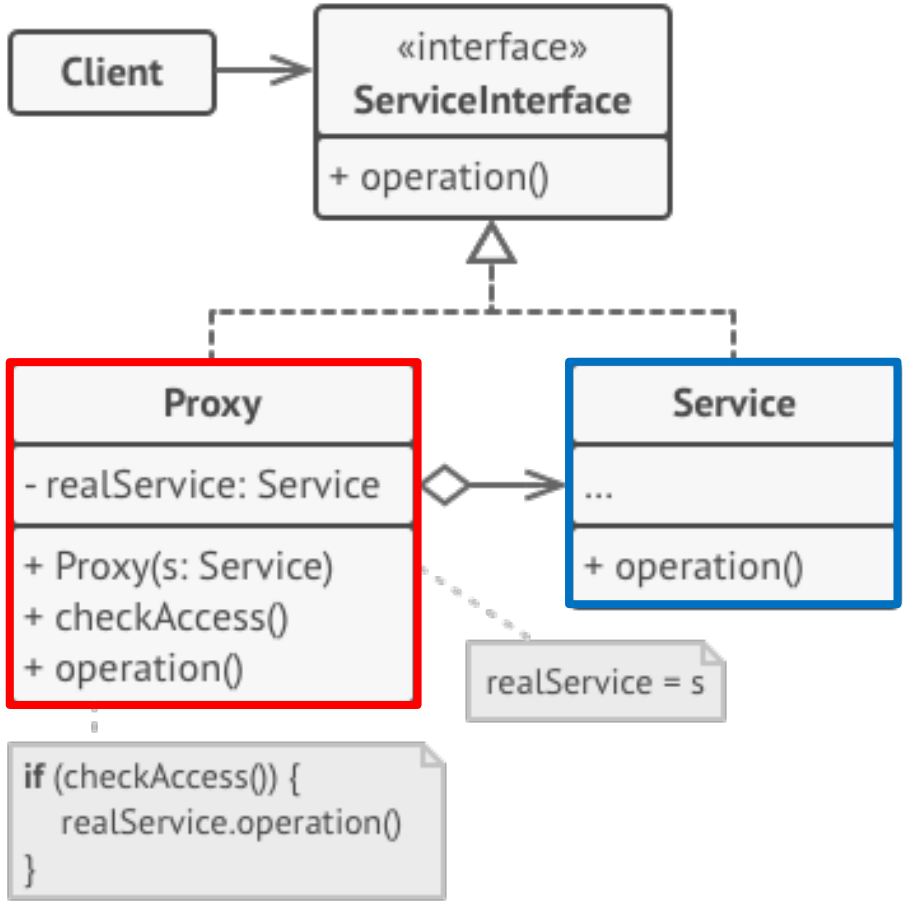
Structural – Proxy

A class functioning as an interface to something else. A **proxy is a wrapper** or agent object that is being called by the client to access the **real serving object** behind the scenes

Use this when: you have a **real service or object that you wish to hide the client** from directly interacting with.

Example: Youtube **database** is a real service. Prevent all users in the world to access one database. Create a **proxy object** that intercepts requests and forwards to Youtube database. Repeat requests are cached

simple_proxy.py
movie_database_proxy_example.py

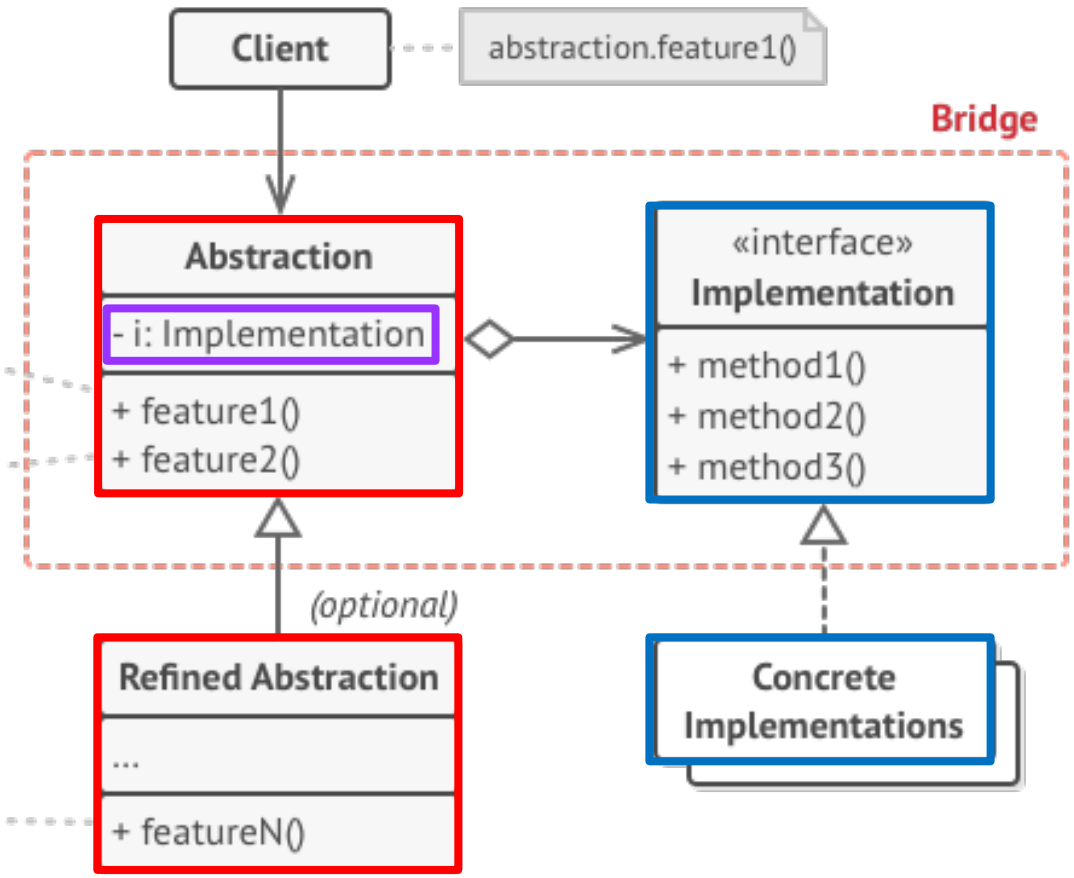


Structural – Bridge

The bridge pattern allows the **Abstraction** and the **Implementation** to be developed independently and the client code can access only the Abstraction part without being concerned about the Implementation part

Use this when: you want to **break down a large class into smaller components**. These components are connected to the main abstraction class via an **instance** of the implementation

Example: Creating a TV **remote** class that controls a **Device**



Structural – Bridge

Example: Creating a TV remote class that controls a Device

Remote is the "**abstraction**" it can control any kind of device

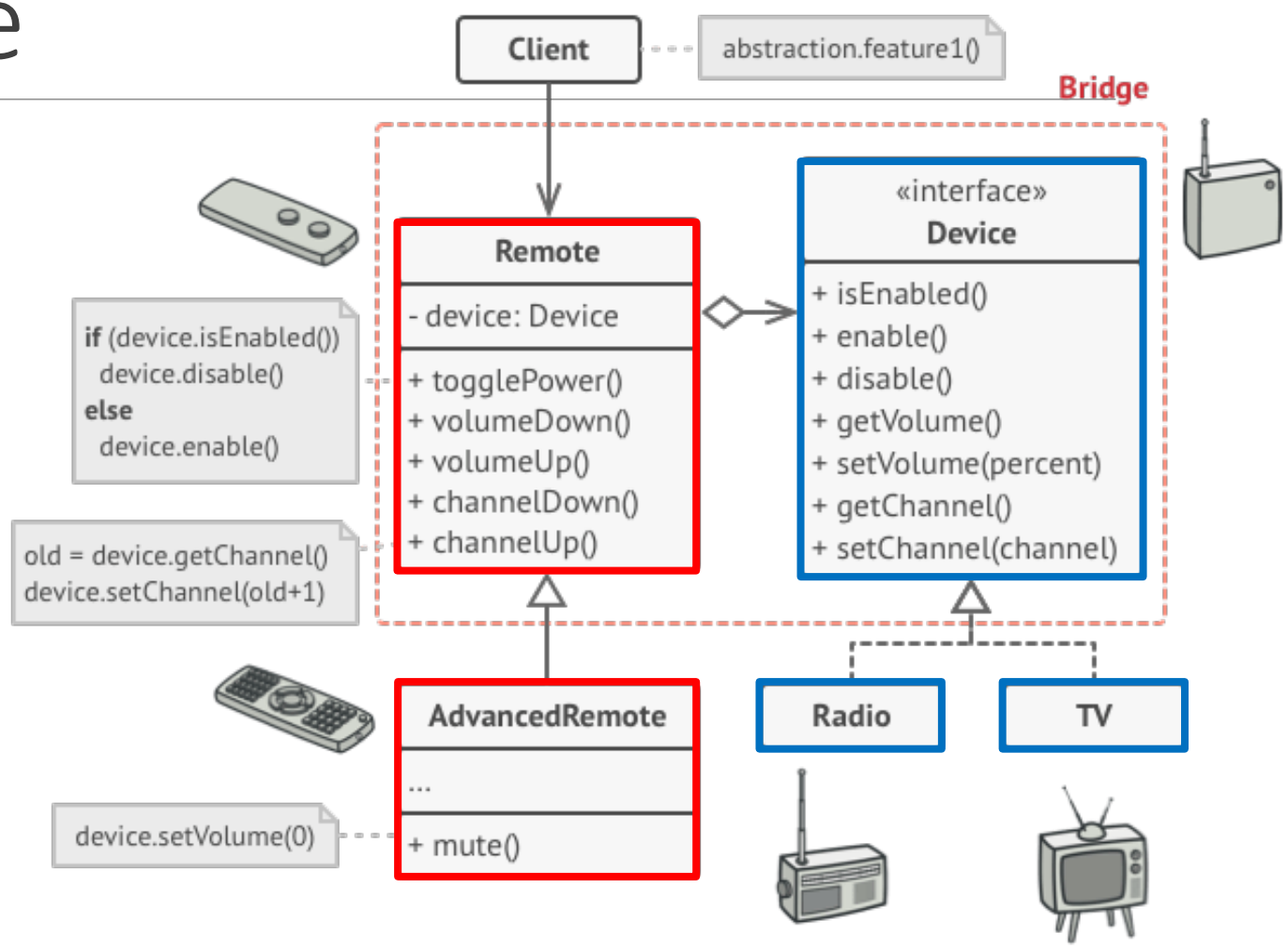
The **device** is the "**implementation**"

This way a remote can control a radio or tv

- Functions in remote simply call functions in device

The advanced remote has the extra functionality of muting any device

simple_bridge_example.py
device_remote_bridge_example.py***

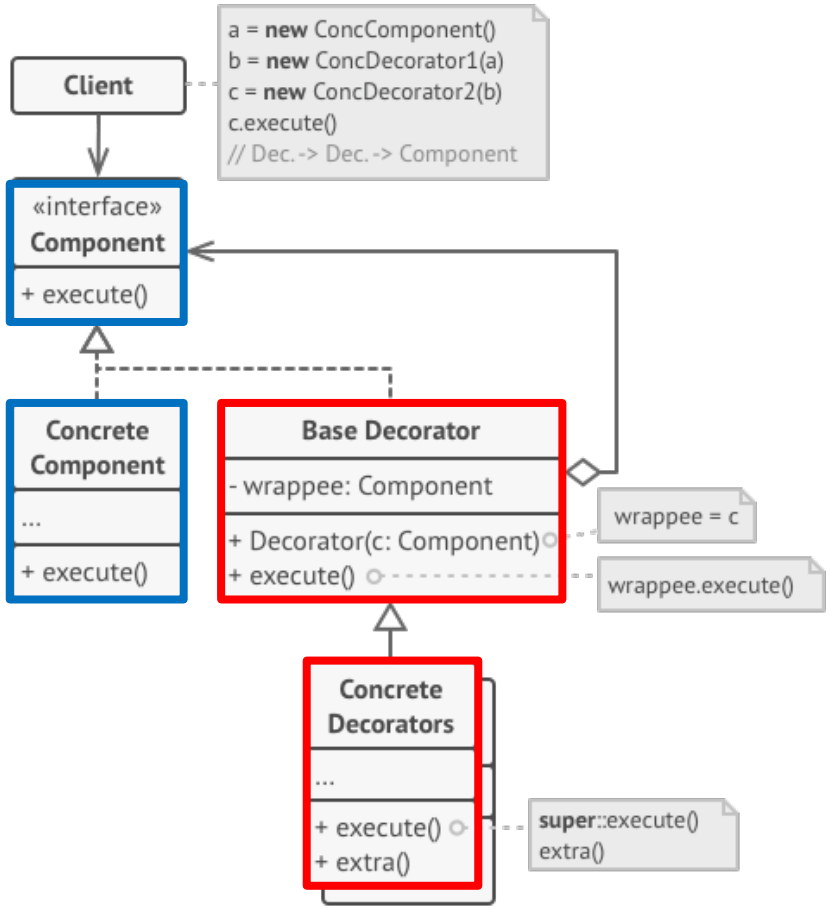


Structural – Decorator

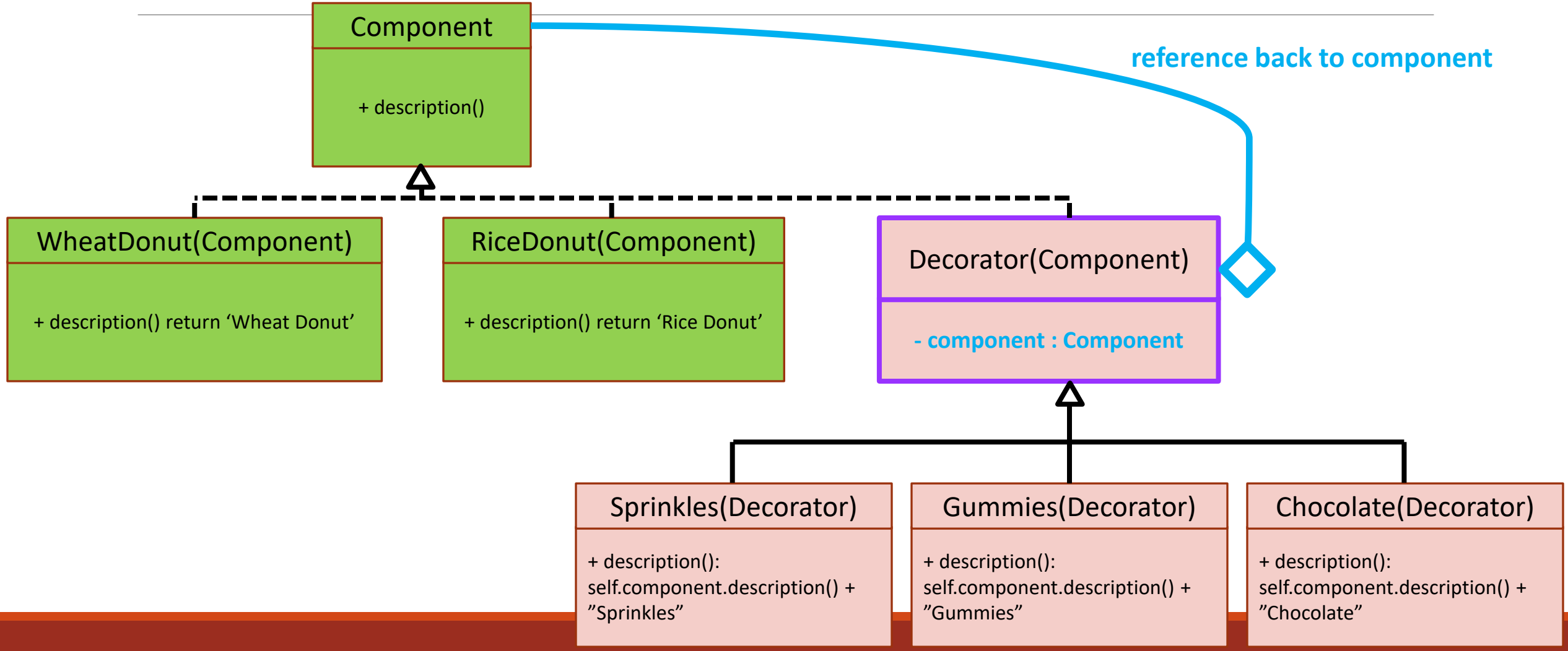
Allows **behavior** to be added to an individual **object**, dynamically, without affecting the behavior of other objects from the same class

Use this when: you want to **dynamically add behavior to an object** without going back to re-write the object

Example: **Donut object** where we can add any number and type of **toppings** to it

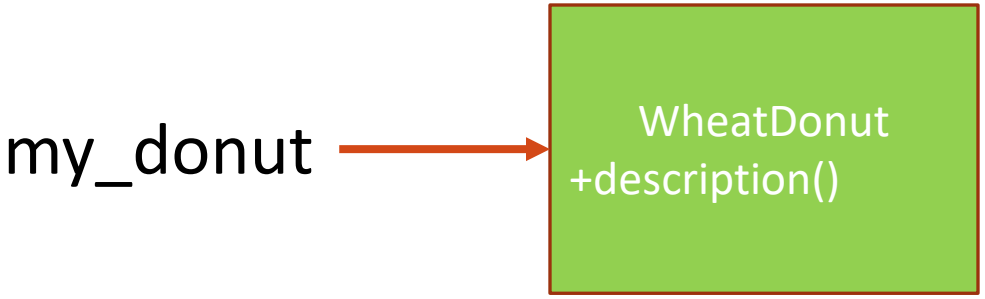


Donuts…


Component
+ description()
reference back to component
WheatDonut(Component)
+ description() return 'Wheat Donut'
RiceDonut(Component)
+ description() return 'Rice Donut'
Decorator(Component)
- component : Component
Sprinkles(Decorator)
+ description():
self.component.description() +
"Sprinkles"
Gummies(Decorator)
+ description():
self.component.description() +
"Gummies"
Chocolate(Decorator)
+ description():
self.component.description() +
"Chocolate"

Introduction - Decorator

- my_donut = **WheatDonut**()



- Create **WheatDonut** component

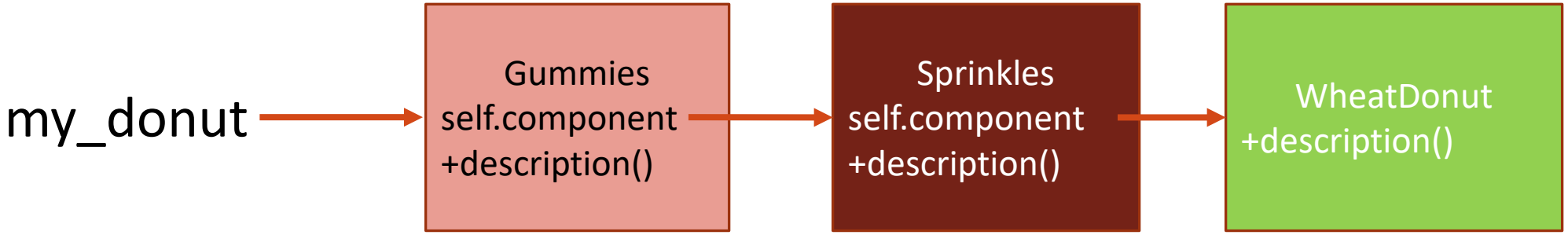
Introduction - Decorator

- my_donut = **WheatDonut**()
- my_donut = **Sprinkles**(my_donut)

- Create **Sprinkles** object, set its self.component to point to my_donut
- **Sprinkles'** grandparent base class is Component, so the reference it returns is saved into my_donut

Introduction - Decorator

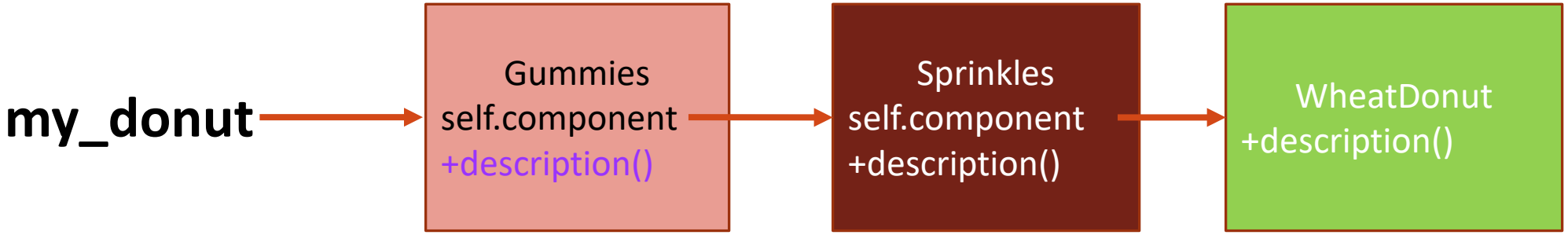
- my_donut = **WheatDonut**()
- my_donut = **Sprinkles**(my_donut)
- my_donut = **Gummies**(my_donut)



- Create **Gummies** object, set its self.component to point to my_donut
- **Gummies**' grandparent base class is Component, so the reference it returns is saved into my_donut

Introduction - Decorator

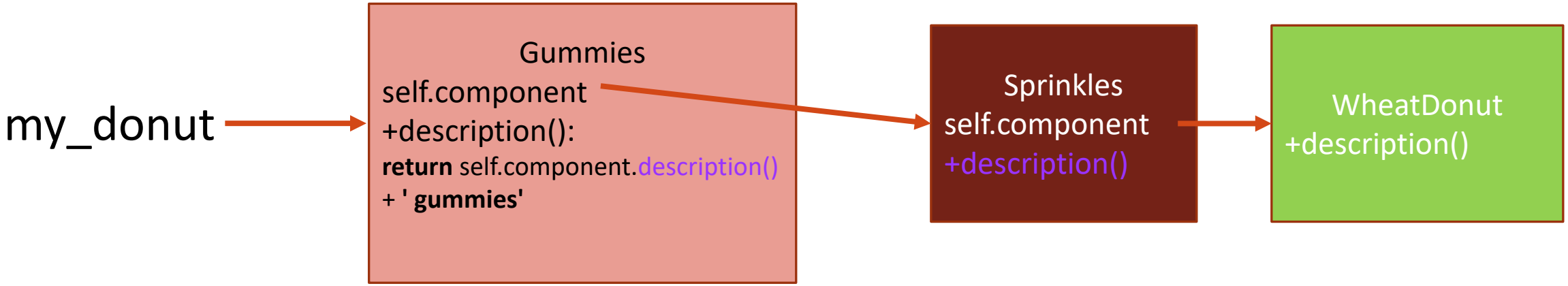
- my_donut = **WheatDonut**()
- my_donut = **Sprinkles**(my_donut)
- my_donut = **Gummies**(my_donut)
- print(**my_donut**.**description**())



- Notice how it looks like a **linked list** starting from **my_donut**
- Get total description of the donut by calling **my_donut**.description();

Introduction - Decorator

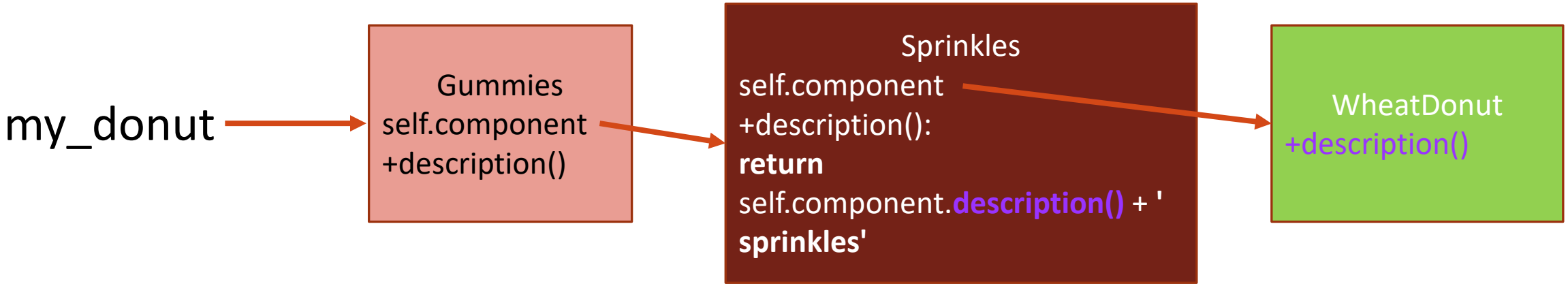
- my_donut = **WheatDonut**()
- my_donut = **Sprinkles**(my_donut)
- my_donut = **Gummies**(my_donut)
- print(my_donut.**description**())



- Calls to description function will chain into toppings until they reach **WheatDonut**
- Gummies' description function calls Sprinkles' description function

Introduction - Decorator

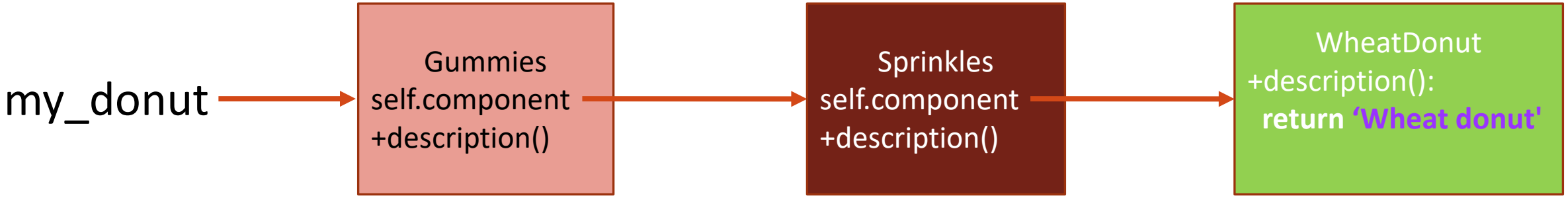
- my_donut = **WheatDonut**()
- my_donut = **Sprinkles**(my_donut)
- my_donut = **Gummies**(my_donut)
- print(my_donut.description())



- Sprinkles' description function calls **WheatDonut's** description function

Introduction - Decorator

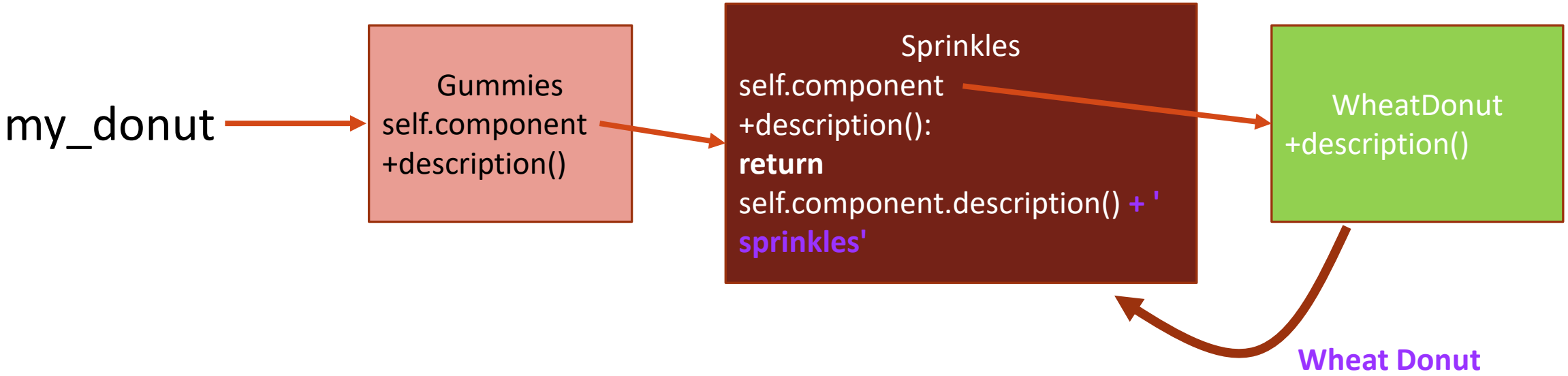
- my_donut = **WheatDonut**()
- my_donut = **Sprinkles**(my_donut)
- my_donut = **Gummies**(my_donut)
- print(my_donut.description())



- String is appended as we return from **WheatDonut's** description() function calls
- **WheatDonut** **description**() function returns "**Wheat donut**"

Introduction - Decorator

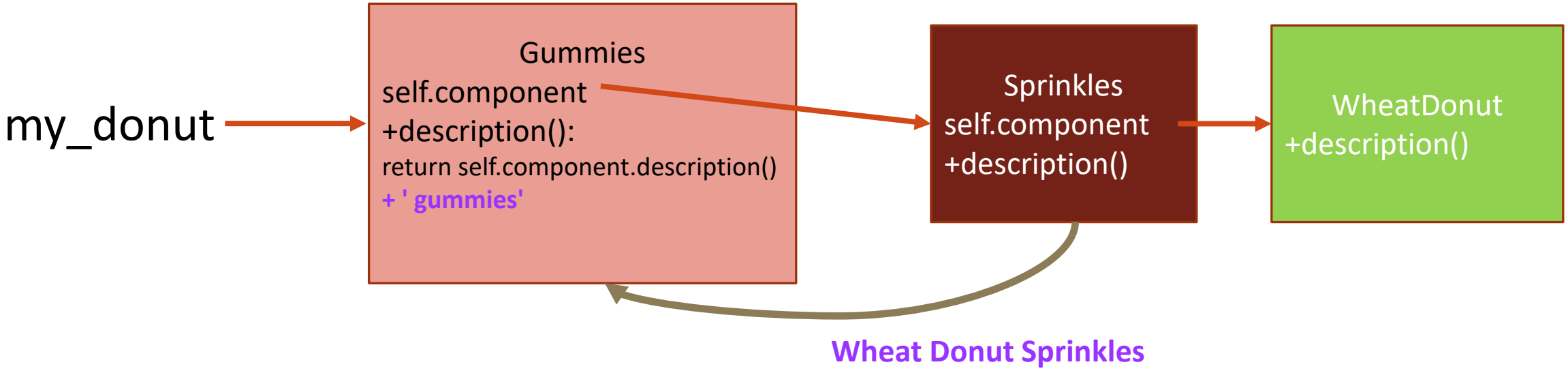
- my_donut = **WheatDonut**()
- my_donut = **Sprinkles**(my_donut)
- my_donut = **Gummies**(my_donut)
- print(my_donut.description())



- "**sprinkles**" appended to returned "**Wheat donut**". This string also returned

Introduction - Decorator

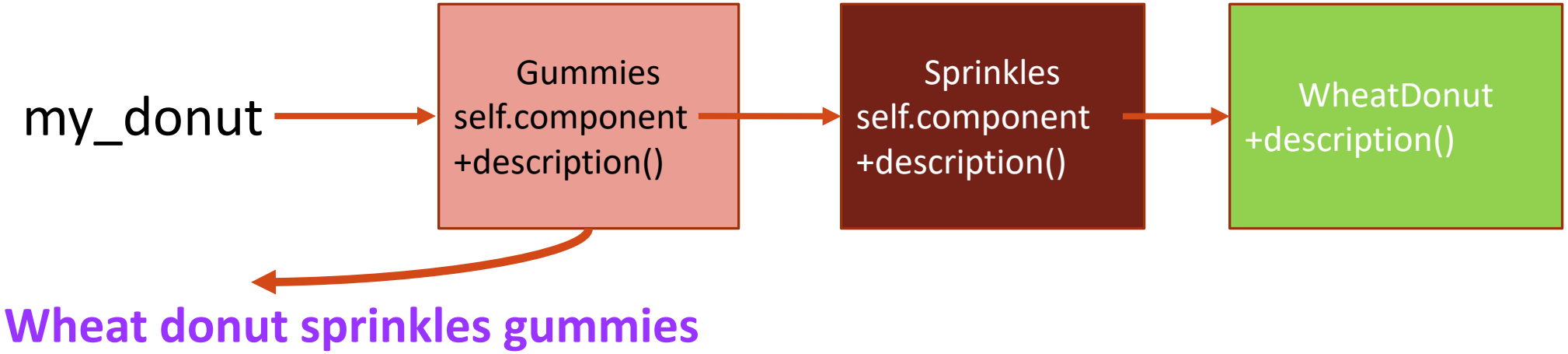
- my_donut = **WheatDonut**()
- my_donut = **Sprinkles**(my_donut)
- my_donut = **Gummies**(my_donut)
- print(my_donut.description())



- "**gummies**" appended to returned "**Wheat donut sprinkles**". This string also returned

Introduction - Decorator

- my_donut = **WheatDonut**()
- my_donut = **Sprinkles**(my_donut)
- my_donut = **Gummies**(my_donut)
- print(my_donut.description())



- Final "**Wheat donut sprinkles gummies**" returned to print out

simple_decorator.py***

Categorizing Design Patterns

- **Behavioural**

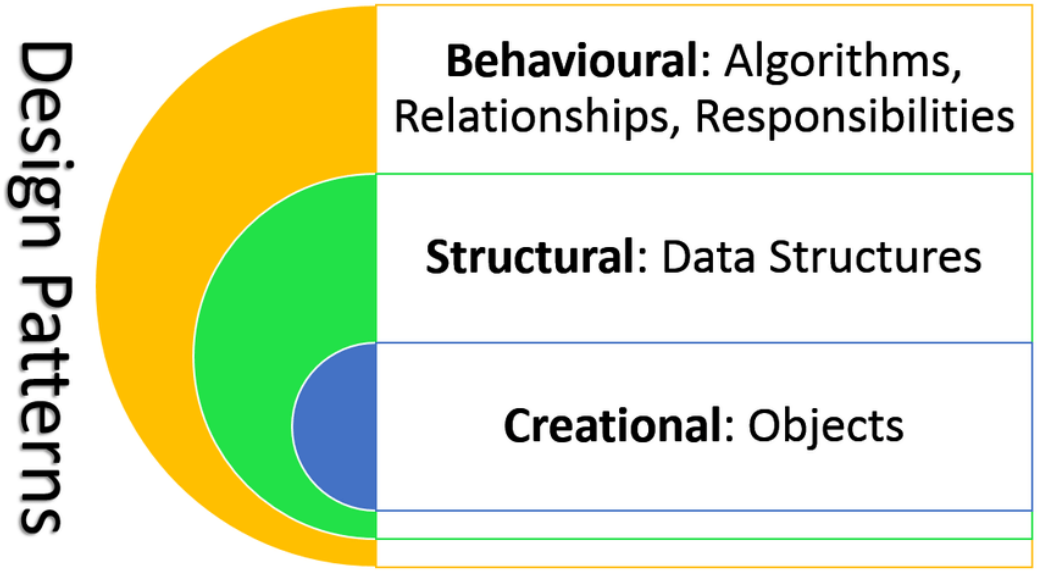
Observer, Chain of Responsibility, State, Strategy, Mediator

- **Structural**

Façade, Proxy, Bridge, Decorator

- **Creational**

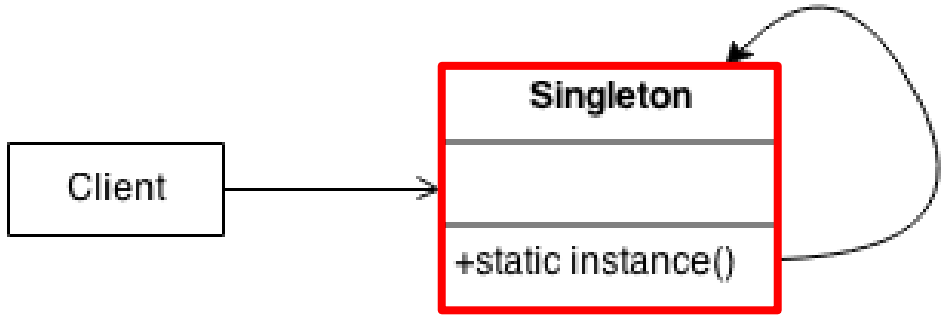
Singleton, Factory, Abstract Factory, Builder, Lazy Initialization



Creational - Singleton

Ensure a class has only one instance and provide a global point of access to it.

Use this when: an application needs **one and only one instance of an object that is globally accessible**



Example: Save system in a game. The **save system** can be accessed by any screen, controller, object in the game.

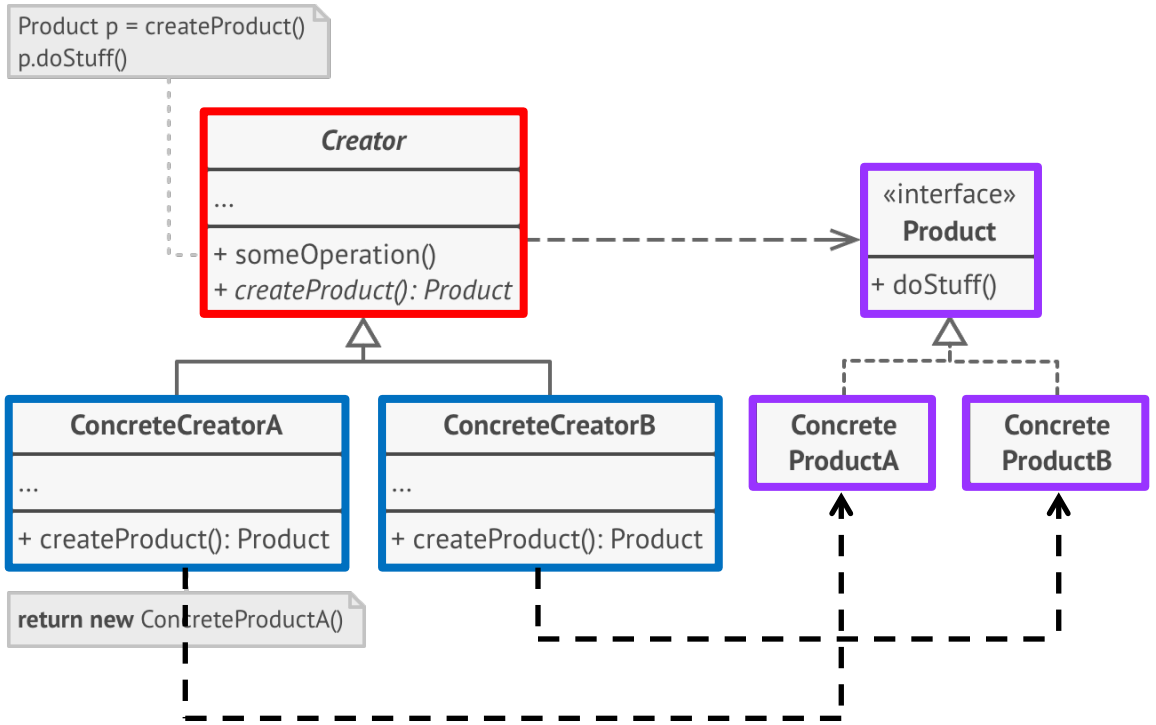
singleton_example.py
singleton_example_screens.py

Creational - Factory

When you want to separate creation code from client code.

Use this when: you want a **client or service to only depend on a base class for object creation and not each concrete class.** In other words, the client depends on a base factory for object creation, instead of each individual concrete factory

Example: Forum that needs access to different types of **users** (**Guest**, **member**). **Factory** can produce both types and the forum uses them without knowing the **concrete type**



forum_user_factory.py

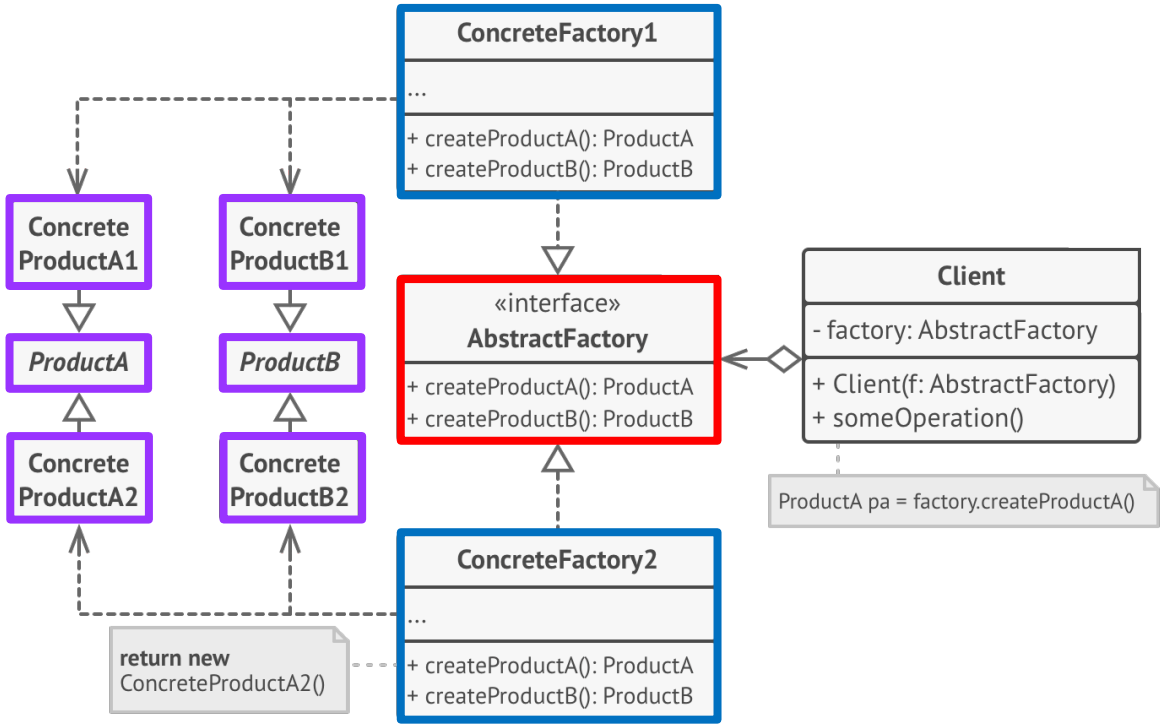
Creational – Abstract Factory

A more evolved form of the Factory Pattern

Allows for the creation of a group of related objects .

Use this when: you want to create **different variants of families of objects.** Client also relies on **base factory** for creation instead of each **individual concrete factory**

Example: Client wants to order different themes of **furniture**. Client gets passed the **appropriate Factory,** which creates a set of **themed furniture**.



game_abstract_factory.py

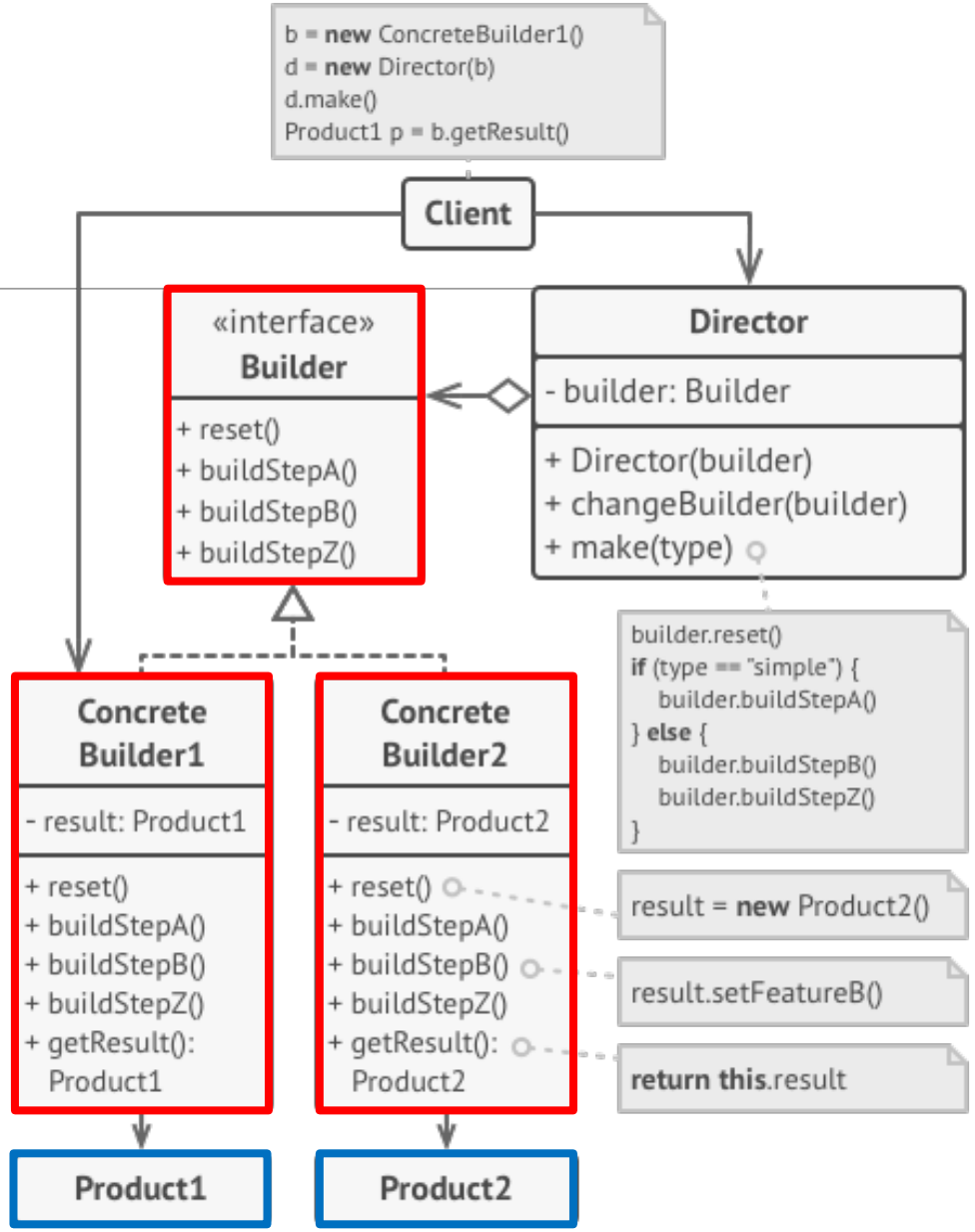
Creational – Builder

Designed to provide a flexible solution to various object creation problems in object-oriented programming.

Use this when: you want to **separate the construction** of a complex object from its **representation**. The code typically written in the initializer of an object is moved into a separate class

Example: Want to create different types of pizza, but the client doesn't want to manually specify all ingredients. They want to order from a menu. The menu tells the **builder** to build a very specific type of **pizza**, and returns that to the client

builder_sample_code.py***



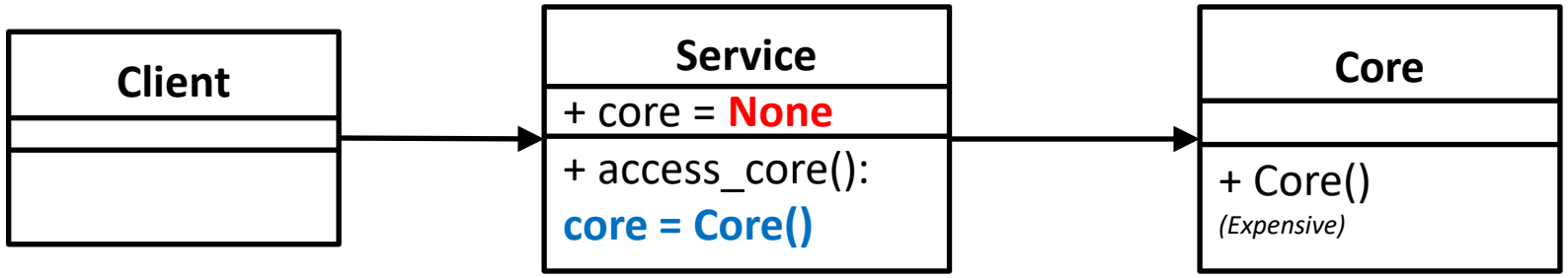
Creational – Lazy initialization

Initialization of an object occurs only when it's needed and not before to preserve simplicity of usage and improve performance.

Use this when: you want to **delay using a resource** **until the last possible moment**. Saving time and resources if the resource is never used

Example: Screen in a program that isn't instantiated until it's actually needed.

Lazy initialization often used in the Singleton pattern



lazy_initialization_sample_code.py***

What I've used in practice

"What is the most commonly used design pattern that you see at work or wish students would utilize more.."

1. Singleton – My favorite for managers/controllers in games. Whenever I need **one central object to control a large overarching task** in the system. SoundManager, Save/Load manager, ScreenManager
2. Observer – Great for having an **object communicate with many objects** when something important happens
3. Lazy initialization – Used as part of singleton and whenever I want to **save performance** and **not initializing everything at once**
4. Creation method – Have a static class who's **only job is to create specific objects**
5. Bridge – Probably used it unknowingly when **creating classes through composition**

Design patterns scenario activity

Scenario

You are writing code to model different kinds of offices. Offices can have multiple conference rooms, bathrooms, cubicles, and executive offices

There may even be duplex offices (offices with a second storey). You need to come up with a way to instantiate all these different kinds of Offices while still allowing for customizability

What design pattern would be most appropriate to address this issue?

There's a **lot of design patterns**, it's **hard to choose**. Let's break it down into several steps

Design patterns scenario activity

Scenario

You are writing code to ***model different kinds of offices****. Offices can have* ***multiple conference rooms****,* ***bathrooms****,* ***cubicles****, and* ***executive offices***

There may even be ***duplex offices*** *(offices with a second storey). You need to come up with a way to* ***instantiate*** *all these different kinds of Offices while still allowing for* ***customizability***

Step 1 – Understand the problem

Appears to be a problem related with **creating** something with **multiple optional components** in a **customizable** way

Design patterns scenario activity

Scenario

You are writing code to model different kinds of offices. Offices can have multiple conference rooms, bathrooms, cubicles, and executive offices

There may even be duplex offices (offices with a second storey). You need tocome up with a way to instantiate all these different kinds of Offices while still allowing for customizability

Step 2 – What broad category of design patterns does this fall under?

1. Behavioral – how objects in system communicate with low coupling (Communication)
2. **Structural – how objects/classes combine to create new structures (Structure/Form)**
3. **Creational – how to instantiate objects or groups of objects in new ways (Instantiation)**

Design patterns scenario activity

Scenario

You are writing code to model different kinds of offices. Offices can have multiple conference rooms, bathrooms, cubicles, and executive offices

There may even be duplex offices (offices with a second storey). You need tocome up with a way to instantiate all these different kinds of Offices while still allowing for customizability

Step 3 – Examine each pattern to see which most closely matches our scenario

- **Structural**

Decorator, Façade, Proxy, Bridge

- **Creational**

Singleton, Factory, Abstract Factory, Builder, Lazy Initialization

Design patterns scenario activity

Scenario

You are writing code to model different kinds of offices. Offices can have multiple conference rooms, bathrooms, cubicles, and executive offices

There may even be duplex offices (offices with a second storey). You need tocome up with a way to instantiate all these different kinds of Offices while still allowing for customizability

Step 3 – Examine each pattern to see which most closely matches our scenario

- **Structural**

Decorator, Façade, Proxy, Bridge

All of these appear to be wrappers, which isn't exactly what we want.

Decorator is close because it adds **behaviors** to a central object. But we don't want to add **behaviors**. We want to add optional **properties or attributes** to an existing object.

Design patterns scenario activity

Scenario

You are writing code to model different kinds of offices. Offices can have multiple conference rooms, bathrooms, cubicles, and executive offices

There may even be duplex offices (offices with a second storey). You need to come up with a way to instantiate all these different kinds of Offices while still allowing for customizability

Step 3 – Examine each pattern to see which most closely matches our scenario

- ❑ **Creational**

Pattern	Description
Singleton	instantiate global object
Factory	instantiate different types of objects
Abstract Factory	instantiate families of objects
Builder	instantiate customizable objects
Lazy Initialization	instantiate object when needed

Design patterns scenario activity

Scenario

You are writing code to model different kinds of offices. Offices can have multiple conference rooms, bathrooms, cubicles, and executive offices

There may even be duplex offices (offices with a second storey). You need to come up with a way to instantiate all these different kinds of Offices while still allowing for customizability

Step 3 – Examine each pattern to see which most closely matches our scenario

- **Creational**

Pattern	Description
Singleton	instantiate global object
Factory	instantiate different types of objects
Abstract Factory	instantiate families of objects
Builder	**instantiate customizable objects**
Lazy Initialization	instantiate object when needed

Design patterns scenario activity

Learning Hub > Activities > Quizzes

1. Purpose is to choose the design pattern that will best solve the scenario that's mentioned
2. Work in pairs to fill out the survey
3. We'll discuss afterwards
4. Design patterns cheat sheet